6

The resultant and gcd computation

We start this chapter with a typical example illustrating the growth of coefficients in the Euclidean Algorithm for polynomials over $\mathbb{Q}$. Much of the rest of this chapter is devoted to getting a handle on this growth. As an application, we obtain modular algorithms for the gcd in $\mathbb{Q}[x]$ and $F[x,y]$ for a field F; these are much more efficient than the direct computation.

Gauß' lemma in Section 6.2 illuminates the non-obvious relation between gcds of integer polynomials in $\mathbb{Z}[x]$ and $\mathbb{Q}[x]$. We then introduce the **resultant**, which gives control over the **Bézout coefficients** s and t in the presentation $sf+tg = \gcd(f,g)$. This yields a modular gcd calculation for bivariate polynomials, and, together with **Mignotte's factor bound**, also for integer polynomials. Section 6.10 discusses the more general **subresultants**, which govern the coefficient growth in the whole Extended Euclidean Algorithm, and provide a modular approach to the EEA.

In between, we digress to two applications: computing the intersection points of two plane algebraic curves, and an unexpectedly efficient way of computing the gcd of many polynomials.

6.1. Coefficient growth in the Euclidean Algorithm

Let F be a field, and $f,g \in F[x]$ with $\deg f = n \geq \deg g = m \geq 0$. We fix the notation from Section 3.4 of the results of the Extended Euclidean Algorithm for f and g:

$$\begin{array}{rcrcrc}
\rho_0 r_0 &=& f, & \rho_0 s_0 &=& 1, \quad \rho_0 t_0 = 0, \\
\rho_1 r_1 &=& g, & \rho_1 s_1 &=& 0, \quad \rho_1 t_1 = 1, \\
\rho_2 r_2 &=& r_0 - q_1 r_1, & \rho_2 s_2 &=& s_0 - q_1 s_1, \quad \rho_2 t_2 = t_0 - q_1 t_1, \\
&\vdots & & \vdots & & \vdots \\
\rho_{i+1} r_{i+1} &=& r_{i-1} - q_i r_i, & \rho_{i+1} s_{i+1} &=& s_{i-1} - q_i s_i, \quad \rho_{i+1} t_{i+1} = t_{i-1} - q_i t_i, \\
&\vdots & & \vdots & & \vdots \\
0 &=& r_{\ell-1} - q_\ell r_\ell, & s_{\ell+1} &=& s_{\ell-1} - q_\ell s_\ell, \quad t_{\ell+1} = t_{\ell-1} - q_\ell t_\ell,
\end{array} \tag{1}$$

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

with $\deg r_{i+1} < \deg r_i$ for all $i \geq 1$. Thus $r_{i-1} = q_i r_i + \rho_{i+1} r_{i+1}$ is the division of r_{i-1} by r_i with remainder $\rho_{i+1} r_{i+1}$; the leading coefficient ρ_{i+1} serves to have a normalized remainder r_{i+1}. A basic invariant is $r_i = s_i f + t_i g$. We define the **degree sequence** $(n_0, n_1, \ldots, n_\ell)$ by $n_i = \deg r_i$ for all i. Then

$$n = n_0 \geq n_1 > n_2 \cdots > n_\ell \geq 0.$$

It is convenient to set $\rho_{\ell+1} = 1$, $r_{\ell+1} = 0$, and $n_{\ell+1} = -\infty$. The number of arithmetic operations in F performed by the (Extended) Euclidean Algorithm for f and g is $O(nm)$ (Theorem 3.16).

In order to get a bound on the number of word operations of Euclid's algorithm over $F = \mathbb{Q}$, we need to get a bound on the length of the numbers involved in the computation. We extend the definition of the **length** of an integer from Section 2.1 to rational numbers and polynomials with rational coefficients. We bring all coefficients of a polynomial $a \in \mathbb{Q}[x]$ to a common denominator, and then $\lambda(a)$ is the maximal number of words required to encode the denominator or a coefficient of the numerator of a. More precisely, we use

- $\lambda(a) = \lfloor (\log_2 |a|)/64 \rfloor + 1$, when $a \in \mathbb{Z} \setminus \{0\}$, and $\lambda(0) = 0$,
- $\lambda(a) = \max\{\lambda(b), \lambda(c)\}$, if $a = b/c \in \mathbb{Q} \setminus \{0\}$ with $b, c \in \mathbb{Z}$ and $\gcd(b, c) = 1$,
- $\lambda(a) = \max\{\lambda(a_0), \ldots, \lambda(a_n), \lambda(b)\}$, when $a = \sum_{0 \leq i \leq n} a_i x^i / b \in \mathbb{Q}[x]$ with all $a_i \in \mathbb{Z}$ and $b \in \mathbb{N}_{\geq 1}$ such that $\gcd(a_0, \ldots, a_n, b) = 1$.

Thus a can be represented with about $\lambda(a)(2 + \deg a)$ words. Then for $a, b \in \mathbb{Z}[x]$ and $c, d \in \mathbb{Q}$, we have

$$\begin{aligned} \lambda(a+b) &\leq \max\{\lambda(a), \lambda(b)\} + 1, \\ \lambda(ab) &\leq \lambda(a) + \lambda(b) + \lambda(\min\{\deg a, \deg b\} + 1), \\ \lambda(cd), \lambda(c/d) &\leq \lambda(c) + \lambda(d), \\ \lambda(c+d) &\leq \lambda(c) + \lambda(d) + 1. \end{aligned}$$

We next consider a division with remainder $a = qb + \rho r$, where the polynomials $a = x^n + \sum_{0 \leq i < n} a_i x^i / c$, $b = x^m + \sum_{0 \leq i < m} b_i x^i / d \in \mathbb{Q}[x]$, and $r \in \mathbb{Q}[x]$ are monic, with all $a_i, b_i \in \mathbb{Z}$ and $c, d \in \mathbb{N}_{\geq 1}$, in the special case where $m = n - 1$. Then

$$\begin{aligned} q &= x + \frac{a_{n-1} d - b_{m-1} c}{cd}, \quad \lambda(q) \leq \lambda(a) + \lambda(b) + 1, \\ \rho r &= a - qb = \frac{acd^2 - xbcd^2 - (a_{n-1} d - b_{m-1} c) bd}{cd^2}, \\ \lambda(\rho r) &\leq \lambda(a) + 2\lambda(b) + 3, \end{aligned}$$

and the latter estimate also holds for $\lambda(r)$ since the cd^2 in the denominator of ρr and the numerator of $1/\rho$ cancel. Assuming that $\lambda(a) \leq \lambda(b)$, we see that the

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

coefficient size grows at most by a factor of about 3 in one division. Some experiments with pseudorandom polynomials of degree $n = 10$ and with 10, 100, and 1000 decimal digit coefficients indicate that this is essentially sharp: the average length ratio between the remainder coefficients and those of the input was 2.92, 2.998, and 2.9999, respectively, for 10 experiments each.

In a typical execution of the Euclidean Algorithm, the degrees of all the quotient polynomials will be 1. From the above worst-case estimate, we find that $\lambda(r_\ell) \in O(3^\ell \cdot \max\{\lambda(f), \lambda(g)\})$. This looks like bad news: an exponential upper bound on the size of the gcd and the number of word operations of the Euclidean Algorithm!

In reality, however, the sizes do not grow like that at every step, and we can prove that the coefficient sizes in the Euclidean Algorithm remain polynomially bounded in the input size. To prove this non-obvious result, we need a "global view" of the Euclidean Algorithm provided by the theory of (sub-)resultants. This theory will give us explicit formulas for the coefficients that appear in the polynomials in the Euclidean Algorithm. As a bonus, this theory will allow us to compute gcds in $\mathbb{Q}[x]$ using a modular approach, yielding a much more practical algorithm.

The following example illustrates the huge coefficients that actually occur in the Euclidean Algorithm in $\mathbb{Q}[x]$. It is typical in the sense that for most pairs of polynomials, with about as many coefficient digits as the degree, a similar growth of intermediate results occurs.

EXAMPLE 6.1. The following is generated on most platforms by the MAPLE commands in `typewriter font`. Most versions of MAPLE output `rho[0],r[0]` as ρ_0, r_0, etc.

```
f := randpoly(x, coeffs = rand(-999 .. 999), degree = 5);
```

$$f := 824x^5 - 65x^4 - 814x^3 - 741x^2 - 979x - 764$$

```
g := randpoly(x, coeffs = rand(-999 .. 999), degree = 4);
```

$$g := 216x^4 + 663x^3 + 880x^2 + 916x + 617$$

```
rho[0] := lcoeff(f, x); r[0] := f/rho[0];
```

$$\rho_0 := 824$$

$$r_0 := x^5 - \frac{65}{824}x^4 - \frac{407}{412}x^3 - \frac{741}{824}x^2 - \frac{979}{824}x - \frac{191}{206}$$

```
rho[1] := lcoeff(g, x); r[1] := g/rho[1];
```

$$\rho_1 := 216$$

$$r_1 := x^4 + \frac{221}{72}x^3 + \frac{110}{27}x^2 + \frac{229}{54}x + \frac{617}{216}$$

```
printlevel := 2:
for i from 1 to 5 do
  q[i] := quo(r[i - 1], r[i], x, 'a[i + 1]');
  a[i + 1] := sort(a[i + 1]);
```

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

```
  if (a[i + 1] <> 0) then
    rho[i + 1] := lcoeff(a[i + 1], x);
    r[i + 1] := a[i + 1] / rho[i + 1];
  fi;
od;
```

$$q_1 := x - \frac{5837}{1854}$$

$$a_2 := \frac{614269}{133488}x^3 + \frac{1539085}{200232}x^2 + \frac{931745}{100116}x + \frac{3230125}{400464}$$

$$\rho_2 := \frac{614269}{133488}$$

$$r_2 := x^3 + \frac{3078170}{1842807}x^2 + \frac{3726980}{1842807}x + \frac{3230125}{1842807}$$

$$q_2 := x + \frac{61877369}{44227368}$$

$$a_3 := -\frac{1292018949205}{4527916852332}x^2 - \frac{386731352527}{1131979213083}x + \frac{914965415267}{2263958426166}$$

$$\rho_3 := \frac{-1292018949205}{4527916852332}$$

$$r_3 := x^2 + \frac{1546925410108}{1292018949205}x - \frac{1829930830534}{1292018949205}$$

$$q_3 := x + \frac{1126368994649461694}{2380941563727618435}$$

$$a_4 := \frac{4794883885762430016087234}{1669312965104792370132025}x + \frac{4044518439139721899509903}{1669312965104792370132025}$$

$$\rho_4 := \frac{4794883885762430016087234}{1669312965104792370132025}$$

$$r_4 := x + \frac{731586548698031843}{867315257966502554}$$

$$q_4 := x + \frac{396448327221414114828389881017}{1120587748227344134908028769570}$$

$$a_5 := \frac{-12899003280815986083083677755852974 95}{752235756701500871961459942888522916}$$

$$\rho_5 := \frac{-12899003280815986083083677755852974 95}{752235756701500871961459942888522916}$$

$$r_5 := 1$$

$$q_5 := x + \frac{731586548698031843}{867315257966502554}$$

$$a_6 := 0$$

Thus already for comparatively small input sizes, the numerators and denominators of the intermediate results in the Euclidean Algorithm are amazingly large. The traditional Euclidean Algorithm, where the remainders are not normalized at

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

each step, is run in Section 6.11 on the same example; its intermediate results are considerably bigger than here. The example also illustrates the phenomenon of **intermediate expression swell**: The 25-digit coefficients of a_4 contract to 18 digits in its normalized version r_4. At the next step, the relation between a_5 and r_5 is even more drastic. In the normal case, where all the quotients have degree 1, this is not a serious problem: the discussion in Section 6.11 implies that then $\lambda(a_i)$ is at most about $3\lambda(r_i)$. A more important issue is that the upper bound on the size of the coefficients of the gcd is smaller by about one order of magnitude than the corresponding bound for the other remainders, even when the gcd is nonconstant. This follows from the estimates in Section 6.6 and 6.11 below. $\diamond$

The basic question now is: does this algorithm really run in polynomial time? In other words: do the coefficients that occur have polynomially bounded length? The naive exponential upper bound and the above example may raise some doubt about this. But not to worry, all is well! Our proofs of polynomial bounds proceed in two stages: first for the computation of the gcd in Sections 6.5 and 6.6, and finally for all results of the EEA in Section 6.11.

Once we have a good bound on the final result, the basic idea to circumvent the intermediate expression swell is to use a modular approach. When the input polynomials f, g are in $\mathbb{Z}[x]$, then we may choose an appropriate prime $p \in \mathbb{N}$, compute $\gcd(f \bmod p, g \bmod p)$ in $\mathbb{F}_p[x]$, and recover the gcd from its image modulo p.

EXAMPLE 6.1 (continued). We let f, g be as in Example 6.1 and take $p = 7$. Then the Euclidean Algorithm in $\mathbb{F}_7[x]$ works as follows.

$$f := 824x^5 - 65x^4 - 814x^3 - 741x^2 - 979x - 764$$

$$g := 216x^4 + 663x^3 + 880x^2 + 916x + 617$$

```
rho[0] := lcoeff(f, x) mod 7; r[0] := f/rho[0] mod 7;
```

$$\rho_0 := 5$$

$$r_0 := x^5 + x^4 + x^3 + 3x^2 + 3x + 4$$

```
rho[1] := lcoeff(g, x) mod 7; r[1] := g/rho[1] mod 7;
```

$$\rho_1 := 6$$

$$r_1 := x^4 + 2x^3 + 2x^2 + x + 6$$

```
printlevel := 2:
for i from 1 to 5 do
  q[i] := Quo(r[i - 1], r[i], x, 'a[i + 1]') mod 7;
  a[i + 1] := a[i + 1];
  if (a[i + 1] <> 0) then
    rho[i + 1] := lcoeff(a[i + 1], x);
    r[i + 1] := a[i + 1] / rho[i + 1] mod 7;
  fi;
od;
```

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

$$
\begin{aligned}
q_1 &:= x+6 & a_4 &:= x+2\\
a_2 &:= x^3+4x^2+5x+3 & \rho_4 &:= 1\\
\rho_2 &:= 1 & r_4 &:= x+2\\
r_2 &:= x^3+4x^2+5x+3 & q_4 &:= x+1\\
q_2 &:= x+5 & a_5 &:= 6\\
a_3 &:= 5x^2+x+5 & \rho_5 &:= 6\\
\rho_3 &:= 5 & r_5 &:= 1\\
r_3 &:= x^2+3x+1 & q_5 &:= x+2\\
q_3 &:= x+1 & a_6 &:= 0
\end{aligned}
$$

Thus $\gcd(f \bmod 7, g \bmod 7) = 1$. If h is a common divisor of f and g in $\mathbb{Z}[x]$, then $h \bmod 7$ divides $f \bmod 7$ and $g \bmod 7$, which implies that $h \bmod 7$ is constant. Now $\operatorname{lc}(h)$ divides $\operatorname{lc}(f)$, and since the latter does not vanish modulo 7, we have $\deg h = \deg(h \bmod 7)$. This shows that h is constant. $\diamond$

In the above example, the modular approach has revealed that f and g have no nonconstant common divisors in $\mathbb{Z}[x]$, but does this also imply that they have no nonconstant common divisors in $\mathbb{Q}[x]$? The answer is yes, but it requires an important tool, **Gauß' lemma**, which we will discuss in Section 6.2 below.

Besides that, the following questions have to be addressed in order to make the idea from the above example into an algorithm.

- How big do we have to choose the modulus p so that we can recover the gcd from its image modulo p? This requires an upper bound on the size of the coefficients of the gcd, which is provided by **Mignotte's bound** 6.33 in Section 6.6. The corresponding question for polynomials with coefficients in $F[y]$ for a field F is trivial: the degree in y of the gcd is at most that of the input polynomials.
- How do we find the denominators of the monic gcd? If the gcd is constant, as in Example 6.1, then this is not an issue, but in general a monic nonconstant gcd will have rational coefficients that are not integers. One solution is rational number reconstruction, as discussed in Section 5.10. Another possibility is to multiply the modular gcd by a known multiple of all denominators; the results of Section 6.2 will provide such a multiple.
- Does the approach work for any prime, or are there primes where the degree of the modular gcd is too large? Unfortunately, there are such "unlucky" primes, but fortunately, not too many of them. This can be shown by using **resultants**, which we discuss in Section 6.3.

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

6.2. Gauß' lemma

In this chapter, we lay the groundwork for computing the gcd in rings like $\mathbb{Z}[x]$. The (Extended) Euclidean Algorithm of Chapter 3 works for polynomials in $R[x]$ only if R is a field. In fact, $\mathbb{Z}[x]$ is not a Euclidean domain (Exercise 3.17), and we first have to make sure that the gcd is well defined. One can of course apply the Euclidean Algorithm over the field of fractions K of an integral domain R such as $\mathbb{Z}$, but does that yield the gcd in $R[x]$? The answer is no: say for $f = 2x^2 + 2$, $g = 6x + 2$, we have $\gcd(f,g) = 1$ in $\mathbb{Q}[x]$, but $\gcd(f,g) = 2$ in $\mathbb{Z}[x]$. In this section, we elucidate the difference between these gcds, and will end up with an algorithm for gcds in $R[x]$. Our two standard examples are: $R = \mathbb{Z}$ and $K = \mathbb{Q}$, and $R = F[y]$ and $K = F(y)$ for a field F and another indeterminate y.

We recall that two elements a, b of a Unique Factorization Domain R are **associate** if $a = ub$ for a unit $u \in R$. We will assume that we have multiplicative functions "normal" and "lu" on R such that $\mathrm{lu}(a)$ is a unit and $a = \mathrm{lu}(a) \cdot \mathrm{normal}(a)$ for all $a \in R$, with the properties required in Section 3.4. We will say that a is **normalized** if $\mathrm{lu}(a) = 1$, and assume that every element $b \in R$ which is associate to a has $\mathrm{normal}(b) = \mathrm{normal}(a)$. In particular, $\mathrm{normal}(a) = 1$ and $\mathrm{lu}(a) = a$ if and only if a is a unit. Then for all $a, b \in R$, $\gcd(a,b)$ is the unique normalized associate of all greatest common divisors of a and b. In our two standard examples, we take $\mathrm{lu}(a) = \mathrm{sign}(a)$ and $\mathrm{normal}(a) = |a|$ for $R = \mathbb{Z}$, $\mathrm{lu}(a) = \mathrm{lc}(a)$ and $\mathrm{normal}(a) = a/\mathrm{lc}(a)$ for $R = F[x]$, and in both cases, we let $\mathrm{lu}(0) = 1$ and $\mathrm{normal}(0) = 0$.

DEFINITION 6.2. *The* ***content*** $\mathrm{cont}(f)$ *of* $f = f_n x^n + \cdots + f_1 x + f_0 \in R[x]$ *with* $f_0, \ldots, f_n \in R$ *and* R *a UFD is defined as* $\mathrm{cont}(f) = \gcd(f_0, f_1, \ldots, f_n) \in R$. *By convention, we take* $\mathrm{cont}(f) = \gcd(f_0) = \mathrm{normal}(f_0)$ *if* $n = 0$. *The polynomial* f *is* ***primitive*** *if* $\mathrm{cont}(f) = 1$. *We define the* ***primitive part*** $\mathrm{pp}(f)$ *of* f *by* $f = \mathrm{cont}(f) \cdot \mathrm{pp}(f)$. *The content and primitive part of* f *are unique since we have a unique* gcd.

The following examples illustrate these notions.

EXAMPLE 6.3. Let $R = \mathbb{Z}$, $K = \mathbb{Q}$,

$$f = 18x^3 - 42x^2 + 30x - 6,\ g = -12x^2 + 10x - 2 \in \mathbb{Z}[x].$$

Then

$$\mathrm{cont}(f) = \gcd(18, -42, 30, -6) = 6,\ \mathrm{cont}(g) = \gcd(-12, 10, -2) = 2,$$
$$\mathrm{pp}(f) = 3x^3 - 7x^2 + 5x - 1,\ \mathrm{pp}(g) = -6x^2 + 5x - 1. \diamond$$

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

EXAMPLE 6.4. Let

$$f = (y^3+3y^2+2y)x^3+(y^2+3y+2)x^2+(y^3+3y^2+2y)x+(y^2+3y+2),$$
$$g = (2y^3+3y^2+y)x^2+(3y^2+4y+1)x+(y+1)$$

in $\mathbb{F}_5[x,y]$. Using the Euclidean Algorithm in $R=\mathbb{F}_5[y]$ we compute

$$\begin{aligned}\operatorname{cont}(f) &= \gcd(y^3+3y^2+2y, y^2+3y+2) = y^2+3y+2,\\ \operatorname{cont}(g) &= \gcd(2y^3+3y^2+y, 3y^2+4y+1, y+1) = y+1,\\ \operatorname{pp}(f) &= yx^3+x^2+yx+1,\\ \operatorname{pp}(g) &= (2y^2+y)x^2+(3y+1)x+1. \diamond\end{aligned}$$

LEMMA 6.5. *For $f \in R[x]$ and $c \in R$, $\operatorname{cont}(cf) = \operatorname{cont}(c)\cdot\operatorname{cont}(f)$ and $\operatorname{pp}(cf) = \operatorname{pp}(c)\cdot\operatorname{pp}(f)$.*

PROOF. Exercise 6.4. □

The following result, due to Gauß, is the cornerstone for unique factorization of polynomials over UFDs.

THEOREM 6.6 *Gauß' lemma.*
For a Unique Factorization Domain R, the product of two primitive polynomials in $R[x]$ is primitive.

PROOF. Let $f,g \in R[x]$ be primitive and $p \in R$ a prime. Then $D = R/\langle p\rangle$ is an integral domain, and so is $D[x]$, since the product of the leading coefficients of two nonzero polynomials is nonzero. By assumption, $f \bmod p$ and $g \bmod p$ are both nonzero in $D[x]$, and hence $fg \bmod p$ is nonzero as well, or equivalently, $p \nmid \operatorname{cont}(fg)$. Thus $\operatorname{cont}(fg) = 1$. □

COROLLARY 6.7.
For $f,g \in R[x]$, $\operatorname{cont}(fg) = \operatorname{cont}(f)\operatorname{cont}(g)$ and $\operatorname{pp}(fg) = \operatorname{pp}(f)\operatorname{pp}(g)$.

PROOF. Let $h = \operatorname{pp}(fg)$. By Gauß' lemma, $h^* = \operatorname{pp}(f)\operatorname{pp}(g)$ is primitive. Then

$$\begin{aligned} fg &= (\operatorname{cont}(f)\cdot\operatorname{pp}(f))(\operatorname{cont}(g)\cdot\operatorname{pp}(g)) = \operatorname{cont}(f)\operatorname{cont}(g)\cdot h^*,\\ \operatorname{cont}(fg) &= \operatorname{cont}(\operatorname{cont}(f)\operatorname{cont}(g))\cdot\operatorname{cont}(h^*) = \operatorname{cont}(f)\operatorname{cont}(g),\end{aligned}$$

by Lemma 6.5 and since $\operatorname{cont}(f)\operatorname{cont}(g)$ is normalized, and the claim follows. □

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

Lemma 6.5 is just the special case of Corollary 6.7 when $g = c$ is constant.

It is convenient to extend the definition of **content** and **primitive part** to polynomials in $K[x]$. If $f = \sum_{0\le i\le n}(a_i/b)x^i \in K[x]$, with a common denominator $b \in R\setminus\{0\}$ and all $a_i \in R$, then we let $\operatorname{cont}(f) = \gcd(a_0,\dots,a_n)/\operatorname{cont}(b) \in K$ and $\operatorname{pp}(f) = f/\operatorname{cont}(f)$. For example, we have $\operatorname{cont}(-3x-9/2) = 3/2 \in \mathbb{Q}$ and $\operatorname{pp}(-3x-9/2) = -2x-3 \in \mathbb{Z}[x]$. Then $\operatorname{pp}(f)$ is a primitive polynomial in $R[x]$, and Exercise 6.4 shows that Lemma 6.5 and Corollary 6.7 hold for $c \in K$ and $f, g \in K[x]$.

We recall the following notions. A nonzero nonunit p of a ring R is **prime** if $p \mid ab$ implies that $p \mid a$ or $p \mid b$, and p is **irreducible** if $p = ab$ implies that one of a and b is a unit. Multiplication by units does not change the property of being (or not being) prime or irreducible. Prime elements are irreducible, and if R is a UFD, then the two notions coincide (Section 25.2). We can now prove the following celebrated theorem of Gauß.

THEOREM 6.8 *Gauß.*
If R is a UFD, then $R[x]$ is a UFD.

PROOF. Since R is an integral domain, $\deg(fg) = \deg f + \deg g$ holds for any nonzero polynomials $f, g \in R[x]$. This implies that the units of $R[x]$ are precisely the units of R, and that a prime $p \in R$ is irreducible in $R[x]$.

Let $f \in R[x]$ be a nonzero nonunit. Since R is a UFD, $\operatorname{cont}(f)$ can be written as a product of irreducibles of R, by the above. Let K denote the field of fractions of R. Then $K[x]$ is a Euclidean domain and therefore a UFD, and $\operatorname{pp}(f) = f_1 f_2 \cdots f_r$ in $K[x]$ with (over K) irreducible nonconstant polynomials $f_1,\dots,f_r$. Extracting contents, Corollary 6.7 yields the factorization

$$\operatorname{pp}(f) = \operatorname{pp}(f_1)\cdots\operatorname{pp}(f_r) \tag{2}$$

into primitive polynomials in $R[x]$. Since each $\operatorname{pp}(f_i)$ is primitive in $R[x]$ and irreducible in $K[x]$, it is irreducible in $R[x]$. This proves the existence of a factorization into irreducibles in $R[x]$.

By the additivity of the degree, every irreducible factor of a constant $f \in R$ belongs to R, and the uniqueness of the factorization of f in $R[x]$ follows from the one in R. Now we assume that $f \in R[x]$ is nonconstant, and let

$$p_1\cdots p_k \cdot f_1\cdots f_r = f = q_1\cdots q_l \cdot g_1\cdots g_s$$

be two factorizations of f into irreducibles, with normalized $p_1,\dots,p_k,q_1,\dots,q_l$ in R and nonconstant primitive $f_1,\dots,f_r,g_1,\dots,g_s \in R[x]$. Then $p_1\cdots p_k = \operatorname{cont}(f) = q_1\cdots q_s$, by Corollary 6.7. Thus $k = l$ and $p_1 = q_1,\dots,p_k = q_k$ after reordering, since R is a UFD. Furthermore,

$$f_1\cdots f_r = \operatorname{pp}(f) = g_1\cdots g_s \tag{3}$$

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

in $R[x]$. Since any factorization in $K[x]$ of a nonconstant primitive polynomial would lead to a nontrivial factorization in $R[x]$, as in (2), these polynomials remain irreducible in $K[x]$. Thus (3) also contains two factorizations of $\mathrm{pp}(f)$ in $K[x]$ into irreducibles, and hence $r = s$ and—after a suitable renumbering—$f_i = b_i g_i$, with $b_i \in K\backslash\{0\}$, for $1 \le i \le r$. Since f_i and g_i are primitive, we have

$$f_i = \mathrm{pp}(f_i) = \mathrm{pp}(b_i g_i) = \mathrm{pp}(b_i)\,\mathrm{pp}(g_i) = \mathrm{pp}(b_i) g_i$$

for $1 \le i \le r$, which concludes the proof since $\mathrm{pp}(b_i)$ is a unit in R. □

In particular, since $R[x]$ is a UFD, any two elements of $R[x]$ have a gcd. In order to have a function gcd on $R[x]$, we extend "lu" to $R[x]$ via $\mathrm{lu}(f) = \mathrm{lu}(\mathrm{lc}(f))$ (Exercise 3.8 (iii)) to define a normal form on $R[x]$. Then a polynomial in $R[x]$ is **normalized** precisely when its leading coefficient is, and $\gcd(f,g)$ is the unique normalized associate in $R[x]$ of all greatest common divisors of f and g, as usual.

Both 5 and $5x+1$ are primes in $\mathbb{Z}[x]$, but 5 is a unit in $\mathbb{Q}[x]$, while $5x+1$ is a prime also in $\mathbb{Q}[x]$. More generally, nonconstant polynomials are not units, so that $R^\times = (R[x])^\times$, and $\{1,-1\} = \mathbb{Z}^\times = (\mathbb{Z}[x])^\times \subset \mathbb{Q}\setminus\{0\} = \mathbb{Q}^\times = (\mathbb{Q}[x])^\times$, where $R^\times$ is the group of units of a ring R.

COROLLARY 6.9.
Let R be $\mathbb{Z}$ or a field, and $n \ge 0$. Then $R[x_1,\dots,x_n]$ is a Unique Factorization Domain.

COROLLARY 6.10.
Let R be a UFD with field of fractions K, $f,g \in R[x]$, and h the normalized gcd *of f and g in $R[x]$.*

(i) The primes of $R[x]$ are the primes of R plus the primitive polynomials in $R[x]$ that are irreducible in $K[x]$.

(ii) $\mathrm{cont}(h) = \gcd(\mathrm{cont}(f),\mathrm{cont}(g))$ *in R and* $\mathrm{pp}(h) = \gcd(\mathrm{pp}(f),\mathrm{pp}(g))$ *in $R[x]$. In particular, $h = \gcd(\mathrm{cont}(f),\mathrm{cont}(g)) \cdot \gcd(\mathrm{pp}(f),\mathrm{pp}(g))$, and h is primitive if one of f and g is.*

(iii) $h/\mathrm{lc}(h) \in K[x]$ is the monic gcd *of f and g in $K[x]$.*

PROOF. (i) Let $p \in R[x]$. We first assume that p is prime. If p is a constant, then p is prime in R. Otherwise, it is a primitive polynomial and irreducible in $K[x]$, since a factorization in $K[x]$ leads to one in $R[x]$, as in (2).

On the other hand, if p is not prime, then a factorization $p = uv$ with $u, v \notin R^\times$ shows that p is not prime in R, and, if p is primitive and nonconstant, that p is reducible in $K[x]$.

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

(ii) The polynomial h divides f, and hence $\mathrm{cont}(h)$ divides $\mathrm{cont}(f)$, and, by symmetry, it divides $\mathrm{cont}(g)$ and hence $\gcd(\mathrm{cont}(f),\mathrm{cont}(g))$. On the other hand, this gcd is in R and a common factor of f and g, hence divides h and then also $\mathrm{cont}(h)$. This proves the first claim. The second one follows similarly, using the fact that $\mathrm{pp}(h)$ divides $\mathrm{pp}(f)$, by Corollary 6.7, and that $\mathrm{pp}(h)$ is normalized since h is (Exercise 3.8 (iv)).

(iii) Since $h/\mathrm{lc}(h)$ is a divisor of f and g in $K[x]$, it also divides their monic gcd h^*. On the other hand, $f = f^*h^*$ for some $f^* \in K[x]$, and taking contents together with Corollary 6.7 shows that $\mathrm{pp}(h^*) \mid \mathrm{pp}(f) \mid f$ in $R[x]$, and similarly $\mathrm{pp}(h^*) \mid g$. Thus $\mathrm{pp}(h^*)$ divides $h = \gcd(f,g)$ in $R[x]$, which implies that h^* and $h/\mathrm{lc}(h)$ divide each other in $K[x]$, and since both are monic, they are equal. □

We note that part (ii) is wrong when h is not normalized: for example, if $R = \mathbb{Z}$, $f = g = x$, and $h = -x$, then $\mathrm{pp}(h) = -x \neq x = \gcd(\mathrm{pp}(f),\mathrm{pp}(g))$. The following examples illustrate the difference between gcds in $R[x]$ and $K[x]$.

EXAMPLE 6.3 (continued). With the Euclidean Algorithm in $\mathbb{Q}[x]$, we find that $\gcd(f,g) = \gcd(\mathrm{pp}(f),\mathrm{pp}(g)) = x - 1/3$ in $\mathbb{Q}[x]$; see the continuation of Example 3.7 on page 58. Hence

$$\begin{aligned}
\mathrm{pp}(\gcd(f,g)) &= \gcd(\mathrm{pp}(f),\mathrm{pp}(g)) = 3x-1 \text{ in } \mathbb{Z}[x],\\
\mathrm{cont}(\gcd(f,g)) &= \gcd(\mathrm{cont}(f),\mathrm{cont}(g)) = \gcd(6,2) = 2,\\
\gcd(f,g) &= \mathrm{cont}(\gcd(f,g)) \cdot \mathrm{pp}(\gcd(f,g)) = 6x-2 \text{ in } \mathbb{Z}[x].
\end{aligned}$$

The polynomials f and $\mathrm{pp}(f)$ are normalized in $\mathbb{Z}[x]$ since their leading coefficients are positive, but g and $\mathrm{pp}(g)$ are not. Both $\gcd(f,g)$ and $\gcd(\mathrm{pp}(f),\mathrm{pp}(g))$ are normalized. ◇

EXAMPLE 6.4 (continued). Since $R = \mathbb{F}_5[y]$ is a Euclidean domain and we can regard f,g as polynomials in $R[x]$, we can apply the above results to compute the greatest common divisor of f and g.

$$\begin{aligned}
\gcd(\mathrm{cont}(f),\mathrm{cont}(g)) &= y+1,\\
\gcd(\mathrm{pp}(f),\mathrm{pp}(g)) &= yx+1 \text{ in } \mathbb{F}_5[y][x] = \mathbb{F}_5[x,y],\\
\gcd(f,g) &= \gcd(\mathrm{pp}(f),\mathrm{pp}(g)) = x + \frac{1}{y} \text{ in } \mathbb{F}_5(y)[x],\\
\gcd(f,g) &= \gcd(\mathrm{cont}(f),\mathrm{cont}(g)) \cdot \gcd(\mathrm{pp}(f),\mathrm{pp}(g))\\
&= (y^2+y)x + (y+1) \text{ in } \mathbb{F}_5[y][x].
\end{aligned}$$

Thus f and $\mathrm{pp}(f)$ are normalized in $R[x]$, while g and $\mathrm{pp}(g)$ are not. Both $\gcd(f,g)$ and $\gcd(\mathrm{pp}(f),\mathrm{pp}(g))$ are normalized. ◇

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

We obtain the following algorithm for calculating gcds in $\mathbb{Z}[x]$ and $F[x,y]$. By Corollary 6.10 (ii) we may assume that the input polynomials are primitive.

ALGORITHM 6.11 Gcd of primitive polynomials.
Input: Primitive $f, g \in R[x]$, where R is a UFD.
Output: $h = \gcd(f,g)$ in $R[x]$.

1. **call** the Euclidean Algorithm 3.14 to determine the monic gcd v of f and g in $K[x]$, where K is the field of fractions of R
2. $b \longleftarrow \gcd(\mathrm{lc}(f), \mathrm{lc}(g))$
3. **return** $\mathrm{pp}(bv) \in R[x]$

THEOREM 6.12.
The algorithm works correctly as specified.

PROOF. Let h be the normalized gcd of f and g in $R[x]$. By Corollary 6.10, v as computed in step 1 equals $h/\mathrm{lc}(h)$. Since h divides f and g in $R[x]$, $\mathrm{lc}(h)$ divides $\mathrm{lc}(f)$ and $\mathrm{lc}(g)$ in R, hence it divides $\gcd(\mathrm{lc}(f), \mathrm{lc}(g))$, and $bv \in R[x]$. Furthermore, $\mathrm{pp}(bv) \in R[x]$ is primitive, by definition, and normalized, since b is (Exercise 3.8 (iv)), so that $\mathrm{pp}(bv) = h$. □

To compute the gcd of arbitrary polynomials, we of course compute the gcd of their contents in R and apply the algorithm to their primitive parts. However, Example 6.1 indicates that Algorithm 6.11 is not the best way to do this, and the modular algorithms of Sections 6.4 through 6.7 will be much more efficient.

6.3. The resultant

The central goal of this whole chapter is to find modular gcd algorithms for domains like $\mathbb{Z}[x]$, $\mathbb{Q}[x]$, and $F[x,y]$. Section 6.13 reports on implementations that show how much these algorithms are superior to the "traditional" one, whose problems are quite visible in Example 6.1. The simplest such approach, the **big prime modular algorithm**, chooses a large prime p, calculates the gcd modulo p, and recovers the true gcd from its modular image. This is quite easy, provided that the modular gcd is indeed the image of the true gcd; this may, in fact, fail in exceptional cases.

This section provides a general tool, the **resultant**, to control modular images of the gcd. This introduces linear algebra into our polynomial problems. We also discuss other applications, such as curve intersection and minimal polynomials of algebraic elements. In Section 6.10, we introduce the **subresultants**, a generalization that gives us control over all results of the EEA. But the reader should

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

realize clearly that for gcd calculations the resultant is purely an (indispensable) conceptual tool and does not enter the algorithms, but only their analysis.

Now let F be a field and $f,g \in F[x]$. The following lemma says that the vanishing linear combination $(-g)\cdot f + f\cdot g = 0$ has the smallest possible coefficient degrees if and only if $\gcd(f,g) = 1$.

LEMMA 6.13. *Let $f,g \in F[x]$ be nonzero. Then $\gcd(f,g) \neq 1$ if and only if there exist $s,t \in F[x]\setminus\{0\}$ such that $sf+tg = 0$, $\deg s < \deg g$, and $\deg t < \deg f$.*

PROOF. Let $h = \gcd(f,g)$. If $h \neq 1$, then $\deg h \geq 1$, and $s = -g/h$, $t = f/h$ suffice. Conversely, let s,t be as assumed. If f and g were coprime, then $sf = -tg$ would imply that $f \mid t$, which is impossible since $t \neq 0$ and $\deg f > \deg t$. This contradiction shows that $h \neq 1$. □

We now reformulate Lemma 6.13 in a different language. Given nonzero $f,g \in F[x]$ of degrees n,m, respectively, we let

$$\begin{array}{rccc} \varphi = \varphi_{f,g}\colon & F[x]\times F[x] & \longrightarrow & F[x] \\ & (s,t) & \longmapsto & sf+tg \end{array} \tag{4}$$

be the "linear combination map". For $d \in \mathbb{N}$, we let $P_d = \{a \in F[x] : \deg a < d\}$, with the convention that $P_0 = \{0\}$. Then φ is a linear mapping of infinite-dimensional vector spaces over F. (It is also an $F[x]$-linear map of $F[x]$-modules, in the natural way.) The restriction of φ to $\varphi_0\colon P_m \times P_n \longrightarrow P_{n+m}$ is an F-linear mapping between vector spaces of the same finite dimension, and Lemma 6.13 says the following.

THEOREM 6.14.
Let $f,g \in F[x]$ be nonzero of degrees n,m, respectively.

(i) $\gcd(f,g) = 1 \iff \varphi_0$ *is an isomorphism.*

(ii) If $\gcd(f,g) = 1$ *and* $n+m \geq 1$, *then the Bézout coefficients s_ℓ, t_ℓ computed by the EEA form the unique solution in $P_m \times P_n$ of $\varphi_0(s_\ell,t_\ell) = 1$.*

PROOF. Lemma 6.13 says that

$$\begin{aligned} \deg\gcd(f,g) \geq 1 &\iff \text{there exists a nonzero } (s,t) \in P_m \times P_n \text{ with } \varphi_0(s,t) = 0 \\ &\iff \varphi_0 \text{ is not injective.} \end{aligned}$$

For our map φ_0 between vector spaces of equal (finite) dimension, the following three properties are equivalent:

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

- φ_0 is an isomorphism,
- φ_0 is injective (or one-to-one),
- φ_0 is surjective (or onto).

Claim (i) now follows. For (ii), we recall from Lemma 3.15 (b) that $(s_\ell, t_\ell) \in P_m \times P_n$. Since φ_0 is an isomorphism, the solution $\varphi_0(s_\ell, t_\ell) = 1$ is unique. □

In order to compute with the linear map φ_0, we represent it by a matrix. We write $f = \sum_{0\le j\le n} f_j x^j$, $g = \sum_{0\le j\le m} g_j x^j$, with all $f_j, g_j \in F$. The natural basis for $P_m \times P_n$ consists of $(x^i, 0)$ for $i < m$ and $(0, x^j)$ for $j < n$, and for P_{n+m} we take all x^k with $0 \le k < n+m$. On these bases, φ_0 is represented by the $(n+m)\times(n+m)$ matrix S with entries in F defined as

$$S = \begin{pmatrix} f_n & & & & g_m & & & & \\ f_{n-1} & f_n & & & g_{m-1} & g_m & & & \\ \vdots & \vdots & \ddots & & \vdots & \vdots & \ddots & & \\ \vdots & \vdots & & f_n & g_1 & \vdots & & \ddots & \\ \vdots & \vdots & & f_{n-1} & g_0 & \vdots & & & \ddots \\ \vdots & \vdots & & \vdots & & g_0 & & & g_m \\ f_0 & \vdots & & \vdots & & & \ddots & & \vdots \\ & f_0 & & \vdots & & & & \ddots & \vdots \\ & & \ddots & \vdots & & & & & \ddots \\ & & & f_0 & & & & & g_0 \end{pmatrix}, \qquad (5)$$

with m columns of f_j's and n columns of g_j's, and all entries outside the two "parallelograms" equal to zero. This means that when we write

$$s = \sum_{0\le j<m} y_j x^j, \quad t = \sum_{0\le j<n} z_j x^j, \quad sf + tg = \sum_{0\le j<n+m} u_j x^j,$$

with all $y_j, z_j, u_j \in F$, then

$$\begin{pmatrix} u_{n+m-1} \\ \vdots \\ \vdots \\ \vdots \\ u_0 \end{pmatrix} = S \cdot \begin{pmatrix} y_{m-1} \\ \vdots \\ y_0 \\ z_{n-1} \\ \vdots \\ z_0 \end{pmatrix}.$$

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

This is the central step; we advise the reader to understand it thoroughly. Theorem 6.14 can now be restated as follows.

COROLLARY 6.15.
Let f, g, n, m be as in Theorem 6.14.

(i) $\gcd(f,g) = 1 \iff \det S \neq 0$.

(ii) If $\gcd(f,g) = 1$, $n+m \geq 1$, *and* $y_0, \ldots, y_{m-1}, z_0, \ldots, z_{n-1} \in F$ *satisfy*

$$S \cdot \begin{pmatrix} y_{m-1} \\ \vdots \\ y_0 \\ z_{n-1} \\ \vdots \\ z_0 \end{pmatrix} = \begin{pmatrix} 0 \\ \vdots \\ \vdots \\ 0 \\ 1 \end{pmatrix},$$

then $s_\ell = \sum_{0 \leq i < m} y_i x^i$ *and* $t_\ell = \sum_{0 \leq i < n} z_i x^i$ *are the Bézout coefficients computed by the EEA, with* $s_\ell f + t_\ell g = 1$.

If R is a (commutative) ring and $f, g \in R[x]$, then $\mathrm{Syl}(f,g) = S$ is their **Sylvester matrix** (sometimes the transpose of S gets this name), and $\mathrm{res}(f,g) = \det S$ their **resultant**. If $n = m = 0$, then S is the "empty" 0×0 matrix with determinant $\mathrm{res}(f,g) = 1$, and it is convenient to define the resultant with the zero polynomial as $\mathrm{res}(f,0) = \mathrm{res}(0,f) = 0$ if f is zero or nonconstant and $\mathrm{res}(f,0) = \mathrm{res}(0,f) = 1$ if f is a nonzero constant. Then the resultant is defined for all pairs of polynomials, and Corollary 6.15 (i) holds in all cases.

EXAMPLE 6.16. Let $f = r_0 = x^4 - 3x^3 + 2x$ and $g = r_1 = x^3 - 1$ in $\mathbb{Q}[x]$. The quotients and remainders in the Euclidean Algorithm are

$$\begin{aligned} r_0 &= q_1 r_1 + \rho_2 r_2 = (x-3) r_1 + 3(x-1), \\ r_1 &= q_2 r_2 = (x^2 + x + 1) r_2. \end{aligned}$$

Thus $\gcd(f,g) = r_2 = x - 1$ (both in $\mathbb{Z}[x]$ and in $\mathbb{Q}[x]$), and the resultant is

$$\mathrm{res}(f,g) = \det \begin{pmatrix} 1 & 0 & 0 & 1 & 0 & 0 & 0 \\ -3 & 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & -3 & 1 & 0 & 0 & 1 & 0 \\ 2 & 0 & -3 & -1 & 0 & 0 & 1 \\ 0 & 2 & 0 & 0 & -1 & 0 & 0 \\ 0 & 0 & 2 & 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & -1 \end{pmatrix} = 0.$$

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

If we divide out the gcd, so that $r_0 = f/(x-1) = x^3 - 2x^2 - 2x$ and $r_1 = g/(x-1) = x^2 + x + 1$, we have

$$r_0 = q_1 r_1 + r_2 = (x-3)r_1 + 3 \cdot 1,$$
$$r_1 = q_2 r_2 = (x^2 + x + 1)r_2.$$

Now r_0, r_1 are coprime and

$$\operatorname{res}(r_0, r_1) = \det \begin{pmatrix} 1 & 0 & 1 & 0 & 0 \\ -2 & 1 & 1 & 1 & 0 \\ -2 & -2 & 1 & 1 & 1 \\ 0 & -2 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 \end{pmatrix} = 9. \diamond$$

COROLLARY 6.17.
Let F be a field, and $f, g \in F[x]$ nonzero. Then the following are equivalent:

(i) $\gcd(f,g) = 1$,

(ii) $\operatorname{res}(f,g) = \det S \neq 0$,

(iii) there do not exist $s, t \in F[x] \setminus \{0\}$ such that

$$sf + tg = 0, \quad \deg s < \deg g, \quad \deg t < \deg f.$$

EXAMPLE 6.18. A quadratic polynomial $f = ax^2 + bx + c \in F[x]$, with $\operatorname{char} F \neq 2$ and $a \neq 0$, is squarefree (that is, it has no double root) if and only if its discriminant $4ac - b^2$ does not vanish. Equivalently, $\gcd(f, f') = 1$, where $f' = 2ax + b$ is the derivative of f (Section 9.3). We calculate

$$\operatorname{res}(f, f') = \det \begin{pmatrix} a & 2a & 0 \\ b & b & 2a \\ c & 0 & b \end{pmatrix} = a(4ac - b^2). \diamond$$

EXAMPLE 6.19. If $F \subseteq K$ are fields, $f, g \in F[x]$, and $h \in K[x]$ is nonconstant and divides f and g, then there is also a nonconstant polynomial $k \in F[x]$ dividing f and g. This is because the resultant $\operatorname{res}(f,g)$ is the same whether we consider it over F or K. By assumption, it is zero in K, hence also in F. Moreover, even the (monic) gcd of f and g is the same over F and over K. This is in contrast to the fact that a polynomial f may very well have a nontrivial factor, with degree between 1 and $\deg f - 1$, over K, but none over F, such as the irreducible $x^2 - 2 \in \mathbb{Q}[x]$, which factors as $x^2 - 2 = (x - \sqrt{2})(x + \sqrt{2})$ in $\mathbb{R}[x]$. $\diamond$

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

Combining Corollaries 6.10 and 6.17, we obtain the following.

COROLLARY 6.20.
Let R be a UFD and $f,g \in R[x]$ not both zero. Then $\gcd(f,g)$ is nonconstant in $R[x]$ if and only if $\operatorname{res}(f,g) = 0$ in R.

COROLLARY 6.21.
Let R be an integral domain and $f,g \in R[x]$ nonzero with $\deg f + \deg g \geq 1$. Then there exist nonzero $s,t \in R[x]$ such that $sf + tg = \operatorname{res}(f,g)$, $\deg s < \deg g$, and $\deg t < \deg f$.

PROOF. Let F be the field of fractions of R. If $r = \operatorname{res}(f,g) = 0$, then Corollary 6.17 yields such $s,t \in F[x]$, and the claim follows from multiplying with a common denominator. If the resultant is nonzero, then f and g are coprime in $F[x]$, and there exist nonzero $s^*,t^* \in F[x]$ with the stated degree bounds such that $s^*f + t^*g = 1$, by Lemma 3.15 (b). Now Theorem 6.14 says that the coefficients of s^* and t^* are the unique solution of a linear system with coefficient matrix $S = \operatorname{Syl}(f,g)$, and Cramer's rule 25.6 implies that each of them is the quotient of the determinant of a submatrix of S by $r = \det S$. Thus $s = rs^*$ and $t = rt^*$ are in $R[x]$. □

When $f,g \in F[x,y]$, we write $\operatorname{res}_x(f,g)$ for the resultant in $F[y]$ with respect to x. Symmetrically, there is also a polynomial $\operatorname{res}_y(f,g) \in F[x]$. We have the following bound on $\deg_y \operatorname{res}_x(f,g)$, where $\deg_y$ denotes the degree with respect to the variable y (Section 25.3).

THEOREM 6.22.
Let $f,g \in F[x,y]$ with $n = \deg_x f$, $m = \deg_x g$, and $\deg_y f, \deg_y g \leq d$. Then

$$\deg_y \operatorname{res}_x(f,g) \leq (n+m)d.$$

PROOF. When we write the determinant $\operatorname{res}_x(f,g)$ as the familiar sum of $(n+m)!$ terms, then each nonzero term has m factors that are coefficients of f, and n factors that are coefficients of g. Hence the degree of each term is at most $md + nd$. □

In Section 6.8, we discuss Bézout's theorem, which corresponds to the bound $\deg_y \operatorname{res}_x(f,g) \leq \deg f \cdot \deg g$, where deg is the total degree (see Exercise 6.11).

The following corollary gives an analogous bound on the size of the resultant, but now for integer polynomials f and g. It is an immediate consequence of Hadamard's inequality 16.6. Two expressions for the "size" of f play a role: The **2-norm** (or Euclidean norm) $\|f\|_2$ of a polynomial $f = \sum_{0 \leq i \leq n} f_i x^i \in \mathbb{Z}[x]$ is

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

$\|f\|_2 = (\sum_{0\le i\le n} f_i^2)^{1/2}$. The **max-norm** is $\|f\|_\infty = \max\{|f_i| : 0 \le i \le n\}$, and the relation $\|f\|_\infty \le \|f\|_2 \le (n+1)^{1/2}\|f\|_\infty$ shows that the two norms differ only by a small factor (Section 25.5).

THEOREM 6.23.
Let $f, g \in \mathbb{Z}[x]$, $n = \deg f$, and $m = \deg g$. Then

$$|\operatorname{res}(f,g)| \le \|f\|_2^m \|g\|_2^n \le (n+1)^{m/2}(m+1)^{n/2}\|f\|_\infty^m \|g\|_\infty^n.$$

6.4. Modular gcd algorithms

Our goal in this and the following sections is to provide a modular gcd algorithm for $\mathbb{Z}[x]$ and $F[x,y]$. We start by investigating the relation between the modular image of the gcd and the gcd of modular images. It turns out that they are usually (essentially) equal, but this fails for the "unlucky primes" that divide a certain resultant.

We let $f, g \in R[x]$ for a Euclidean domain R, $p \in R$ a prime, and denote by a bar the reduction modulo p. Corollary 6.20 says that $\gcd(f,g)$ is constant if and only if $\operatorname{res}(f,g) \ne 0$. The resultant $\operatorname{res}(f,g) \in R$ is a polynomial expression in the coefficients of f and g, and one might be tempted to say: since $\overline{\operatorname{res}(f,g)} = \operatorname{res}(\overline{f},\overline{g})$, $\overline{f}$ and $\overline{g}$ are coprime in $(R/\langle p\rangle)[x]$ if and only if $p \nmid \operatorname{res}(f,g)$.

EXAMPLE 6.24. To get a taste of what can go wrong without further assumptions, we let $R = \mathbb{Z}$ and $p = 2$. When $f = x+2$ and $g = x$, then $\operatorname{res}(f,g) = -2 \ne 0$ and $\operatorname{res}(\overline{f},\overline{g}) = 0$, as expected. But when $f = 4x^3 - x$ and $g = 2x+1$, then $\operatorname{res}(f,g) = 0$ and $\operatorname{res}(\overline{f},\overline{g}) = \operatorname{res}(x,1) = 1 \ne 0$; in particular, $\overline{\operatorname{res}(f,g)} \ne \operatorname{res}(\overline{f},\overline{g})$. $\diamond$

The reason for the unexpected behavior in the last example is that the two relevant Sylvester matrices are formed in rather different ways. Fortunately, this nuisance disappears when p does not divide at least one of the leading coefficients.

LEMMA 6.25. *We let R be a ring (commutative, with 1), $f, g \in R[x]$ nonzero, $r = \operatorname{res}(f,g) \in R$, $I \subseteq R$ an ideal, denote by a bar the reduction modulo I, and assume that $\overline{\operatorname{lc}(f)}$ is not a zero divisor.*

(i) $\overline{r} = 0 \iff \operatorname{res}(\overline{f},\overline{g}) = 0$.

(ii) If R/I is a UFD, then $\overline{r} = 0$ if and only if $\gcd(\overline{f},\overline{g})$ is nonconstant.

PROOF. We write $f = \sum_{0\le j\le n} f_j x^j$, $g = \sum_{0\le j\le m} g_j x^j$, with nonzero f_n, g_m and all $f_j, g_j \in R$. If $\deg f = 0$, then both Sylvester matrices $\operatorname{Syl}(f,g)$ and $\operatorname{Syl}(\overline{f},\overline{g})$ are

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

diagonal with f and $\overline{f}$ on the diagonal, respectively, and both $\overline{r}$ and $\operatorname{res}(\overline{f},\overline{g})$ are nonzero. So let $\deg f \geq 1$. If $\overline{g} = 0$, then $\operatorname{res}(\overline{f},\overline{g}) = 0$ and each column of g_j's in the Sylvester matrix $\operatorname{Syl}(f,g)$ vanishes modulo I, so that $\overline{r} = 0$.

We now assume that $\overline{g} \neq 0$, and let i be the smallest index with $\overline{g}_{m-i} \neq 0$. Then we can partition $\operatorname{Syl}(f,g)$ as in Figure 6.1. The lower right submatrix, taken

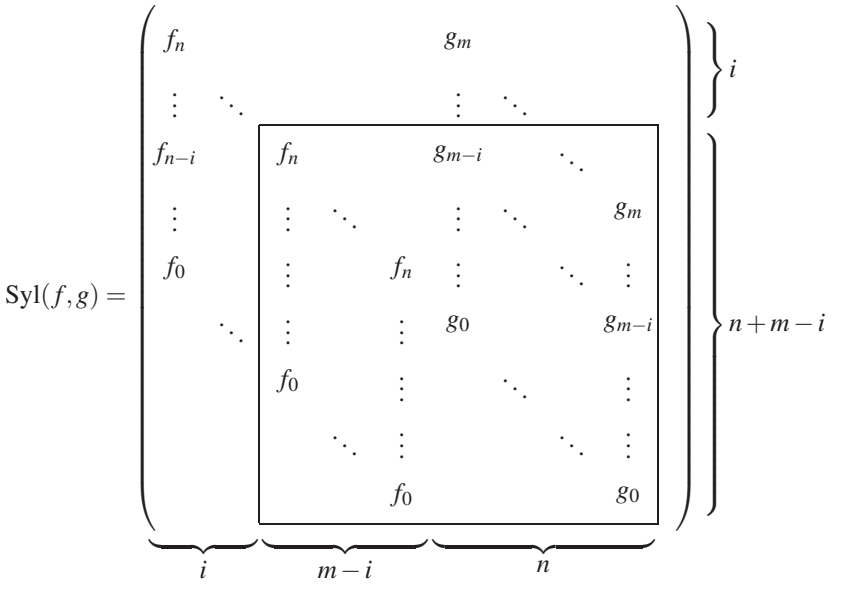


FIGURE 6.1: The Sylvester matrix of f and g.

modulo I, is $\operatorname{Syl}(\overline{f},\overline{g})$. All g_j in the first i rows are zero modulo I, and repeated Laplace expansion (Section 25.5) of $r = \det \operatorname{Syl}(f,g)$ along the first row yields modulo I that $\overline{r} = \overline{f_n^i}\operatorname{res}(\overline{f},\overline{g})$. This proves (i), and the second claim follows from Corollary 6.20. □

The conclusion may be false when both leading coefficients vanish modulo I, as in the second case of Example 6.24.

THEOREM 6.26.

Let R be a Euclidean domain, $p \in R$ prime, and $f,g \in R[x]$ nonzero. Furthermore, let $h = \gcd(f,g) \in R[x]$, $e = \deg h$, $\alpha = \operatorname{lc}(h)$, and assume that p does not divide $b = \gcd(\operatorname{lc}(f),\operatorname{lc}(g)) \in R$. A bar denotes reduction modulo p, and we let $e^* = \deg\gcd(\overline{f},\overline{g})$. Then

(i) α divides b,

(ii) $e^* \geq e$,

(iii) $e^* = e \iff \overline{\alpha}\cdot\gcd(\overline{f},\overline{g}) = \overline{h} \iff p \nmid \operatorname{res}(f/h, g/h)$ in R.

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

PROOF. Since h divides f and g in $R[x]$, $\mathrm{lc}(h)$ divides $\mathrm{lc}(f)$ and $\mathrm{lc}(g)$ in R, and (i) follows. Let $u = f/h$ and $v = g/h \in R[x]$. Then $\deg \overline{h} = e$ since $p \nmid b$ and by (i), and

$$\overline{u}\,\overline{h} = \overline{f} \ \text{ and } \ \overline{v}\,\overline{h} = \overline{g} \tag{6}$$

imply that $\overline{h}$ divides $\gcd(\overline{f}, \overline{g})$, which shows (ii) and the first equivalence in (iii). (Recall that over a field such as $R/\langle p \rangle$, polynomial gcds are always taken to be monic.)

Now $p \nmid b$ implies that p divides at most one of $\mathrm{lc}(u)$ and $\mathrm{lc}(v)$, say $p \nmid \mathrm{lc}(u)$. Then Lemma 6.25 (ii) implies that p divides $\mathrm{res}(u, v)$ if and only if $\gcd(\overline{u}, \overline{v}) \neq 1$ in $R/\langle p \rangle$. From (6), we find that $\gcd(\overline{f}, \overline{g}) = \gcd(\overline{u}, \overline{v}) \cdot \overline{h}/\overline{\alpha}$, and this implies the second equivalence in (iii). □

EXAMPLE 6.3 (continued). Let $R = \mathbb{Z}$,

$$f = 18x^3 - 42x^2 + 30x - 6, \ g = -12x^2 + 10x - 2 \in \mathbb{Z}[x]$$

as in Example 6.3, and $p = 17$. Then $h = \gcd(f, g) = 6x - 2$, so that $e = 1$, and $p \nmid \mathrm{lc}(h)$. Computing $f/h = 3x^2 - 6x + 3$, $g/h = -2x + 1$, and

$$\mathrm{res}\left(\frac{f}{h}, \frac{g}{h}\right) = \det \begin{pmatrix} 3 & -2 & 0 \\ -6 & 1 & -2 \\ 3 & 0 & 1 \end{pmatrix} = 3 \not\equiv 0 \bmod 17,$$

we conclude that $\deg \gcd(\overline{f}, \overline{g}) = 1$. In fact, $\gcd(\overline{f}, \overline{g}) = x + 11$ in $\mathbb{F}_{17}[x]$, and $\overline{\gcd(f, g)} = 6 \cdot \gcd(\overline{f}, \overline{g})$. ◇

EXAMPLE 6.16 (continued). Let $R = \mathbb{Z}$, $f = x^4 - 3x^3 + 2x$ and $g = x^3 - 1$ in $\mathbb{Z}[x]$. We have seen in Example 6.16 that $h = \gcd(f, g) = x - 1$, whence $e = 1$, and that $\mathrm{res}(f/h, g/h) = 9$. Thus for $p = 3$, we have $\deg \gcd(\overline{f}, \overline{g}) > 1$. Inspection reveals that $\gcd(\overline{f}, \overline{g}) = \overline{g}$ in $\mathbb{F}_3[x]$, and 3 is an "unlucky prime"; in fact, it is the only one, by Theorem 6.26. ◇

EXAMPLE 6.27. Let $R = \mathbb{F}_5[y]$,

$$f = yx^3 + x^2 + yx + 1, \quad g = (2y^2 + y)x^2 + (3y + 1)x + 1 \in R[x].$$

We have seen in Example 6.4 that $h = \gcd(f, g) = yx + 1$ in $R[x]$, and hence $e = 1$. Moreover, $f/h = x^2 + 1$, $g/h = (2y + 1)x + 1$, and

$$\mathrm{res}_x\left(\frac{f}{h}, \frac{g}{h}\right) = \det \begin{pmatrix} 1 & 2y+1 & 0 \\ 0 & 1 & 2y+1 \\ 1 & 0 & 1 \end{pmatrix} = 4y^2 + 4y + 2 = 4(y-1)(y-3).$$

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

We write res_x to indicate that we consider x as the main variable; there is also $\mathrm{res}_y(f,g) = x^2+1 \in \mathbb{F}_5[x]$. If we now let $\overline{a} = a(-1)$ for $a \in R$, corresponding to $p = y+1$, then $b = \gcd(\mathrm{lc}_x(f), \mathrm{lc}_x(g)) = y$ does not vanish modulo p, $\overline{\mathrm{res}(f/h, g/h)} = \mathrm{res}(f/h, g/h)(-1) \neq 0$, and hence $\deg \gcd(\overline{f}, \overline{g}) = 1$. Actually

$$\overline{\mathrm{lc}(h)} \cdot \gcd(\overline{f}, \overline{g}) = -\gcd(-x^3+x^2-x+1, x^2-2x+1) = -x+1 = h(-1) = \overline{h}.$$

On the other hand, if $\overline{a} = a(1)$ for $a \in R$, then $\overline{b} \neq 0$, $\overline{\mathrm{res}(f/h, g/h)} = 0$, and $\deg \gcd(\overline{f}, \overline{g}) > 1$. In fact, $\gcd(\overline{f}, \overline{g}) = x^2+3x+2 = \overline{g}/3$. $\diamond$

6.5. Modular gcd algorithm in $F[x,y]$

In Section 6.11, we present a modular algorithm that computes all results of the Extended Euclidean Algorithm, including the gcd and the Bézout coefficients. But if just the gcd is required, there is a better way which we now describe.

In a modular algorithm, say with a big prime p, we need two conditions to be satisfied: p has to be large enough so that the coefficients of the gcd can be recovered from their images modulo p, and p should not divide the resultant and the leading coefficient of the gcd, so that its degree does not change modulo p. When both input polynomials have degree about n and coefficients of length n, then the bound for the first condition is $O(n)$, but for the second one it is about n^2. The trick now is to choose *p randomly* so that coefficient recovery is always guaranteed, but the non-divisibility condition only with high probability.

This introduces the important method of **probabilistic algorithms**. Such an algorithm takes an input, makes some random choices (say, chooses several times a bit, either 0 or 1, each with equal probability), does some calculations, and returns an output. If one can prove that the probability of returning the correct output is at least some value greater than $1/2$, say $2/3$, then one can run the algorithm repeatedly, and will obtain the correct answer by a majority vote with probability arbitrarily close to 1. This is called a **Monte Carlo algorithm**. In some applications, as in this chapter, one can easily test the output for correctness. Then the error probability becomes zero, and only the running time is a random variable. This is called a **Las Vegas algorithm**. See Notes 6.5 and Section 25.8 for discussions.

These probabilistic algorithms actually started in computer algebra, with Berlekamp's polynomial factorization (Section 14.8) and Solovay & Strassen's primality test (Section 18.5). Their power and simplicity has made them a ubiquitous tool in many areas of computer science. We have seen examples of probabilistic modular testing in Section 4.1. These methods have an inherent uncertainty, but it can be made arbitrarily small, and thus they are like playing a highly attractive lottery: the stakes are only a tiny fraction of the jackpot (say, polynomial time vs. exponential time), but you are almost guaranteed to win!

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

In a probabilistic algorithm, the error probability bound has to hold no matter what the inputs are. This must not be confused with the *average case analysis* of an algorithm, where the average cost is determined for some reasonable probability distribution of the inputs. This sometimes provides a valuable insight, but its Achilles heel is whether the inputs in a real environment actually follow the assumed distribution.

Now here is our first probabilistic algorithm. It works over a field F where sufficiently many irreducible univariate polynomials of any degree can be found. By $\mathrm{lc}_x(f)$ we denote the leading coefficient of a bivariate polynomial $f \in F[x,y]$ with respect to the variable x.

ALGORITHM 6.28 Modular bivariate gcd: big prime version.
Input: Primitive polynomials $f, g \in F[x,y] = R[x]$, where $R = F[y]$ and F is a field, with $\deg_x f = n \geq \deg_x g \geq 1$ and $\deg_y f, \deg_y g \leq d$.
Output: $h = \gcd(f,g) \in R[x]$.

1. $b \longleftarrow \gcd(\mathrm{lc}_x(f), \mathrm{lc}_x(g))$
2. **repeat**
3. choose a random monic irreducible polynomial $p \in R$ with $\deg p = d + 1 + \deg b$
4. $\overline{f} \longleftarrow f \bmod p, \quad \overline{g} \longleftarrow g \bmod p$
   call the Euclidean Algorithm 3.14 over $R/\langle p \rangle$ to compute the monic $v \in R[x]$ with $\deg_y v < \deg_y p$ and $v \bmod p = \gcd(\overline{f}, \overline{g}) \in (R/\langle p \rangle)[x]$
5. compute $w, f^*, g^* \in R[x]$ of degrees in y less than $\deg_y p$ such that
$$w \equiv bv \bmod p, \quad f^* w \equiv bf \bmod p, \quad g^* w \equiv bg \bmod p \qquad (7)$$
6. **until** $\deg_y(f^* w) = \deg_y(bf)$ and $\deg_y(g^* w) = \deg_y(bg)$
7. **return** $\mathrm{pp}_x(w)$

If also the cofactors f/h and g/h are needed, they can easily be obtained as $\mathrm{pp}_x(f^*)$ and $\mathrm{pp}_x(g^*)$. Before computing f^* and g^* in step 5, one will first test whether the constant coefficient of w divides the constant coefficients of bf and bg, and go back to step 3 if this test fails. We may compute f^* and g^* as $f^* \equiv f/v \bmod p$ and $g^* \equiv g/v \bmod p$.

To compute the gcd of non-primitive polynomials, we first compute the gcd of their contents, then apply the algorithm to their primitive parts, and finally multiply its result by the gcd of the contents. If the gcd of the constant coefficients of

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

f and g is smaller than b, then exchanging the roles of the leading and the constant coefficients decreases the required degree of p.

The remarks above also apply to the modular gcd algorithms 6.34, 6.36, and 6.38 below.

THEOREM 6.29.
Let f,g be an input, $h = \gcd(f,g)$ in $R[x]$, and $r = \operatorname{res}_x(f/h, g/h) \in R = F[y]$. Then r is a nonzero polynomial of degree at most $2nd$, the halting condition in step 6 is satisfied if and only if p does not divide r, and then the correct output is returned in step 7. The cost for one iteration of steps 4 through 6 is no more than $48n^2d^2 + O(nd(n+d))$ or $O(n^2d^2)$ operations in F. If $b = 1$, then the cost is at most $12n^2d^2 + O(nd(n+d))$. Steps 1 and 7 take $O(nd^2)$ operations in F.

PROOF. We have $\gcd(f/h, g/h) = 1$. Since h and f/h divide f, their degrees in y are at most $\deg_y f \le d$, and similarly for g/h, and Corollary 6.20 and Theorem 6.22 yield the first claim. Moreover, $\deg_y b < \deg_y p$, and hence $p \nmid b$. We first assume that $p \nmid r$, and let $\alpha = \operatorname{lc}(h) \in R$. Then Theorem 6.26 implies that $\alpha v \equiv h \bmod p$. Moreover, $\alpha \mid b$, and hence $w \equiv bv \equiv (b/\alpha)h \bmod p$. Both w and $(b/\alpha)h$ have degree in y less than $\deg_y p$, whence they are equal. Similarly, we find that $f^* = bf/w$ and $g^* = bg/w$, and the degree conditions in step 6 are satisfied since all congruences in (7) are in fact equalities. Now h is primitive, by Corollary 6.10, and the algorithm returns the correct result $\operatorname{pp}_x(w) = \operatorname{pp}_x((b/\alpha)h) = h$ in step 7 since h, α, and b are all normalized.

On the other hand, if $p \mid r$, then Theorem 6.26 implies that $\deg_x w = \deg_x v > \deg_x h$. If the degree conditions in step 6 were true, then the congruences in (7) would be equalities, and $\operatorname{pp}(w)$ would be a common divisor of f and g of higher degree in x than $\deg_x h$. This contradiction finishes the correctness proof.

Computationally, nothing happens in reducing f and g modulo p in step 4. The cost for the Euclidean Algorithm is at most $2n^2 + O(n)$ additions and multiplications in $R/\langle p \rangle$, plus at most $n+2$ modular inversions, by Theorem 3.16. The cost for one addition or multiplication in this residue class ring is at most $4(\deg p)^2 + O(\deg p)$ operations in F, by Corollary 4.6. Since $\deg p = d + 1 + \deg b \le 2d + 1$, the total cost for step 4 is at most $32n^2d^2 + O(nd(n+d))$ operations in F (the cost for the modular inversions is subsumed by the "O" term), and only at most $8n^2d^2 + O(nd(n+d))$ if $b = 1$.

By Section 2.4, the cost for the three multiplications by leading coefficients and the two modular divisions in step 5 is at most $4 \deg_x w \cdot (n - \deg_x w) + O(n)$ additions and multiplications modulo p. Since $m(n-m) \le n^2/4$ for all $m \in \mathbb{R}$, this amounts to at most $n^2 + O(n)$ modular operations or $16n^2d^2 + O(nd(n+d))$ operations in F, and only $4n^2d^2 + O(nd(n+d))$ if $b = 1$. Steps 1 and 7 use at most $n+1$ gcds and divisions of polynomials in $F[y]$ of degree at most $2d$, or $O(nd^2)$ operations in F. □

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

We have ignored the cost for finding p in step 3. For a finite field $F = \mathbb{F}_q$, we will discuss this in Section 14.9: Corollary 14.44 implies that this can be done with an expected number of $O^{\sim}(d^2 \log q)$ operations in $\mathbb{F}_q$, and that the expected number of iterations of the algorithm is at most two if $d \geq 4 + 2\log_2 n$. Here the $O^{\sim}$ notation ignores logarithmic factors (Section 25.7).

6.6. Mignotte's factor bound and a modular gcd algorithm in $\mathbb{Z}[x]$

In order to adapt Algorithm 6.28 to $\mathbb{Z}[x]$, we need an a priori bound on the coefficient size of h. Over $F[y]$, the bound

$$\deg_y h \leq \deg_y f \tag{8}$$

is trivial and quite sufficient. Over $\mathbb{Z}$, we could use the subresultant bound of Theorem 6.52 below, but we now derive a much better bound. It actually depends only on one argument of the gcd, say f, and is valid for all factors of f. We will use this again for the factorization of f in Chapter 15.

We extend the **2-norm** to a complex polynomial $f = \sum_{0\leq i\leq n} f_i x^i \in \mathbb{C}[x]$ by $\|f\|_2 = \left(\sum_{0\leq i\leq n} |f_i|^2\right)^{1/2} \in \mathbb{R}$, where $|a| = (a \cdot \overline{a})^{1/2} \in \mathbb{R}$ is the norm of $a \in \mathbb{C}$ and $\overline{a}$ is the complex conjugate of a. We will derive a bound for the norm of factors of f in terms of $\|f\|_2$, that is, a bound $B \in \mathbb{R}$ such that any factor $h \in \mathbb{Z}[x]$ of f satisfies $\|h\|_2 \leq B$. One might hope that we can take $B = \|f\|_2$, but this is not the case. For example, let $f = x^n - 1$ and $h = \Phi_n \in \mathbb{Z}[x]$ be the nth cyclotomic polynomial (Section 14.10). Thus Φ_n divides $x^n - 1$, and the direct analog of (8) would say that each coefficient of Φ_n is at most 1 in absolute value, but for example Φ_{105}, of degree 48, contains the term $-2x^7$. In fact, the coefficients of Φ_n are unbounded in absolute value if $n \longrightarrow \infty$, and hence this is also true for $\|h\|_2$. Worse yet, for infinitely many integers n, Φ_n has a very large coefficient, namely larger than $\exp(\exp(\ln 2 \cdot \ln n / \operatorname{lnln} n))$, where ln is the logarithm in base e; such a coefficient has word length somewhat less than n. It is not obvious how to control the coefficients of factors at all, and it is not surprising that we have to work a little bit to establish a good bound.

LEMMA 6.30. *For $f \in \mathbb{C}[x]$ and $z \in \mathbb{C}$, we have $\|(x-z)f\|_2 = \|(\overline{z}x - 1)f\|_2$.*

PROOF. Writing $f = \sum_{0\leq i\leq n} f_i x^i$ and letting $f_{-1} = f_{n+1} = 0$, we calculate

$$\begin{aligned}\|(x-z)f\|_2^2 &= \sum_{0\leq i\leq n+1} |f_{i-1} - z f_i|^2 = \sum_{0\leq i\leq n+1} (f_{i-1} - z f_i)(\overline{f_{i-1}} - \overline{z}\,\overline{f_i}) \\ &= \|f\|_2^2 (1 + |z|^2) - \sum_{0\leq i\leq n+1} (z\overline{f_{i-1}} f_i + \overline{z} f_{i-1}\overline{f_i})\end{aligned}$$

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

$$= \sum_{0\le i\le n+1} (\overline{z}f_{i-1} - f_i)(z\overline{f_{i-1}} - \overline{f_i}) = \sum_{0\le i\le n+1} |\overline{z}f_{i-1} - f_i|^2$$
$$= \|(\overline{z}x-1)f\|_2^2. \ \square$$

Let

$$f = \sum_{0\le i\le n} f_i x^i = f_n \prod_{1\le i\le n} (x - z_i),$$

with $f_0,\ldots,f_n,z_1,\ldots,z_n \in \mathbb{C}$. We define $M(f) = |f_n| \cdot \prod_{1\le i\le n} \max\{1,|z_i|\}$, and note that $M(f) \ge |\mathrm{lc}(f)|$ and $M(f) = M(g)M(h)$ if $f = gh$ for $g,h \in \mathbb{C}[x]$. The following theorem of Landau (1905) relates $M(f)$ and $\|f\|_2$.

THEOREM 6.31 *Landau's inequality.*
For any $f \in \mathbb{C}[x]$, we have $M(f) \le \|f\|_2$.

PROOF. We arrange the roots so that $|z_1|,\ldots,|z_k| > 1$ and $|z_{k+1}|,\ldots,|z_n| \le 1$ for some $k \in \{0,\ldots,n\}$, so that $M(f) = |f_n \cdot z_1 \cdots z_k|$. Let

$$g = f_n \prod_{1\le i\le k} (\overline{z_i}x - 1) \prod_{k<i\le n} (x - z_i) = g_n x^n + \cdots + g_0 \in \mathbb{C}[x].$$

Then

$$M(f)^2 = |f_n\overline{z_1}\cdots\overline{z_k}|^2 = |g_n|^2 \le \|g\|_2^2 = \left\| \frac{g}{\overline{z_1}x-1}(x-z_1) \right\|_2^2 = \ \cdots$$
$$= \left\| \frac{g}{(\overline{z_1}x-1)\cdots(\overline{z_k}x-1)}(x-z_1)\cdots(x-z_k) \right\|_2^2 = \|f\|_2^2,$$

using repeatedly the previous lemma, and the assertion follows. $\square$

It is convenient to use also the **1-norm** $\|f\|_1 = \sum_{0\le i\le n} |f_i|$, so that $\|f\|_\infty \le \|f\|_2 \le \|f\|_1 \le (n+1)\|f\|_\infty$.

THEOREM 6.32.
If $h = \sum_{0\le i\le m} h_i x^i \in \mathbb{C}[x]$ of degree m divides $f = \sum_{0\le i\le n} f_i x^i \in \mathbb{C}[x]$ of degree $n \ge m$, then

$$\|h\|_2 \le \|h\|_1 \le 2^m M(h) \le \left| \frac{h_m}{f_n} \right| 2^m \|f\|_2.$$

PROOF. We write $h = h_m \prod_{1\le i\le m}(x - u_i)$ with $u_i \in \mathbb{C}$ for $1 \le i \le m$, and note that each u_i equals some root z_j of f. By Viète's rule, we can express the coefficients of h in terms of its roots as

$$h_i = (-1)^{m-i} h_m \sum_{\substack{S\subseteq\{1,\ldots,m\} \\ \#S = m-i}} \prod_{j\in S} u_j,$$

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

where the sum is the $(m-i)$th elementary symmetric polynomial in $u_1,\dots,u_m$, and hence

$$|h_i| \le |h_m| \cdot \sum_S \prod_{j\in S} |u_j| \le \binom{m}{i} M(h)$$

for $0 \le i \le m$. Thus

$$\|h\|_2 \le \|h\|_1 = \sum_{0\le i\le m} |h_i| \le 2^m M(h) \le \left|\frac{h_m}{f_n}\right| 2^m M(f) \le \left|\frac{h_m}{f_n}\right| 2^m \|f\|_2,$$

by the sum formula for the binomial coefficients and Landau's inequality. □

COROLLARY 6.33 *Mignotte's bound.*
Suppose that $f,g,h \in \mathbb{Z}[x]$ have degrees $\deg f = n \ge 1$, $\deg g = m$, $\deg h = k$, and that gh divides f (in $\mathbb{Z}[x]$). Then

(i) $\|g\|_\infty \|h\|_\infty \le \|g\|_2 \|h\|_2 \le \|g\|_1 \|h\|_1 \le 2^{m+k}\|f\|_2 \le (n+1)^{1/2} 2^{m+k} \|f\|_\infty$,

(ii) $\|h\|_\infty \le \|h\|_2 \le 2^k \|f\|_2 \le 2^k \|f\|_1$ *and* $\|h\|_\infty \le \|h\|_2 \le (n+1)^{1/2} 2^k \|f\|_\infty$.

PROOF. By Theorem 6.32 and Landau's inequality, we have

$$\|g\|_1 \|h\|_1 \le 2^{m+k} M(g) M(h) \le 2^{m+k} M(f) \le 2^{m+k} \|f\|_2.$$

This proves (i), and (ii) follows by taking $g = 1$. □

Suppose that the polynomials $f,g \in \mathbb{Z}[x]$ have degrees $n = \deg f \ge \deg g$ and max-norm $\|f\|_\infty, \|g\|_\infty$ at most A. Then the max-norm of $\gcd(f,g) \in \mathbb{Z}[x]$ is at most $(n+1)^{1/2} 2^n A$, by Corollary 6.33. We now have the following algorithm for computing a gcd in $\mathbb{Z}[x]$, completely analogous to Algorithm 6.28.

ALGORITHM 6.34 Modular gcd in $\mathbb{Z}[x]$: big prime version.
Input: Primitive polynomials $f,g \in \mathbb{Z}[x]$ with $\deg f = n \ge \deg g \ge 1$ and max-norm $\|f\|_\infty, \|g\|_\infty \le A$.
Output: $h = \gcd(f,g) \in \mathbb{Z}[x]$.

1. $b \longleftarrow \gcd(\mathrm{lc}(f), \mathrm{lc}(g))$, $\quad B \longleftarrow (n+1)^{1/2} 2^n A b$
2. **repeat**
3. choose a random prime $p \in \mathbb{N}$ with $2B < p \le 4B$
4. $\overline{f} \longleftarrow f \bmod p$, $\quad \overline{g} \longleftarrow g \bmod p$
   call the Euclidean Algorithm 3.14 over $\mathbb{Z}_p$ to compute the monic v in $\mathbb{Z}[x]$ with $\|v\|_\infty < p/2$ such that $v \bmod p = \gcd(\overline{f},\overline{g}) \in \mathbb{Z}_p[x]$

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

5. compute $w, f^*, g^* \in \mathbb{Z}[x]$ of max-norm less than $p/2$ such that

$$w \equiv bv \bmod p, \quad f^* w \equiv bf \bmod p, \quad g^* w \equiv bg \bmod p \tag{9}$$

6. **until** $\|f^*\|_1 \|w\|_1 \leq B$ and $\|g^*\|_1 \|w\|_1 \leq B$

7. **return** $\mathrm{pp}(w)$

In step 3, we need primes satisfying certain conditions. We do not yet have the tools to solve this task, and postpone its discussion to Section 18.4. The following is analogous to Theorem 6.29.

THEOREM 6.35.
Let h be the normalized gcd *of f and g in $\mathbb{Z}[x]$, so that $\mathrm{lc}(h) > 0$. Then $r = \mathrm{res}(f/h, g/h)$ is a nonzero integer with $|r| \leq (n+1)^n A^{2n}$, the halting condition in step 6 is true if and only if p does not divide r, and then the output in step 7 is correct. The cost for one execution of steps 4 and 5 is $O(n^2(n^2 + \log^2 A))$ word operations, and steps 1 and 7 take $O(n(n^2 + \log^2 A))$ word operations.*

PROOF. For the correctness, it is sufficient to see that the condition in step 6 holds if and only if $\mathrm{pp}(w) = h$. If the condition holds, then $\|f^* w\|_\infty \leq \|f^* w\|_1 \leq \|f^*\|_1 \|w\|_1 \leq B < p/2$, and $\|bf\|_\infty < p/2$ and $f^* w \equiv bf \bmod p$ imply that $f^* w = bf$. Similarly, we find $g^* w = bg$. Thus $w \mid \gcd(bf, bg)$, and Theorem 6.26 (ii) implies that $\deg w = \deg \gcd(bf, bg)$, and hence $\mathrm{pp}(w) = \gcd(f, g)$ since both polynomials are normalized.

On the other hand, if $\mathrm{pp}(w) = \gcd(f, g)$ with the w calculated in step 5, then w divides bf, Mignotte's bound 6.33 shows that $\|bf/w\|_\infty \leq B < p/2$, and hence the congruence $f^* \equiv bf/w \bmod p$ is an equality. Similarly, we find $g^* = bg/w$, and another application of Corollary 6.33 implies the condition in step 6. Exercise 6.25 shows that $p \nmid r$ if and only if $\mathrm{pp}(w) = h$.

With $k = \deg h$, we have $\|f/h\|_2, \|g/h\|_2 \leq (n+1)^{1/2} 2^{n-k} A$, again by Corollary 6.33, and Theorem 6.23 gives $|r| \leq 4^{n^2} (n+1)^n A^{2n}$; Exercise 6.24 yields the better bound stated in the theorem. Step 4 takes $O(n^2)$ arithmetic operations in $\mathbb{Z}_p$, and the cost for each of these is $O(\log^2 p)$ word operations. Now $\log p \leq \log(4B) \in O(n + \log A)$, whence step 4 uses $O(n^2(n^2 + \log^2 A))$ word operations, and the same bound holds for the divisions in step 5. Steps 1 and 7 take $O(n)$ gcd's and divisions on integers of length $O(n + \log A)$, or $O(n(n^2 + \log^2 A))$ word operations. □

In Section 18.4, we show that we can find a random number p between $2B$ and $4B$ such that p is prime and $p \nmid r$ with probability at least $1/2$ by a probabilistic algorithm using $O^\sim(\log^3 B)$ or $O^\sim(n^3 + \log^3 A)$ word operations (Corollary 18.11). Then the expected number of iterations of the algorithm is at most two.

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

6.7. Small primes modular gcd algorithms

We have seen in Section 5.5 that the small primes modular approach for computing the determinant is computationally superior to the big prime scheme. The reason that we have discussed big prime modular gcd algorithms at all in the preceding sections is that they are easier and the main idea is more clearly visible than for their small prime variants that we will present now. In practice, we strongly recommend the use of the latter. We start with the algorithm for $F[x,y]$ since it is simpler to describe and analyze than the corresponding algorithm for $\mathbb{Z}[x]$.

ALGORITHM 6.36 Modular bivariate gcd: small primes version.
Input: Primitive polynomials $f,g \in F[x,y] = R[x]$ with $\deg_x f = n \geq \deg_x g \geq 1$ and $\deg_y f, \deg_y g \leq d$, where $R = F[y]$ for a field F with at least $(4n+2)d$ elements.
Output: $h = \gcd(f,g) \in R[x]$.

1. $b \longleftarrow \gcd(\mathrm{lc}_x(f), \mathrm{lc}_x(g)), \quad l \longleftarrow d + 1 + \deg_y b$
2. **repeat**
3. choose a set $S \subseteq F$ of $2l$ evaluation points
4. $S \longleftarrow \{u \in S : b(u) \neq 0\}$
   for each $u \in S$ **call** the Euclidean Algorithm 3.14 over F to compute the monic $v_u = \gcd(f(x,u), g(x,u)) \in F[x]$
5. $e \longleftarrow \min\{\deg v_u : u \in S\}, \quad S \longleftarrow \{u \in S : \deg v_u = e\}$
   if $\#S \geq l$ **then** remove $\#S - l$ elements from S **else goto** 3
6. compute by interpolation each coefficient in $F[y]$ of the polynomials $w, f^*, g^* \in R[x]$ of degrees in y less than l such that

$$w(x,u) = b(u)v_u,$$

$$f^*(x,u)w(x,u) = b(u)f(x,u), \quad g^*(x,u)w(x,u) = b(u)g(x,u)$$

   for all $u \in S$
7. **until** $\deg_y(f^*w) = \deg_y(bf)$ and $\deg_y(g^*w) = \deg_y(bg)$
8. **return** $\mathrm{pp}_x(w)$

In practice, one will choose the points from F adaptively, starting with about l or even fewer elements in S, remove "unlucky" points that are detected in steps 4, 5, or 7 from S, and add some new random points to S if the condition in step 7 is violated. If the gcd is constant, then only one "lucky" point is sufficient to detect this. The analysis of the above algorithm is somewhat easier, though. These remarks also apply to Algorithm 6.38 below.

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

THEOREM 6.37.
Algorithm 6.36 correctly computes the gcd *of f and g. One iteration of the loop uses at most $10n^2d + 36nd^2 + O((n+d)d)$ arithmetic operations in F, and only $5n^2d + 13nd^2 + O((n+d)d)$ if $b = 1$. If $d \geq 1$ and we choose S in step 3 as a uniform random subset with $2l$ elements of a fixed finite set $U \subseteq F$ of cardinality $\#U \geq (4n+2)d$, then the expected number of iterations is at most 2. The cost for steps 1 and 8 is at most $10nd^2 + O(nd)$ operations in F, or even $\frac{5}{2}nd^2 + O((n+d)d)$ if $b = 1$.*

PROOF. We have $\deg_y b \leq d$, $l \leq 2d+1$, and $\#U \geq 6d \geq 2l$, which implies that enough points can be chosen in step 3. The correctness then follows as in the proof of Theorem 6.29. In step 4, the cost per u is $O(d)$ operations in F for evaluating b at it, at most $4nd + O(n+d)$ operations in F for evaluating (all coefficients of) f and g at $y = u$, plus at most $2n^2 + O(n)$ for the gcd, by Theorem 3.16, in total $4n^2l + 8ndl + O((n+d)d)$ operations for all $u \in S$. In step 6, we first compute the modular cofactors $f(x,u)/v_u$ and $g(x,u)/v_u$ for all $u \in S$. As in the proof of Theorem 6.29, this takes at most $n^2 + O(n)$ operations in F per point u, altogether no more than $n^2l + O(nl)$ operations. Then we solve an interpolation problem at l points for each of the at most $2n+2$ coefficients of w, f^*, and g^*, taking a total of $5nl^2 + O((n+d)d)$ operations in F, by Exercise 5.11. Thus one iteration of the loop 2 takes $5n^2l + 8ndl + 5nl^2 + O((n+d)d)$ operations in F. The cost for steps 1 and 8 is at most $n+1$ gcd's of polynomials of degree at most l, plus the same number of divisions, in total no more than $\frac{5}{2}nl^2 + O((n+d)d)$ operations.

It remains to determine the expected number of iterations. The primes in the algorithm are linear polynomials $y - u$. Let $r = \operatorname{res}_x(f/h, g/h) \in F[y]$. As in the big prime approach, the halting condition in step 7 is satisfied if and only if $y - u$ does not divide the polynomial r, or equivalently, $r(u) \neq 0$, for all $u \in S$. Thus the choice of S in step 3 is successful if and only if the polynomial br does not vanish for at least l points in S. We will say that $u \in U$ is **unlucky** if $b(u)r(u) = 0$. The degree of br in y is at most $(2n+1)d$, by Theorem 6.29. Since U has at least $(4n+2)d$ elements, at most half of the points in U are unlucky. Thus the probability that at most half of the points in S in step 3 are unlucky is at least $1/2$ (Exercise 6.31). Therefore the expected number of iterations of the algorithm is at most two, and the claim follows. □

By increasing the size of U, the failure probability in a single run can be reduced and the expected number of iterations of the algorithm can be brought down arbitrarily close to one. A variant of the algorithm is analyzed in Section 24.3.

The running time of the small primes modular gcd algorithm is better by about one order of magnitude than for the big prime variant when $n \approx d$. If fast polynomial arithmetic, as described in Part II, is used, then the cost even drops to $O^\sim(nd)$ (Corollary 11.12).

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

When F does not have sufficiently many elements, say when $F = \mathbb{F}_2$, then we have a problem in step 3. This can be circumvented by either making a suitable field extension, which increases all timings by a factor of $O(\log^2(nd))$ (Exercise 6.32), or by choosing nonlinear moduli.

Here is the analogous algorithm for $\mathbb{Z}[x]$. We denote the natural (base e) logarithm by ln.

ALGORITHM 6.38 Modular gcd in $\mathbb{Z}[x]$: small primes version.
Input: Primitive polynomials $f, g \in \mathbb{Z}[x]$ with $\deg f = n \geq \deg g \geq 1$ and max-norm $\|f\|_\infty, \|g\|_\infty \leq A$.
Output: $h = \gcd(f, g) \in \mathbb{Z}[x]$.

1. $b \longleftarrow \gcd(\mathrm{lc}(f), \mathrm{lc}(g)), \quad k \longleftarrow \left\lceil 2\log_2\left((n+1)^n b A^{2n}\right)\right\rceil$
   $B \longleftarrow (n+1)^{1/2} 2^n A b, \quad l \longleftarrow \lceil \log_2(2B+1) \rceil$
2. **repeat**
3. choose a set S of $2l$ primes, each less than $2k \ln k$
4. $S \longleftarrow \{p \in S : p \nmid b\}$
   for each $p \in S$ **call** the Euclidean Algorithm 3.14 over $\mathbb{Z}_p$ to compute the monic $v_p \in \mathbb{Z}[x]$ with coefficients in $\{0, \ldots, p-1\}$ such that $v_p \bmod p = \gcd(\overline{f}, \overline{g}) \in \mathbb{Z}_p[x]$, where the bar indicates reduction of each coefficient modulo p
5. $e \longleftarrow \min\{\deg v_p : p \in S\}, \quad S \longleftarrow \{p \in S : \deg v_p = e\}$
   if $\#S \geq l$ **then** remove $\#S - l$ elements from S **else goto** 3
6. **call** the Chinese Remainder Algorithm 5.4 to compute each coefficient of the unique polynomials $w, f^*, g^* \in \mathbb{Z}[x]$ with max-norms less than $(\prod_{p \in S} p)/2$ and
   $$w \equiv b v_p \bmod p, \quad f^* w \equiv b f \bmod p, \quad g^* w \equiv b g \bmod p$$
   for all $p \in S$
7. **until** $\|f^*\|_1 \|w\|_1 \leq B$ and $\|g^*\|_1 \|w\|_1 \leq B$
8. **return** $\mathrm{pp}(w)$

In step 3, we need primes satisfying certain conditions. This is discussed in Section 18.4. In practice, $2k \ln k$ will be less than the word size of the processor, and we may take (possibly precomputed) single-word primes (just under 2^{32} or 2^{64}, depending on the computer's word size).

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

THEOREM 6.39.
Algorithm 6.38 works correctly. One execution of steps 4 through 7 can be performed with $O(n(n^2+\log^2 A)(\log n+\log\log A)^2)$ word operations, and the same estimate holds for steps 1 and 8.

PROOF. Correctness follows as in the proof of Theorem 6.35. For the running time estimate, we first note that $\log p \in O(\log k)$ for each prime $p \in S$. In step 4, the cost per prime p is $O(n \log A \cdot \log k)$ word operations for reducing b and all coefficients of f and g modulo p, and $O(n^2)$ operations in $\mathbb{Z}_p$ or $O(n^2 \log^2 k)$ word operations for the gcd, a total of $O(n(n \log k + \log A) l \log k)$ word operations. In step 6, we perform two divisions with remainder f/v_p and g/v_p modulo p for each $p \in S$, taking $O(n^2 \log^2 k)$ word operations, and then apply the Chinese Remainder Algorithm to each of the at most $2n+2$ coefficients of w, f^*, and g^*. We have $\log \prod_{p \in S} p = \sum_{p \in S} \log p \in O(l \log k)$, Theorem 5.8 implies that the cost for each coefficient is $O(l^2 \log^2 k)$ word operations, and the cost for all coefficients is $O(nl^2 \log^2 k)$. The cost for steps 1 and 8 is as in Theorem 6.35. We have $l \in O(n + \log A)$ and $\log k \in O(\log n + \log\log A)$, and the claims follow. □

As in the polynomial case, the cost estimate for the small primes algorithm is smaller by about one order of magnitude as for the big prime variant. If we use single precision primes, then the cost is about $O(nl(n+l))$ word operations. In Section 11.1, we will show that the cost drops to $O^\sim(n^2 + n \log A)$ when using the fast methods for polynomial and integer arithmetic from Part II. In Section 18.4, we show that the first k primes $p_1 = 2, \ldots, p_k$ can be computed deterministically by the sieve of Eratosthenes, taking $O(k \log^2 k \log\log k)$ or $O^\sim(n(n + \log A))$ word operations, and that each of them is at most $2k \ln k$. The value k is an upper bound on $2 \log_2 |b \operatorname{res}(f/h, g/h)|$, by Theorem 6.35, and this guarantees that at least $k/2$ of our k primes do not divide $b \operatorname{res}(f/h, g/h)$. By Theorem 6.26, at least half of the primes $p_1, \ldots, p_k$ are "lucky". We have $2l \leq k$, and if we choose the set S as a uniform random subset with $2l$ elements of $\{p_1, \ldots, p_k\}$ in step 3, then Exercise 6.31 shows that at least l of the primes in S are lucky with probability no less than $1/2$. As in the bivariate case, the condition in step 7 is satisfied if and only if at least l primes are lucky, and the expected number of iterations of the algorithm is at most two.

In practice, one would use an adaptive approach as described for the bivariate case, in particular since the Mignotte bound on the coefficients of h is often too large. We present running times of such an implementation in Section 6.13.

6.8. Application: intersecting plane curves

This and the following section discuss two applications of resultants which are not used later.

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

The historical purpose of the resultant was to solve geometric problems by elimination of variables. As an example, we want to determine the common roots of two polynomials in two variables, or, equivalently, the intersection of two plane curves. Suppose we are given $f, g \in F[x,y]$, where F is a field, and want to intersect the two plane curves

$$X = \{(a,b) \in F^2 : f(a,b) = 0\}, \quad Y = \{(a,b) \in F^2 : g(a,b) = 0\}.$$

We eliminate the variable y by considering the resultant $r = \operatorname{res}_y(f,g) \in F[x]$ with respect to y. We assume that F is **algebraically closed**, so that every nonconstant univariate polynomial over F has a root. This is often required to make general statements about geometric objects such as our curves X and Y true. The reader may imagine that $F = \mathbb{C}$. Now we let Z be the projection of $X \cap Y$ onto the x-axis. If $a \in F$ and $\operatorname{lc}_y(f), \operatorname{lc}_y(g)$ do not both vanish at $x = a$, then Lemma 6.25 implies that

$$\begin{aligned} a \in Z &\iff \exists b \in F\ (a,b) \in X \cap Y \iff \exists b \in F\ f(a,b) = g(a,b) = 0 \\ &\iff \gcd(f(a,y), g(a,y)) \neq 1 \\ &\iff r(a) = \operatorname{res}_y(f,g)(a) = 0. \end{aligned}$$

Thus to determine $X \cap Y$, we first compute the $(d+e) \times (d+e)$ determinant over $F[x]$ giving $r \in F[x]$, where d and e are the degrees in y of f and g, respectively, then find all roots of r, and for each such root a we find all the roots $b \in F$ of $\gcd(f(a,y), g(a,y)) \in F[y]$. The fact that $r(a) = 0$ guarantees that such b's exist if not both leading coefficients with respect to y vanish at $x = a$. In other words, Z is contained in the set of roots of r.

This means that intersecting two plane curves is reduced to finding roots of univariate polynomials, a much easier task. (For $F = \mathbb{Q}$ or a finite field, we will solve this task in Part III.) If $n = \deg f$, $m = \deg g$, where deg is the total degree, and $\gcd(f,g) = 1$ in $F[x,y]$, then $X \cap Y$ has "in general" nm points. This is called **Bézout's theorem** after the French geometer Étienne Bézout. It is valid for arbitrary algebraic varieties, provided one counts points "at infinity" and "multiple points and components" properly, and all intersection components have the "correct" dimension.

If $\gcd(f,g) = h$ for some nonconstant $h \in F[x,y]$, then r is the zero polynomial, all points of the curve $\{h = 0\}$ belong to $X \cap Y$, and the intersection is infinite. A trivial example is when $f = h = g$.

EXAMPLE 6.40. We start with a simple example, illustrated in Figure 6.2 on page 174. Let $f = x^2 + y^2 - 1$ and $g = 3x + 4y$ in $\mathbb{C}[x]$. Then $X = \{(a,b) \in \mathbb{C}^2 : f(a,b) = 0\}$ is a circle of radius 1 centered at the origin (blue), and $Y = \{(a,b) \in \mathbb{C}^2 : g(a,b) = 0\}$ is a line with slope $-3/4$ through the origin (green).

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

Bézout's theorem says that there are $\deg f \cdot \deg g = 2$ intersection points; they are depicted in Figure 6.2 (black). We might compute them by solving $g = 0$ for y and plugging this into $f = 0$, but let us proceed systematically to illustrate the resultant method. We have

$$\operatorname{res}_y(f,g) = \det \begin{pmatrix} 1 & 4 & 0 \\ 0 & 3x & 4 \\ x^2-1 & 0 & 3x \end{pmatrix} = 25x^2 - 16,$$

and the projection Z of $X \cap Y$ onto the x axis consists of the two zeroes $Z = \{4/5, -4/5\}$ of $\operatorname{res}_y(f,g)$ (red). We obtain the corresponding values for y by taking gcds:

$$\begin{aligned} \gcd\left(f\left(\frac{4}{5},y\right), g\left(\frac{4}{5},y\right)\right) &= \gcd\left(y^2 - \frac{9}{25}, 4y + \frac{12}{5}\right) &&= y + \frac{3}{5}, \\ \gcd\left(f\left(-\frac{4}{5},y\right), g\left(-\frac{4}{5},y\right)\right) &= \gcd\left(y^2 - \frac{9}{25}, 4y - \frac{12}{5}\right) &&= y - \frac{3}{5}, \end{aligned}$$

and hence the two intersection points are

$$X \cap Y = \left\{ \left(\frac{4}{5}, -\frac{3}{5}\right), \left(-\frac{4}{5}, \frac{3}{5}\right) \right\}. \diamond$$

EXAMPLE 6.41. We consider the two plane curves $X, Y \subseteq \mathbb{C}^2$ given by the two polynomials

$$f = (y^2+6)(x-1) - y(x^2+1),\ g = (x^2+6)(y-1) - x(y^2+1) \in \mathbb{Z}[x,y].$$

Exchanging x and y corresponds to a swap of f and g, and hence the whole situation is symmetric with respect to this exchange.

Before any calculations, let us look at a picture, in Figure 6.3 on page 174. This is easy to generate with MAPLE's `implicitplot` command. The projection Z of $X \cap Y$ onto the x-axis is readily calculated: the resultant

$$\begin{aligned} \operatorname{res}_y(f,g) &= 2x^6 - 22x^5 + 102x^4 - 274x^3 + 488x^2 - 552x + 288 \\ &= 2(x-2)^2(x-3)^2(x^2 - x + 4) \end{aligned}$$

has the four distinct roots in $Z = \{2, 3, (1 \pm \sqrt{15}i)/2\}$, where $i = \sqrt{-1}$. For each of these values for x we calculate the corresponding values for y:

$$\begin{aligned} \gcd(f(2,y), g(2,y)) &= \gcd(y^2 - 5y + 6, -2y^2 + 10y - 12) \\ &= y^2 - 5y + 6 = (y-2)(y-3), \\ \gcd(f(3,y), g(3,y)) &= \gcd(2y^2 - 10y + 12, -3y^2 + 15y - 18) \\ &= y^2 - 5y + 6 = (y-2)(y-3), \end{aligned}$$

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

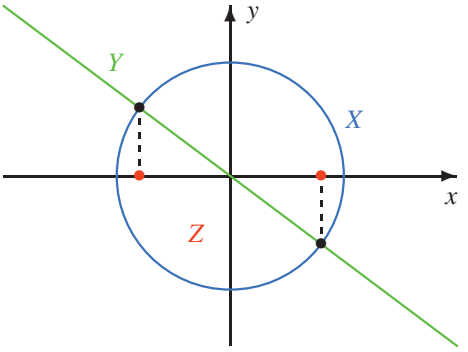


FIGURE 6.2: Intersection of the circle X with the line Y.

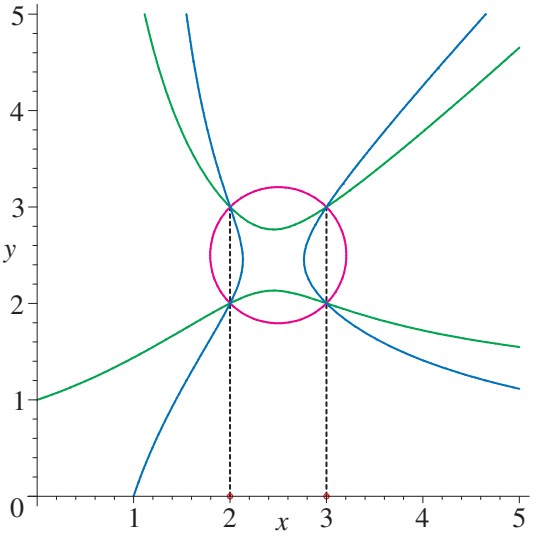


FIGURE 6.3: The three curves $f = 0$ (blue), $g = 0$ (green), and $f + g = 0$ (pink), and the projection of their intersection points to the x-axis (red).

and

$$\gcd\left(f\Big(\frac{1\pm\sqrt{15}i}{2},y\Big),g\Big(\frac{1\pm\sqrt{15}i}{2},y\Big)\right)=\Big(y-\frac{1\mp\sqrt{15}i}{2}\Big).$$

Thus $X \cap Y$ consists of the six points

$$\Big\{(2,2),(2,3),(3,2),(3,3),\Big(\frac{1+\sqrt{15}i}{2},\frac{1-\sqrt{15}i}{2}\Big),\Big(\frac{1-\sqrt{15}i}{2},\frac{1+\sqrt{15}i}{2}\Big)\Big\}.$$

Only the four real points are visible in Figure 6.3. Staring a bit at the equations,

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

one observes that in $f+g$ the terms of degree 3 cancel, and in fact $f+g=0$ is the equation of a circle, the pink curve in Figure 6.3.

Bézout's theorem says that $X \cap Y$ consists of $3 \cdot 3 = 9$ points. We only found six of them. Where are the others? This book's margin is too narrow to contain them, because they lie at infinity! $\diamond$

EXAMPLE 6.42. As a simple example, we take our two curves to be lines: $X = \{(a,b): ua+vb+w=0\}$, $Y = \{(a,b): pa+qb+r=0\}$, where $u,v,w,p,q,r, \in F$ are given. The corresponding resultant is

$$\begin{aligned} \operatorname{res}_y(ux+vy+w, px+qy+r) &= \det \begin{pmatrix} v & q \\ ux+w & px+r \end{pmatrix} \\ &= (vp-uq)x + (vr-wq) \in F[x]. \end{aligned}$$

This linear polynomial has a root if and only if either the leading coefficient $vp-uq$ is nonzero (then $X \cap Y$ consists of one point), or if it and $vr-wq$ both vanish (then $X=Y$). $\diamond$

The general theory of linear algebra generalizes this well-known criterion for simultaneous solvability of two linear equations in two variables. In a similar way, *geometric elimination theory* tries to generalize our curve intersection method to higher dimensions. This is a much more difficult problem, and the current algorithmic methods are feasible only for a fairly small number of variables. We give an introduction to one successful method in Chapter 21: Gröbner bases.

We give a further application of resultants from the theory of algebraic field extensions. Suppose we have two elements α, β in an algebraic extension E of a field F, with minimal polynomials $f, g \in F[x]$, respectively (Section 25.3). How can we find the minimal polynomial h of $\alpha+\beta$? Since $(\alpha+\beta, \beta) \in E^2$ is a common zero of $g(y)$ and of $f(x-y)$, the resultant $r = \operatorname{res}_y(f(x-y), g(y)) \in F[x]$ is nonconstant and has $\alpha+\beta$ as a root. Thus h is a factor of r.

EXAMPLE 6.43. We let $F=\mathbb{Q}$, $\alpha = i = \sqrt{-1}$, $\beta = \sqrt{3}$. Then $f = x^2+1$, $g = x^2-3$, $f(x-y) = y^2-2xy+x^2+1$, and

$$r = \operatorname{res}_y(f(x-y), g(y)) = \det \begin{pmatrix} 1 & 0 & 1 & 0 \\ -2x & 1 & 0 & 1 \\ x^2+1 & -2x & -3 & 0 \\ 0 & x^2+1 & 0 & -3 \end{pmatrix} = x^4 - 4x^2 + 16.$$

This polynomial is irreducible in $\mathbb{Q}[x]$, as can be checked by verifying that it has no monic linear or quadratic factors in $\mathbb{Z}[x]$, and hence it is the minimal polynomial of $\alpha+\beta = i+\sqrt{3}$. Its four complex roots are $\pm i \pm \sqrt{3}$. $\diamond$

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

6.9. Nonzero preservation and the gcd of several polynomials

In this section, we discuss the following problem: Given nonzero polynomials $f_1,\dots,f_n \in F[x]$ over a field F, compute $h = \gcd(f_1,\dots,f_n)$. Let $d \in \mathbb{N}$ be such that $\deg f_i \le d$ for all i. We are particularly interested in the case where d is close to n. A simple approach is to set $h_1 = f_1$ and compute $h_i = \gcd(h_{i-1}, f_i)$ for $i = 2,\dots,n$. If $\deg h$ is fairly large, say $d/10$, then this will take $n-1$ gcd calculations of polynomials of degree at least $d/10$.

We now present a more efficient algorithm that uses only one gcd calculation. The basic tool for this probabilistic algorithm is the following useful lemma. It says that a nonzero polynomial is likely to take a nonzero value at a random point. In other words, random evaluations probably preserve nonzeroness.

LEMMA 6.44. *Let R be an integral domain, $n \in \mathbb{N}$, $S \subseteq R$ finite with $s = \#S$ elements, and $r \in R[x_1,\dots,x_n]$ a polynomial of total degree at most $d \in \mathbb{N}$.*

(i) If r is not the zero polynomial, then r has at most ds^{n-1} zeroes in S^n.

(ii) If $s > d$ and r vanishes on S^n, then $r = 0$.

PROOF. (i) We prove the claim by induction on n. The case $n = 1$ is clear, since a nonzero univariate polynomial of degree at most d over an integral domain has at most d zeroes (Lemma 25.4). For the induction step, we write r as a polynomial in x_n with coefficients in $x_1,\dots,x_{n-1}$: $r = \sum_{0\le i\le k} r_i x_n^i$ with $r_i \in R[x_1,\dots,x_{n-1}]$ for $0 \le i \le k$ and $r_k \neq 0$. Then $\deg r_k \le d-k$, and by the induction hypothesis, r_k has at most $(d-k)s^{n-2}$ zeroes in S^{n-1}, so that there are at most $(d-k)s^{n-1}$ common zeroes of r and r_k in S^n. Furthermore, for each $a \in S^{n-1}$ with $r_k(a) \neq 0$, the univariate polynomial $r_a = \sum_{0\le i\le k} r_i(a)x_n^i \in R[x_n]$ of degree k has at most k zeroes, so that the total number of zeroes of r in S^n is bounded by

$$(d-k)s^{n-1} + ks^{n-1} = ds^{n-1}.$$

(ii) follows immediately from (i). □

In the example $r = \prod_{1\le i\le d}(x_n - a_i)$, where $a_1,\dots,a_d \in S$ are distinct, the bound in (i) is achieved. A typical application of (i) is in the analysis of probabilistic algorithms, rewriting it as

$$\operatorname{prob}\{r(a) = 0 : a \in S^n\} \le \frac{d}{\#S}, \tag{10}$$

where a is chosen in S^n uniformly at random. In the applications, the tricky part is usually to show that r is nonzero. An amazing fact is that this bound on the probability is independent of the number of variables.

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

For the probabilistic algorithm below, we assume that we have a finite set $S \subseteq F$ and a "random element generator for S", which produces a uniform random member of S. Instead of computing many gcds, it just uses one.

ALGORITHM 6.45 Gcd of many polynomials.
Input: $f_1, \ldots, f_n \in F[x]$, where F is a field.
Output: $h \in F[x]$ monic, so that $h = \gcd(f_1, \ldots, f_n)$ is probably true.

1. choose $a_3, \ldots, a_n \in S$ independently at random
2. $g \longleftarrow f_2 + \sum_{3 \le i \le n} a_i f_i$
3. **return** $\gcd(f_1, g)$

Lemma 6.44 yields the following theorem.

THEOREM 6.46.
Suppose that $\deg f_i \le d$ *for each* i, *and* $h^* = \gcd(f_1, \ldots, f_n)$. *Then the algorithm uses at most* $2(n-2)(d+1) + 2d^2 + O(d)$ *operations in* F, h^* *divides* h, *and* $\text{prob}\{h \ne h^*\} \le d/\#S$.

PROOF. The cost estimate is immediate from Theorem 3.16. Since h^* divides $f_1, \ldots, f_n$, it divides g and $\gcd(f_1, g) = h$. It remains to establish the bound on the error probability.

Dividing each f_i by h^* if necessary, we may assume that $\gcd(f_1, \ldots, f_n) = 1$, and also that $f_1 \ne 0$. Let $A_3, \ldots, A_n$ be new indeterminates over $F(x)$, $R = F[A_3, \ldots, A_n]$, $K = F(A_3, \ldots, A_n)$ the field of fractions of R, $G = f_2 + \sum_{3 \le i \le n} A_i f_i \in R[x]$, and $r = \text{res}_x(f_1, G) \in R$. Then r is a polynomial in $A_3, \ldots, A_n$ of degree at most d, and Lemma 6.25, applied to the ideal $I = \langle A_3 - a_3, \ldots, A_n - a_n \rangle$ for which $R/I \cong F$, shows that

$$r(a_3, \ldots, a_n) \ne 0 \iff \text{res}_x(f_1, g) \ne 0 \iff \gcd(f_1, g) = 1.$$

In order to apply Lemma 6.44, we have to show that r is not the zero polynomial. Let u be a common divisor of f_1 and G in $R[x]$. Since u divides f_1, its coefficients lie in the splitting field E of f_1 over F. But $E[x] \cap K[x] = F[x]$, and hence $u \in F[x]$. If we think of G as a linear polynomial in $A_3, \ldots, A_n$ with coefficients in $F[x]$, then $u \mid G$ implies that u divides the coefficients of G in that representation, so that u divides $f_1, \ldots, f_n$. Since $\gcd(f_1, \ldots, f_n) = 1$, it follows that $u \in F$. Therefore the gcd of f_1 and G in $R[x]$ is a constant. By Corollary 6.20, r is not the zero element of R, and Lemma 6.44 yields the bound on the error probability. □

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

The $2d^2 + O(d)$ from the time bound can be replaced by $O(d \log^2 d \cdot \operatorname{loglog} d)$ when using the fast Euclidean Algorithm from Chapter 11. The dominating cost of about dn for Algorithm 6.45 is unavoidable, since this is the input size.

In practice, one would choose f_1 to have minimal degree among $f_1, \ldots, f_n$. To reduce the error probability to zero, we can in addition compute the remainders $f_1 \operatorname{rem} h, \ldots, f_n \operatorname{rem} h$; this is somewhat cheaper than n gcds, and $h = h^*$ if and only if all remainders are zero. In the rare event that they are not, one can rerun the algorithm with h and these remainders. This is particularly useful for computing the primitive part of a bivariate polynomial with respect to one variable, since then the quotients $f_1/h^*, \ldots, f_n/h^*$ are needed anyway.

Using the Extended Euclidean Algorithm in step 3 yields a representation of h as a linear combination of $f_1, \ldots, f_n$ (Exercise 6.38). Using Algorithm 6.45, one can also compute the least common multiple of several polynomials (Exercise 6.39).

6.10. Subresultants

In this section, we extend the resultant theory—which governs the gcd—to the subresultants which cover all results of the Extended Euclidean Algorithm. As before, this leads to efficient modular methods, but now for the whole algorithm. The reader only interested in efficient gcd algorithms may skip this and proceed directly to the implementation report in Section 6.13.

So now let F be an arbitrary field, and $f, g \in F[x]$ nonzero of degrees $n \geq m$, respectively. We use the notation for the results of the Extended Euclidean Algorithm, as in (1) on page 141, and $n_i = \deg r_i$ for $0 \leq i \leq \ell + 1$, with $r_{\ell+1} = 0$ and $\deg r_{\ell+1} = -\infty$.

THEOREM 6.47.
Let $0 \leq k \leq m \leq n$. Then k does not *appear in the degree sequence if and only if there exist $s, t \in F[x]$ satisfying*

$$t \neq 0, \quad \deg s < m - k, \quad \deg t < n - k, \quad \deg(sf + tg) < k. \tag{11}$$

PROOF. "$\Longrightarrow$": Suppose that k does not appear in the degree sequence. Then there exists an i with $2 \leq i \leq \ell + 1$ such that $n_i < k < n_{i-1}$. We claim that $s = s_i$ and $t = t_i$ do the job. We have $sf + tg = r_i$, and $\deg r_i = n_i < k$. Furthermore, from Lemma 3.15 (b) we have

$$\begin{aligned} \deg s &= m - n_{i-1} < m - k, \\ 0 \leq \deg t &= n - n_{i-1} < n - k. \end{aligned}$$

The case $i = \ell + 1$ gives $s = g/r_\ell$ and $t = -f/r_\ell$, where $k < n_\ell$ and $r_{\ell+1} = 0$.

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

"$\Longleftarrow$": Suppose there exist $s,t \in F[x]$ satisfying (11). The Uniqueness Lemma 5.15 implies that there exist $i \in \{1,\dots,\ell+1\}$ and $\alpha \in F[x] \setminus \{0\}$ such that $t = \alpha t_i$ and $r = sf + tg = \alpha r_i$. Then from Lemma 3.15 (b) we find

$$\begin{aligned} n - n_{i-1} \leq \deg\alpha + n - n_{i-1} = \deg(\alpha t_i) = \deg t < n-k, \\ n_i \leq \deg\alpha + n_i = \deg(\alpha r_i) = \deg r < k. \end{aligned}$$

Together these imply that $n_i < k < n_{i-1}$, so that k is between two consecutive remainder degrees and does not occur in the degree sequence. □

As we did for the resultant, we now restate Theorem 6.47 in the language of linear algebra. The reader should keep comparing our development with the material about the resultant in Section 6.3, which is just the special case $k = 0$. We recall that $P_d \subseteq F[x]$ denotes the vector space of all polynomials of degree less than $d \in \mathbb{N}$. For $0 \leq k \leq m$, we consider the restriction of the map φ from (4) to $P_{m-k} \times P_{n-k}$. These polynomials are mapped to P_{n+m-k}. But now

$$\dim(P_{m-k} \times P_{n-k}) = n+m-2k < n+m-k = \dim P_{n+m-k},$$

if $k > 0$. In order to find an isomorphism, we need an image space of the proper dimension. It turns out that the right thing to do is to ignore the k low order coefficients of $a \in P_{n+m-k}$, so that we consider the quotient a quo x^k of a on division by x^k, and the corresponding linear map $P_{n+m-k} \longrightarrow P_{n+m-2k}$. Thus we take

$$\begin{aligned} \varphi_k : P_{m-k} \times P_{n-k} &\longrightarrow P_{n+m-2k} \\ (s,t) &\longmapsto (sf+tg) \text{ quo } x^k. \end{aligned}$$

This is now a linear map between spaces of the same dimensions. Then Theorem 6.47 becomes the following.

COROLLARY 6.48.
Let $0 \leq k \leq m \leq n$, and $1 \leq i \leq \ell+1$.

(i) k appears in the degree sequence $\iff$ φ_k is an isomorphism.

(ii) If $k = n_i < n$, then the linear coefficients s_i, t_i computed in the EEA are the unique solution of $\varphi_k(s_i,t_i) = 1$.

PROOF. From Theorem 6.47 we have

k does not appear in the degree sequence
$\iff$ there exists a nonzero $(s,t) \in P_{m-k} \times P_{n-k}$ with $\varphi_k(s,t) = 0$
$\iff$ φ_k is not injective.

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

We have used the fact that $s \neq 0$ and $\varphi_k(s,t) = 0$ imply $t \neq 0$. This proves (i). For (ii), we note that if $k = n_i < n$, then $s_i \in P_{m-k}$ and $t_i \in P_{n-k}$ satisfy $\varphi_k(s_i,t_i) = 1$. Since φ_k is an isomorphism, this implies the claim. □

As for the resultant, it is easy to specify the matrix of φ_k. Namely, we write $f = \sum_{0\le j\le n} f_j x^j$ and $g = \sum_{0\le j\le m} g_j x^j$, with all $f_j, g_j \in F$. Then the $(n+m-2k)\times(n+m-2k)$ matrix S_k with entries in F defined as

$$S_k = \begin{pmatrix} f_n & & & & g_m & & & & & \\ f_{n-1} & f_n & & & g_{m-1} & g_m & & & & \\ \vdots & & \ddots & & \vdots & & \ddots & & & \\ f_{n-m+k+1} & \cdots & \cdots & f_n & g_{k+1} & \cdots & \cdots & g_m & & \\ \vdots & & & \vdots & \vdots & & & & \ddots & \\ f_{k+1} & \cdots & \cdots & f_m & g_{m-n+k+1} & \cdots & \cdots & \cdots & \cdots & g_m \\ \vdots & & & \vdots & \vdots & & & & & \vdots \\ \vdots & & & \vdots & \vdots & & & & & \vdots \\ f_{2k-m+1} & \cdots & \cdots & f_k & g_{2k-n+1} & \cdots & \cdots & \cdots & \cdots & g_k \end{pmatrix}, \quad \begin{matrix} \\ \\ \\ \\ \\ \\ \\ \\ \end{matrix}$$

(with $m-k$ columns of f_j's and $n-k$ columns of g_j's)

with $m-k$ columns of f_j's and $n-k$ columns of g_j's, is the matrix of φ_k with respect to the standard bases $(x^i,0)$ for $i < m-k$ plus $(0,x^j)$ for $j < n-k$ of $P_{m-k}\times P_{n-k}$ and x^p for $p < n+m-2k$ of P_{n+m-2k}. An f_j or g_j with $j<0$ is zero. In other words, if

$$s = \sum_{0\le j<m-k} y_j x^j, \quad t = \sum_{0\le j<n-k} z_j x^j, \quad sf+tg = \sum_{0\le j<n+m-k} u_j x^j \in F[x],$$

then

$$S_k \cdot (y_{m-k-1},\dots,y_0,z_{n-k-1},\dots,z_0)^T = (u_{n+m-k-1},\dots,u_k)^T$$

where ^{T} denotes transposition. Again, the reader is advised to understand this relation carefully. We have immediately the following consequence of Corollary 6.48.

COROLLARY 6.49.
Let $0 \le k \le m \le n$, and $1 \le i \le \ell+1$.

(i) k appears in the degree sequence $\iff \det S_k \neq 0$.

(ii) If $k = n_i < n$, and $y_0,\dots,y_{m-k-1},z_0,\dots,z_{n-k-1} \in F$ form the unique solution to

$$S_k \cdot (y_{m-k-1},\dots,y_0,z_{n-k-1},\dots,z_0)^T = (0,\dots,0,1)^T, \tag{12}$$

then $s_i = \sum_{0\le j<m-k} y_j x^j$ and $t_i = \sum_{0\le j<n-k} z_j x^j$.

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

If R is a (commutative) ring and $f, g \in R[x]$, then in Section 6.3 we called $S_0 = \mathrm{Syl}(f,g)$ their **Sylvester matrix**, and $\mathrm{res}(f,g) = \det S_0$ their **resultant**. The newly defined elements $\sigma_k = \det S_k \in R$ for $0 \le k \le \deg g$ are the **subresultants**. (In the literature and in computer algebra systems, one finds various definitions that are slightly different from ours.) In fact, S_k is a submatrix of S_i if $k > i$. If $k = \deg f = \deg g$, then S_k is the empty matrix with determinant $\sigma_k = 1$.

It is convenient to define the subresultants also in the following cases: if $0 \le k \le n < m$, we let $\sigma_k = \det S_k$, as before. If $\min\{n,m\} < k < \max\{n,m\}$, we let $\sigma_k = 0$. Then Corollary 6.49 (i) with $\det S_k$ replaced by σ_k is in fact valid for arbitrary $n, m, k \in \mathbb{N}$ with $k < \max\{n,m\}$, and part (ii) is valid if $k \le \min\{n,m\}$ and $k < \max\{n,m\}$.

EXAMPLE 6.16 (continued). Let $f = r_0 = x^4 - 3x^3 + 2x$ and $g = r_1 = x^3 - 1$ in $\mathbb{Z}[x]$. We have seen in Example 6.16 that the degree sequence is $4, 3, 1$, and 0 and 2 do not appear. Moreover, we have calculated $\mathrm{res}(f,g) = \sigma_0 = \det S_0 = 0$. The other subresultants are

$$\sigma_1 = \det S_1 = \det \begin{pmatrix} 1 & 0 & 1 & 0 & 0 \\ -3 & 1 & 0 & 1 & 0 \\ 0 & -3 & 0 & 0 & 1 \\ 2 & 0 & -1 & 0 & 0 \\ 0 & 2 & 0 & -1 & 0 \end{pmatrix} = 9,$$

$$\sigma_2 = \det S_2 = \det \begin{pmatrix} 1 & 1 & 0 \\ -3 & 0 & 1 \\ 0 & 0 & 0 \end{pmatrix} = 0,$$

$$\sigma_3 = \det S_3 = \det \begin{pmatrix} 1 \end{pmatrix} = 1. \diamond$$

EXAMPLE 6.1 (continued). The following MAPLE code computes the subresultants of the two polynomials from Example 6.1.

$$f := 824x^5 - 65x^4 - 814x^3 - 741x^2 - 979x - 764$$

$$g := 216x^4 + 663x^3 + 880x^2 + 916x + 617$$

```
with(LinearAlgebra):
S[0] := Transpose(SylvesterMatrix(f, g, x));
```

$$S_0 := \begin{bmatrix} 824 & 0 & 0 & 0 & 216 & 0 & 0 & 0 & 0 \\ -65 & 824 & 0 & 0 & 663 & 216 & 0 & 0 & 0 \\ -814 & -65 & 824 & 0 & 880 & 663 & 216 & 0 & 0 \\ -741 & -814 & -65 & 824 & 916 & 880 & 663 & 216 & 0 \\ -979 & -741 & -814 & -65 & 617 & 916 & 880 & 663 & 216 \\ -764 & -979 & -741 & -814 & 0 & 617 & 916 & 880 & 663 \\ 0 & -764 & -979 & -741 & 0 & 0 & 617 & 916 & 880 \\ 0 & 0 & -764 & -979 & 0 & 0 & 0 & 617 & 916 \\ 0 & 0 & 0 & -764 & 0 & 0 & 0 & 0 & 617 \end{bmatrix}$$

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

```
S[1] := SubMatrix(S[0], 1..7, [1,2,3,5,6,7,8]);
```

$$S_1 := \begin{bmatrix} 824 & 0 & 0 & 216 & 0 & 0 & 0 \\ -65 & 824 & 0 & 663 & 216 & 0 & 0 \\ -814 & -65 & 824 & 880 & 663 & 216 & 0 \\ -741 & -814 & -65 & 916 & 880 & 663 & 216 \\ -979 & -741 & -814 & 617 & 916 & 880 & 663 \\ -764 & -979 & -741 & 0 & 617 & 916 & 880 \\ 0 & -764 & -979 & 0 & 0 & 617 & 916 \end{bmatrix}$$

```
S[2] := SubMatrix(S[1], 1..5, [1,2,4,5,6]);
```

$$S_2 := \begin{bmatrix} 824 & 0 & 216 & 0 & 0 \\ -65 & 824 & 663 & 216 & 0 \\ -814 & -65 & 880 & 663 & 216 \\ -741 & -814 & 916 & 880 & 663 \\ -979 & -741 & 617 & 916 & 880 \end{bmatrix}$$

```
S[3] := SubMatrix(S[2], 1..3, [1,3,4]);
```

$$S_3 := \begin{bmatrix} 824 & 216 & 0 \\ -65 & 663 & 216 \\ -814 & 880 & 663 \end{bmatrix}$$

```
S[4] := SubMatrix(S[3], 1..1, [2]);
```

$$S_4 := \begin{bmatrix} 216 \end{bmatrix}$$

```
for k from 0 to 4 do
  sigma[k] := Determinant(S[k]);
od;
```

$$\sigma_0 := 319475271814004272732076 48$$

$$\sigma_1 := 27754088254928081728$$

$$\sigma_2 := -41344606374560$$

$$\sigma_3 := 176909472$$

$$\sigma_4 := 216 \diamond$$

Corresponding to the bound of Theorem 6.23 on the resultant, Hadamard's inequality 16.6 yields the following estimate of the subresultants.

THEOREM 6.50.
Let $f, g \in \mathbb{Z}[x]$, $n = \deg f$, $m = \deg g$, *and* $0 \le k \le \min\{n, m\}$. *Then*

$$|\sigma_k| = |\det S_k| \le \|f\|_2^{m-k} \|g\|_2^{n-k} \le (n+1)^{n-k} \|f\|_\infty^{m-k} \|g\|_\infty^{n-k}.$$

The analogous result for bivariate polynomials reads as follows.

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

THEOREM 6.51.
Let $f,g \in F[x,y]$ with $n = \deg_x f$, $m = \deg_x g$, $\deg_y f, \deg_y g \le d$, and $0 \le k \le \min\{n,m\}$. Then $\deg_y \sigma_k \le (n+m-2k)d$.

6.11. Modular Extended Euclidean Algorithms

In this section, we use the subresultants from Section 6.10 to prove a bound on the coefficients of the Extended Euclidean Algorithm 3.14 over $\mathbb{Q}[x]$ and $F(y)[x]$, and to derive modular algorithms for the results of the EEA.

THEOREM 6.52.
Let $f,g \in \mathbb{Z}[x]$ have degrees $n \ge m$ and max-norm $\|f\|_\infty, \|g\|_\infty$ at most A, and let $\delta = \max\{n_{i-1} - n_i : 1 \le i \le \ell\}$ be the maximal degree difference of consecutive remainders. The results r_i, s_i, t_i of the Extended Euclidean Algorithm 3.14 for f and g in $\mathbb{Q}[x]$ have numerators and denominators (in lowest terms) absolutely bounded by $B = (n+1)^n A^{n+m}$. The corresponding bound for q_i and ρ_i is $C = (2B)^{\delta+2}$. The algorithm can be performed with $O(n^3 m \delta^2 \log^2(nA))$ word operations.

PROOF. Let $2 \le i \le \ell$ and $n_i = \deg r_i$. In the EEA, s_i and t_i form the unique solution to the system (12) of linear equations, so that $\sigma_{n_i} s_i$, $\sigma_{n_i} t_i$, and $\sigma_{n_i} r_i = \sigma_{n_i} s_i f + \sigma_{n_i} t_i g$ are in $\mathbb{Z}[x]$, and by Cramer's rule 25.6 and Hadamard's inequality 16.6 we have

$$\begin{aligned}
|\sigma_{n_i}| &\le \|f\|_2^{m-n_i} \|g\|_2^{n-n_i} \le (n+1)^{n-n_i} A^{n+m-2n_i} \le B,\\
\|\sigma_{n_i} s_i\|_\infty &\le \|f\|_2^{m-n_i-1} \|g\|_2^{n-n_i} \le (n+1)^{n-n_i-1/2} A^{n+m-2n_i-1} \le B,\\
\|\sigma_{n_i} t_i\|_\infty &\le \|f\|_2^{m-n_i} \|g\|_2^{n-n_i-1} \le (n+1)^{n-n_i-1/2} A^{n+m-2n_i-1} \le B,\\
\|\sigma_{n_i} r_i\|_\infty &= \|(\sigma_{n_i} s_i) f + (\sigma_{n_i} t_i) g\|_\infty \le (n_i+1)(\|\sigma_{n_i} s_i\|_\infty \cdot \|f\|_\infty + \|\sigma_{n_i} t_i\|_\infty \cdot \|g\|_\infty)\\
&\le (n_i+1) \cdot 2(n+1)^{n-n_i-1/2} A^{n+m-2n_i} \le 2(n+1)^{1/2} B.
\end{aligned}$$

Exercise 6.45 gives the slightly better bound $\|\sigma_{n_i} r_i\|_\infty \le B$.

Finally, we consider the division with remainder $r_{i-1} = q_i r_i + \rho_{i+1} r_{i+1}$, and let $k = \deg q = n_{i-1} - n_i$. Multiplying up, we find the pseudodivision (Section 6.12)

$$\sigma_{n_i}^{k+1}(\sigma_{n_{i-1}} r_{i-1}) = (\sigma_{n_i}^k \sigma_{n_{i-1}} q_i) \cdot (\sigma_{n_i} r_i) + (\sigma_{n_i}^{k+1} \sigma_{n_{i-1}} \rho_{i+1} r_{i+1}), \tag{13}$$

where the four terms in parentheses are in $\mathbb{Z}[x]$. By Exercise 6.44, we have

$$\begin{aligned}
\|\sigma_{n_i}^k \sigma_{n_{i-1}} q_i\|_\infty &\le \|\sigma_{n_{i-1}} r_{i-1}\|_\infty \cdot (\|\sigma_{n_i} r_i\|_\infty + |\sigma_{n_i}|)^k \le (2B)^{k+1},\\
\|\sigma_{n_i}^{k+1} \sigma_{n_{i-1}} \rho_{i+1} r_{i+1}\|_\infty &\le \|\sigma_{n_{i-1}} r_{i-1}\|_\infty \cdot (\|\sigma_{n_i} r_i\|_\infty + |\sigma_{n_i}|)^{k+1} \le (2B)^{k+2}
\end{aligned}$$

Since $|\sigma_{n_i}^k \sigma_{n_{i-1}}| \le B^{k+1}$ and $|\sigma_{n_i}^{k+1} \sigma_{n_{i-1}}| \le B^{k+2}$, both numerators and denominators of q_i and ρ_{i+1} are absolutely bounded by $(2B)^{k+2} \le C$.

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

The algorithm uses $O(nm)$ arithmetic operations (Theorem 3.16) on rational numbers with numerator and denominator absolutely bounded by C. A single such operation can be done with $O(\log^2 C)$ or $O(n^2\delta^2\log^2(nA))$ word operations, and the claimed time bound follows. □

For random inputs (say, of fixed degrees and coefficient lengths), the expected value of δ is quite small if the degrees n and m of the two inputs are close to each other (Exercise 6.46). It is conceivable that the δ in the estimates for q_i and ρ_i is an artifact and that a more careful analysis would in fact reveal that it can be replaced by 1. Lickteig & Roy (1996, 2001) discuss a variant of the EEA where this is indeed the case.

For comparison, we state the analogous bounds for the *traditional* EEA 3.6, where the remainders are given by $r^*_{i+1} = r^*_{i-1}$ rem r^*_i for all i, without dividing out the leading coefficient.

THEOREM 6.53.

We denote by $q^*_i, r^*_i, s^*_i, t^*_i \in \mathbb{Q}[x]$ the results of the traditional Extended Euclidean Algorithm, and

$$\alpha_i = \begin{cases} \rho_i\rho_{i-2}\cdots\rho_2\rho_0 & \text{if } i \geq 0 \text{ is even,} \\ \rho_i\rho_{i-2}\cdots\rho_3\rho_1 & \text{if } i \geq 1 \text{ is odd.} \end{cases}$$

(i) The length of the algorithm equals that of the monic EEA, and for all i we have

$$q^*_i = \frac{\alpha_{i-1}}{\alpha_i} q_i, \quad r^*_i = \alpha_i r_i, \quad s^*_i = \alpha_i s_i, \quad t^*_i = \alpha_i t_i.$$

(ii) Let n, m, δ, A, C be as in Theorem 6.52. The numerators and denominators of the coefficients of all results of the traditional algorithm in $\mathbb{Q}[x]$ are bounded by C^{m+2} in absolute value, and the computing time is $O(n^3m^3\delta^2\log^2(nA))$.

Exercise 6.47 asks for a proof, and Exercise 6.49 gives a slightly better bound for the r^*_i, s^*_i, t^*_i in the traditional EEA, essentially replacing δ by 1.

We compare the two bounds from Theorems 6.52 and 6.53 with Mignotte's bound, say when A is an n-digit number and $\delta = 1$. Then the "traditional" bound is a number of about n^3 digits, the "monic" one has about n^2 digits, and Mignotte's only about n digits! Of course Mignotte's bound only applies to the gcd, and one cannot hope for a bound of similar quality for all results of the EEA.

Theorem 6.52 provides a clear explanation for the coefficient growth, as in Example 6.1. The results of the EEA are governed by subresultants and grow at a quadratic rate in length. But the leading coefficients α_i in the traditional Euclidean Algorithm are a product of $i/2$ such entries, and thus grow at a cubic rate.

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

This does not literally follow from Theorem 6.53, which gives only upper bounds, but there seems to be typically little cancellation in the product defining α_i. For instance, in the example below, the products of numerators and denominators in $r_5 = \alpha_5 = \rho_1 \cdot \rho_3 \cdot \rho_5$ have 50 and 48 digits, respectively, and only the 2-digit factor 24 cancels. A practical recommendation is therefore to use the monic version wherever possible.

EXAMPLE 6.1 (continued). The traditional Euclidean Algorithm for the polynomials from Example 6.1 produces the following quotients and remainders; almost any (random) input will exhibit a similar behavior:

$$r_0 := 824x^5 - 65x^4 - 814x^3 - 741x^2 - 979x - 764$$

$$r_1 := 216x^4 + 663x^3 + 880x^2 + 916x + 617$$

```
for i from 1 to 5 do
  q[i] := quo(r[i - 1], r[i], x, 'r[i + 1]');
  r[i + 1] := sort(r[i + 1]);
od;
```

$$q_1 := \frac{103}{27}x - \frac{5837}{486}$$

$$r_2 := \frac{614269}{162}x^3 + \frac{1539085}{243}x^2 + \frac{1863490}{243}x + \frac{3230125}{486}$$

$$q_2 := \frac{34992}{614269}x + \frac{30072401334}{377326404361}$$

$$r_3 := -\frac{23256341085690}{377326404361}x^2 - \frac{27844657381944}{377326404361}x + \frac{32938754949612}{377326404361}$$

$$q_3 := -\frac{231779913080427109}{3767527255881780}x - \frac{21250438136739791430061202 3767}{730157490936836182695747 7350}$$

$$r_4 := \frac{16363047386796678464177161 8997}{15023816685943131331188225}x + \frac{27604692189910198127667206 7323}{30047633371886262662376450}$$

$$q_4 := -\frac{34939900525717422066436421955424400025 0}{61742098348486478706658122441075651245917}x$$
$$- \frac{53605502942609915156276524064879156029311616760832823425}{26774931978255360791810790390285343980469602246030531286009}$$

$$r_5 := \frac{14999180998204546086628509444183593910034968673275}{14191920665397666679466196080912938207431541 8338}$$

$$q_5 := 232223070357561067969371778322000546147238377985961441623 2408\backslash$$
$$1189217602966986 \Big/ 225344945756306612080710638588525392490642 34\backslash$$
$$5609867489818460486857552186875x + 19588180077596405579156628228 91\backslash$$
$$8052861081903680682675547410956194774022384587 \Big/ 22534494575630\backslash$$
$$6612080710638588525392490642345609867489818460486857552186875$$

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

$$r_6 := 0$$

One can see clearly that the numerators and denominators are considerably larger than in the monic Euclidean Algorithm on page 143. $\diamond$

The following analog of Theorems 6.52 and 6.53 for bivariate polynomials is proven in Exercise 6.48.

THEOREM 6.54.
Let F be a field, $f,g \in F[x,y]$ with $n = \deg_x f \geq m = \deg_x g$ and $\deg_y f, \deg_y g \leq d$, and let $\delta = \max\{n_{i-1} - n_i : 1 \leq i \leq \ell\}$ be the maximal degree difference of consecutive remainders in the Euclidean Algorithm for f and g in $F(y)[x]$.

(i) The results r_i, s_i, t_i of the Extended Euclidean Algorithm 3.14 for f and g in $F(y)[x]$ have numerators and denominators (in lowest terms) of degree in y at most $(n+m-2n_i)d \leq (n+m)d$. The corresponding bound for the q_i and ρ_i is $(\delta+2)(n+m)d$. The Extended Euclidean Algorithm 3.14 can be performed with $O(n^3 m \delta^2 d^2)$ operations in F.

(ii) For the traditional EEA, the degree bound is $(m+2)(\delta+2)(n+m)d$, and the number of operations in F is $O(n^3 m^3 \delta^2 d^2)$.

The following generalization of Theorem 6.26 will lead to modular algorithms for the EEA.

THEOREM 6.55.
Let R be a Euclidean domain with field of fractions K, $p \in R$ prime, and $f,g \in R[x]$ nonzero with $\deg f \geq \deg g$ and such that p does not divide $b = \gcd(\mathrm{lc}(f), \mathrm{lc}(g))$. Furthermore let $1 \leq i \leq \ell$ and $r_i, s_i, t_i \in K[x]$ be the results in the ith row of the monic EEA, $n_i = \deg r_i < \deg f$, and $\sigma = \sigma_{n_i} \in R$ the n_ith subresultant of f,g. A bar denotes the reduction modulo p.

(i) The polynomials $\sigma r_i, \sigma s_i, \sigma t_i$ are in $R[x]$.

(ii) The remainder degree n_i occurs in the EEA for $\overline{f}, \overline{g}$ over $R/\langle p \rangle$ if and only if $p \nmid \sigma$.

(iii) If $p \nmid \sigma$, then p divides no denominator in r_i, s_i, or t_i, and $\overline{r_i}, \overline{s_i}, \overline{t_i}$ form a row in the EEA for $\overline{f}, \overline{g}$ over $R/\langle p \rangle$, with $\deg \overline{r_i} = n_i$.

PROOF. (i) follows from Cramer's rule, as in the proof of Theorem 6.52. We first assume that $n_i \leq \min\{\deg \overline{f}, \deg \overline{g}\}$. Then $\overline{\sigma}$ is, up to a unit modulo p, equal to the

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

n_ith subresultant of $\overline{f}$ and $\overline{g}$, as in the proof of Lemma 6.25, and Corollary 6.49 gives (ii). The coefficients of $\overline{s_i}, \overline{t_i}$ are a solution over $R/\langle p\rangle$ of the system (12) for $f = \overline{f}$ and $g = \overline{g}$, and (iii) follows since the solution is unique, by Corollary 6.49 (ii).

It remains to investigate the degenerate cases where $\deg \overline{f} < n_i$ or $\deg \overline{g} < n_i$. Then $p \nmid b$ implies that $\deg \overline{g} = \deg g$ or $\deg \overline{f} = \deg f$, respectively. If $i \geq 2$ and $\deg \overline{f} < n_i < n_1 = \deg \overline{g}$, then n_i does not occur in the degree sequence of the EEA for $\overline{f}$ and $\overline{g}$, and $p \mid \sigma$ since it divides each entry of the $(n_1 - n_i)$th column of the matrix S_{n_i}, which has determinant σ. A similar argument works if $\deg \overline{g} < n_i < \deg f = \deg \overline{f}$. Finally, if $i = 1$ and $\deg \overline{f} < n_i = \deg \overline{g}$, then $p \nmid \sigma = \mathrm{lc}(g)^{\deg f - \deg g}$ and $\overline{r_i} = \overline{g}/\mathrm{lc}(\overline{g})$, $\overline{s_i} = 0$, $\overline{t_i} = 1$ form a row in the EEA for $\overline{f}$ and $\overline{g}$. □

The restriction $n_i < \deg f$ is somewhat non-intuitive. It excludes precisely the case where $i = 1$ and $\deg f = \deg g$. Then $\sigma = 1$, and the claims (i) and (iii) of the theorem may fail to hold if $p \mid \mathrm{lc}(g)$. This is due to the fact that we consider the Euclidean Algorithm with monic remainders; both claims hold trivially for the traditional Euclidean Algorithm in this case.

It is interesting to note that p might divide some previous subresultants, so that some rows are "missing" modulo p, but still the correct results pop up modulo p whenever p does not divide that particular subresultant nor the leading coefficients of f and g.

EXAMPLE 6.56. Let $f = r_0 = x^4 + x^3 + x^2 + x - 4$ and $g = r_1 = x^3 - 2x^2 + x + 3$ in $\mathbb{Z}[x]$. Then the EEA in $\mathbb{Q}[x]$ computes

$$\begin{aligned}
r_0 &= q_1 r_1 + \rho_2 r_2 = (x+3) r_1 + 6\Big(x^2 - \frac{5}{6}x - \frac{13}{6}\Big),\\
r_1 &= q_2 r_2 + \rho_3 r_3 = \Big(x - \frac{7}{6}\Big) r_2 + \frac{79}{36}\Big(x + \frac{17}{79}\Big),\\
r_2 &= q_3 r_3 + \rho_4 r_4 = \Big(x - \frac{497}{474}\Big) r_3 - \frac{12\,114}{6241} \cdot 1,\\
r_3 &= q_4 r_4 = r_3 r_4.
\end{aligned}$$

Modulo $p = 3$, the computation is

$$\begin{aligned}
r_0^* &= q_1^* r_1^* + \rho_2^* r_2^* = x \cdot r_1^* + 1 \cdot (x+2),\\
r_1^* &= q_2^* r_2^* = (x^2 + 2x) r_2^*.
\end{aligned}$$

The degrees 2 and 0 are missing in the degree sequence modulo 3, but nonetheless the two remainders r_3 and r_2^* of degree 1 are equal modulo 3. ◇

We obtain the following modular algorithm for the results of the EEA in $\mathbb{Q}[x]$.

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

ALGORITHM 6.57 Modular EEA in $\mathbb{Q}[x]$: small primes version.
Input: $f, g \in \mathbb{Z}[x]$ with $\deg f = n \geq \deg g = m \geq 1$ and $\|f\|_\infty, \|g\|_\infty \leq A$.
Output: The results r_i, s_i, t_i in $\mathbb{Q}[x]$ of the EEA for f and g.

1. $B \longleftarrow (n+1)^n A^{n+m}$, $\quad r \longleftarrow \lceil \log_2(2A^2B^3+1) \rceil$
   choose a set $S \subseteq \mathbb{N}$ of r primes

2. $S \longleftarrow \{p \in S : p \nmid \mathrm{lc}(f) \text{ and } p \nmid \mathrm{lc}(g)\}$
   for each $p \in S$ **call** the Euclidean Algorithm 3.14 to compute all results in $\mathbb{Z}_p[x]$ of the EEA for f mod p and g mod p

3. Let $n_0 = n \geq n_1 = m > n_2 > \ldots > n_\ell \geq 0$ be the degrees of all remainders that were computed in step 2
   for $i = 2, \ldots, \ell$ **do**
   $S_i \longleftarrow \{p \in S : n_i \text{ occurs in the degree sequence of } f \bmod p, g \bmod p\}$
   compute the coefficients of the monic remainder $r_i \in \mathbb{Q}[x]$ of degree n_i and of $s_i, t_i \in \mathbb{Q}[x]$ from their images modulo the primes in S_i by rational number reconstruction (Section 5.10)

4. **return** r_i, s_i, t_i for $2 \leq i \leq \ell$

THEOREM 6.58.
The algorithm returns the correct values as specified. If S in step 1 consists of the first r primes, then the algorithm uses $O(n^3 m \log^2(nA)(\log^2 n + (\log\log A)^2))$ word operations.

PROOF. We denote by $\sigma_{m-1}, \sigma_{m-2} \ldots, \sigma_0 \in \mathbb{Z}$ the subresultants and by $m_0 = n$, $m_1 = m, m_2, \ldots, m_{\ell^*}$ the degree sequence of (f, g).

Let $0 \leq k < m$. Then for any prime $p \in S$, k occurs as a remainder degree, say $k = n_i$, for f mod p and g mod p if and only if $\sigma_k \not\equiv 0 \bmod p$. By the choice of r, we have $\prod_{p \in S} p > 2B^3A^2$. After removing divisors of $\mathrm{lc}(f)$ and $\mathrm{lc}(g)$ from S in step 2, we still have $\prod_{p \in S} p > 2B^3$. Since $|\sigma_k| \leq B$, by Theorem 6.50, we have $\prod_{p \in S_i} p > 2B^2$ if $\sigma_k \neq 0$. (If $\sigma_k = 0$, then no modular remainder has degree k.) It follows that precisely those k with $\sigma_k \neq 0$ occur as modular remainder degrees, and $\ell^* = \ell$ and $n_i = m_i$ for $0 \leq i \leq \ell$. Theorem 6.55 says that r_i mod p, s_i mod p, t_i mod p have been computed in step 2 for the primes p in S_i. The numerators and denominators of the coefficients of the monic r_i are absolutely bounded by B, by Theorem 6.52, and they can indeed be uniquely reconstructed from their images modulo the $p \in S_i$ (Theorem 5.26). The same holds for s_i and t_i.

For the cost analysis, Theorem 18.10 says that the first r primes can be computed in time $O(r \log^2 r \log\log r)$ in step 1, and that $\log p \in O(\log r)$ for all $p \in S$. Thus the cost per prime p in step 2 is $O(n \log A \log r)$ word operations for reducing all coefficients of f and g modulo p, a total of $O(n \log A \cdot r \log r)$ operations. The EEA

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

modulo p takes $O(nm\log^2 r)$ word operations (Theorem 3.16 and Corollary 4.7), and $O(nmr\log^2 r)$ for all $p \in S$. Since $\log \prod_{p\in S} p = \sum_{p\in S} \log p \in O(r\log r)$, we have $O(r^2\log^2 r)$ word operations for reconstructing one rational coefficient in step 3 from its image modulo all primes in S by means of the Chinese Remainder Algorithm 5.4 and the (traditional) Extended Euclidean Algorithm 3.6 over $\mathbb{Z}$. Since all r_i, s_i, t_i together have $O(nm)$ coefficients, the total cost for step 3 is $O(nmr^2\log^2 r)$ word operations. This dominates the cost for the other steps, and the claim follows from $r \in O(n\log(nA))$. □

If also some quotient q_i and the corresponding ρ_{i+1} is required, we calculate them by one division with remainder: $r_{i-1} = q_i r_i + \rho_{i+1} r_{i+1}$.

As for the small primes modular determinant computation, actually a slightly smaller value for r is sufficient; see Exercise 18.21.

Our timing estimate is—up to logarithmic factors—the same as for direct calculation (Theorem 6.52) and for the big prime variant (Exercise 6.51). However, in many applications we are only interested in one particular row r_i, s_i, t_i of the EEA. Then the rational number reconstruction in step 3 takes only $O(nr^2\log^2 r)$ word operations, and the overall cost drops to $O^\sim(n^3\log^2 A)$, which is faster by about a factor of m than both direct calculation and the big prime version.

In practice, one would not take the first r primes but instead sufficiently many (possibly precomputed) single precision primes just below the word size, and then the cost is $O(nmr^2)$ word operations. When using fast algorithms for integer arithmetic as discussed in Part II, the cost for the algorithm drops to $O^\sim(n^3\log A)$ word operations. Theorem 6.52 implies that the output size is at most about $n^2\log_{2^{64}} B \in O^\sim(n^3\log A)$ words, and hence the above time estimate is—up to logarithmic factors—optimal when the size of the output is close to the upper bound. When in addition using the fast Euclidean Algorithm from Chapter 11 in step 2, then we may compute a single row r_i, s_i, t_i in the EEA at a cost of only $O^\sim(n^2\log A)$ operations (Corollary 11.14).

Here is the analogous algorithm for $F[x,y]$.

ALGORITHM 6.59 Modular bivariate EEA: small primes version.
Input: $f, g \in F[x,y] = R[x]$ of degree $n \geq m \geq 1$ in x and at most d in y, where $R = F[y]$ for a field F with at least $3(n+m+1)d$ elements.
Output: The results r_i, s_i, t_i in $F(y)[x]$ of the EEA for f and g.

1. let $S \subseteq F$ with $\#S = 3(n+m+1)d$
2. $S \longleftarrow \{u \in S : \mathrm{lc}_x(f), \mathrm{lc}_x(g)$ do not vanish at $y = u\}$
   for each $u \in S$ **call** the Euclidean Algorithm 3.14 to compute all results in $F[x]$ of the EEA for $f(x,u)$ and $g(x,u)$

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

3. let $n_0 = n \geq n_1 = m > n_2 \cdots > n_\ell \geq 0$ be the degrees of all remainders that were computed in step 2
   for $i = 2, \ldots, \ell$ **do**
   $S_i \longleftarrow \{u \in S : n_i$ occurs in the degree sequence of $f(x,u), g(x,u)\}$
   compute the coefficients of the monic remainder $r_i \in F(y)[x]$ and of $s_i, t_i \in F(y)[x]$ from their values at the points in S_i by Cauchy interpolation (Section 5.8)
4. **return** r_i, s_i, t_i for $2 \leq i \leq \ell$

THEOREM 6.60.
The algorithm returns the correct values as specified. It uses $O(n^3 m d^2)$ operations in F.

See Exercise 6.50 for the proof. As for the modular gcd algorithm, we make a suitable field extension when F does not have sufficiently many elements.

The timing estimate for the small primes modular EEA is the same as for both the big prime variant and direct calculation, and better by a factor of m if we only want to compute a single row r_i, s_i, t_i of the EEA, as in the integer case. The estimate drops to $O^\sim(n^3 d)$ when using fast arithmetic (Part II), or even to $O^\sim(n^2 d)$ when only one row of the EEA is required (Corollary 11.12). Both bounds are optimal up to logarithmic factors since the output size for all results is about $n^3 d$, and for one row about $n^2 d$, at least in a generic sense.

Our purpose in studying subresultants has been to gain a conceptual understanding of Euclid's algorithm and a bound on the coefficients occurring in it. One might be tempted to actually execute Euclid's algorithm by calculating subresultants via Gaussian elimination. This would be highly inefficient; in Section 11.2 we show how to calculate subresultants efficiently from the ρ_i in the Euclidean Algorithm. They may then be used to replace the rational number reconstruction and the Cauchy interpolation, respectively, in step 3 of the modular EEA algorithms, by a (computationally easier) polynomial Chinese remainder or interpolation algorithm, after multiplying all modular images by the corresponding subresultant.

The same modular techniques also apply to gcds of multivariate polynomials over $\mathbb{Q}$ or a finite field. The rational case is reduced to the finite field case by computing modulo small prime numbers, and the computation of multivariate polynomial gcds over finite fields is reduced to univariate gcd computations by evaluating one of the variables at distinct points and proceeding recursively.

6.12. Pseudodivision and primitive Euclidean Algorithms

Gauß' lemma 6.6 suggests another way of computing the gcd of two primitive polynomials $f, g \in \mathbb{Z}[x]$: use the primitive versions of the remainders r_i, that is,

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

an integral multiple $\alpha_i r_i \in \mathbb{Z}[x]$ which is primitive. This corresponds to taking essentially the primitive part of a polynomial as its normal form in $\mathbb{Q}[x]$; see Exercise 6.5. It is convenient to replace the usual division with remainder by a **pseudo-division** computing $q, r \in \mathbb{Z}[x]$ from $a, b \in \mathbb{Z}[x]$ with

$$\mathrm{lc}(b)^{1+\deg a-\deg b} a = qb + r, \quad \deg r < \deg b$$

(assuming $b \neq 0$). The integer factor multiplied to a ensures that the division can be carried out in $\mathbb{Z}[x]$ (see also Exercise 2.9). This works with any integral domain R instead of $\mathbb{Z}$, and is useful when R is a ring of multivariate polynomials over an integral domain. As in Section 6.2, we assume that we have some normal form normal on R, which we extend to $R[x]$ via $\mathrm{normal}(f) = \mathrm{normal}(\mathrm{lc}(f))f/\mathrm{lc}(f)$.

ALGORITHM 6.61 Primitive Euclidean Algorithm.
Input: Primitive polynomials $f, g \in R[x]$, where R is a UFD, of degrees $n \geq m$.
Output: $h = \gcd(f, g) \in R[x]$.

1. $r_0 \longleftarrow f, \quad r_1 \longleftarrow g, \quad n_0 \longleftarrow n, \quad n_1 \longleftarrow m$
2. $i \longleftarrow 1$
   while $r_i \neq 0$ **do**
   { Pseudodivision }
   $a_{i-1} \longleftarrow \mathrm{lc}(r_i)^{1+n_{i-1}-n_i} r_{i-1}, \quad q_i \longleftarrow a_{i-1} \text{ quo } r_i,$
   $r_{i+1} \longleftarrow \mathrm{pp}(a_{i-1} \text{ rem } r_i), \quad n_{i+1} \longleftarrow \deg r_{i+1}$
   $i \longleftarrow i+1$
3. $\ell \longleftarrow i-1$
   return $\mathrm{normal}(r_\ell)$

THEOREM 6.62.
The algorithm correctly computes the gcd *as specified. We let* $\delta = \max\{n_{i-1} - n_i : 1 \leq i \leq \ell\}$ *be the maximal quotient degree.*

(i) If $R = \mathbb{Z}$ *and* $\|f\|_\infty, \|g\|_\infty \leq A$, *then the max-norm of the intermediate results is at most* $(2(n+1)^n A^{n+m})^{\delta+2}$, *and the algorithm uses* $O(n^3 m \delta^2 \log^2(nA))$ *word operations.*

(ii) If $R = F[y]$ *for a field* F *and* $\deg_y f, \deg_y g \leq d$, *then the degree in* y *of all intermediate results is at most* $(\delta+2)(n+m)d$, *and the time is* $O(n^3 m \delta^2 d^2)$ *operations in* F.

PROOF. Let $\alpha_i = \mathrm{lc}(r_i) \in R$ for all i, and K be the field of fractions of R. As in Exercise 6.47, we find that $r_i/\alpha_i \in K[x]$ is the ith remainder in the Euclidean

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

Algorithm for f, g. In particular, r_ℓ is a primitive multiple of the monic gcd, and hence normal$(r_\ell) = h$.

We only prove the claim (i) for $R = \mathbb{Z}$; the bivariate case is left as Exercise 6.53, which also discusses a primitive *Extended* Euclidean Algorithm. For $1 \le i \le \ell$, the polynomial r_i is a primitive multiple of the corresponding monic remainder, and thus α_i divides the subresultant σ_{n_i} of f and g, by Theorem 6.55 (i). Let $B = (n+1)^n A^{n+m}$. Then Theorem 6.52 implies that $|\alpha_i|, \|r_i\|_\infty \le B$. Thus

$$\|a_{i-1}\|_\infty = |\alpha_i|^{n_{i-1}-n_i+1} \|r_{i-1}\|_\infty \le B^{\delta+2},$$

and Exercise 6.44 implies that

$$\|q_i\|_\infty, \|a_{i-1} \text{ rem } r_i\|_\infty \le \|r_{i-1}\|_\infty (\|r_i\|_\infty + |\alpha_i|)^{n_{i-1}-n_i+1} \le (2B)^{\delta+2}.$$

The latter number bounds all integers in the algorithm. The number of operations in $\mathbb{Z}$ is $O(nm)$, and the estimate follows from $\log B \in O(n \log(nA))$. □

The time estimate for the primitive Euclidean Algorithm is the same as for the monic variant; Exercise 6.53 gives a slightly better bound. Its main advantage over the monic algorithm is that it avoids rational numbers or functions completely. Experiments with $R = \mathbb{Z}$ in Section 6.13 show that the primitive algorithm clearly beats the monic algorithm in practice, but it is still slower than the modular algorithms.

We can profitably use Algorithm 6.45 for computing the content and the primitive part of a_{i-1} rem r_i in step 2 when $R = F[y]$. This content is usually quite large: Exercise 6.54 shows that its degree is about $(n_{i-1} - n_i + 1)(n + m - 2n_i)d - 2(n_{i-1} - n_{i+1})d \ge 2(n + m - 2n_i - 2)d$.

EXAMPLE 6.1 (continued). We run the primitive Euclidean Algorithm 6.61 on the polynomials from Example 6.1 in MAPLE.

$$r_0 := 824x^5 - 65x^4 - 814x^3 - 741x^2 - 979x - 764$$
$$r_1 := 216x^4 + 663x^3 + 880x^2 + 916x + 617$$

```
for i from 1 to 5 do
  a[i - 1] := r[i - 1] * lcoeff(r[i], x)
    ^ (degree(r[i - 1], x) - degree(r[i], x) + 1);
  q[i] := quo(a[i - 1], r[i], x, 'r[i + 1]');
  r[i + 1] := sort(primpart(r[i + 1], x));
od;
```

$$a_0 := 38444544x^5 - 3032640x^4 - 37977984x^3 - 34572096x^2 - 45676224x - 35645184$$

$$q_1 := 177984x - 560352$$
$$r_2 := 1842807x^3 + 3078170x^2 + 3726980x + 3230125$$

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

$$a_1 := 733522530077784x^4 + 2251506654822087x^3 + 2988425122539120x^2 + 3110678877552084x + 2095293523416633$$

$$q_2 := 398046312x + 556896321$$

$$r_3 := -1292018949205x^2 - 1546925410108x + 1829930830534$$

$$a_2 := 307622161728586711322588659417 5x^3 + 513842908979661872996929539425 0x^2 + 622149603468625906763465453450 0x + 539208954140911745457270725312 5$$

$$q_3 := -238094156372761843 5x - 112636899464946169 4$$

$$r_4 := 867315257966502554x + 731586548698031843$$

$$a_3 := -971902851927901193438036722668528690384882481780x^2 - 116365260643337094569810102052324256995965603492 8x + 137653940301814944695275958692916694652371517144$$

$$q_4 := -112058774822734413490802876957 0x - 396448327221414114828389881017$$

$$r_5 := 1$$

$$a_4 := 867315257966502554x + 731586548698031843$$

$$q_5 := 867315257966502554x + 731586548698031843$$

$$r_6 := 0$$

In the example, the leading coefficients of the remainders agree—up to sign—with the denominators of the corresponding coefficients in the monic Euclidean Algorithm. $\diamond$

6.13. Implementations

Table 6.5 gives an overview of the different variants of the Euclidean Algorithm for computing gcds in $\mathbb{Z}[x]$ and $F[x,y]$ that we have discussed in this chapter.

We have implemented all algorithms discussed in this chapter for computing the gcd of two polynomials with integral coefficients in `C++`, using Victor Shoup's "Number Theory Library" NTL 1.5 for integer and polynomial arithmetic (see `http://www.shoup.net/ntl/`); we will describe parts of it in Section 9.7. The running times are given in Figures 6.4 and 6.6 for various degrees and coefficient sizes. The integer arithmetic of NTL uses Karatsuba's multiplication algorithm (Section 8.1) which is asymptotically faster than classical multiplication, so that, for example, the running time of the big prime gcd algorithm is only about $n^{3.18}$. All timings are the averages over 10 pseudorandom inputs. The software ran in 1998 on a Sun Sparc Ultra 1 clocked at 167 MHz.

The experiments were as follows. For each choice of n and k, we pseudorandomly and independently chose three polynomials $a,b,c \in \mathbb{Z}[x]$ of degree less than $\frac{n}{2}$ and with nonnegative coefficients less than $2^{k/2}$, and computed the gcd of

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

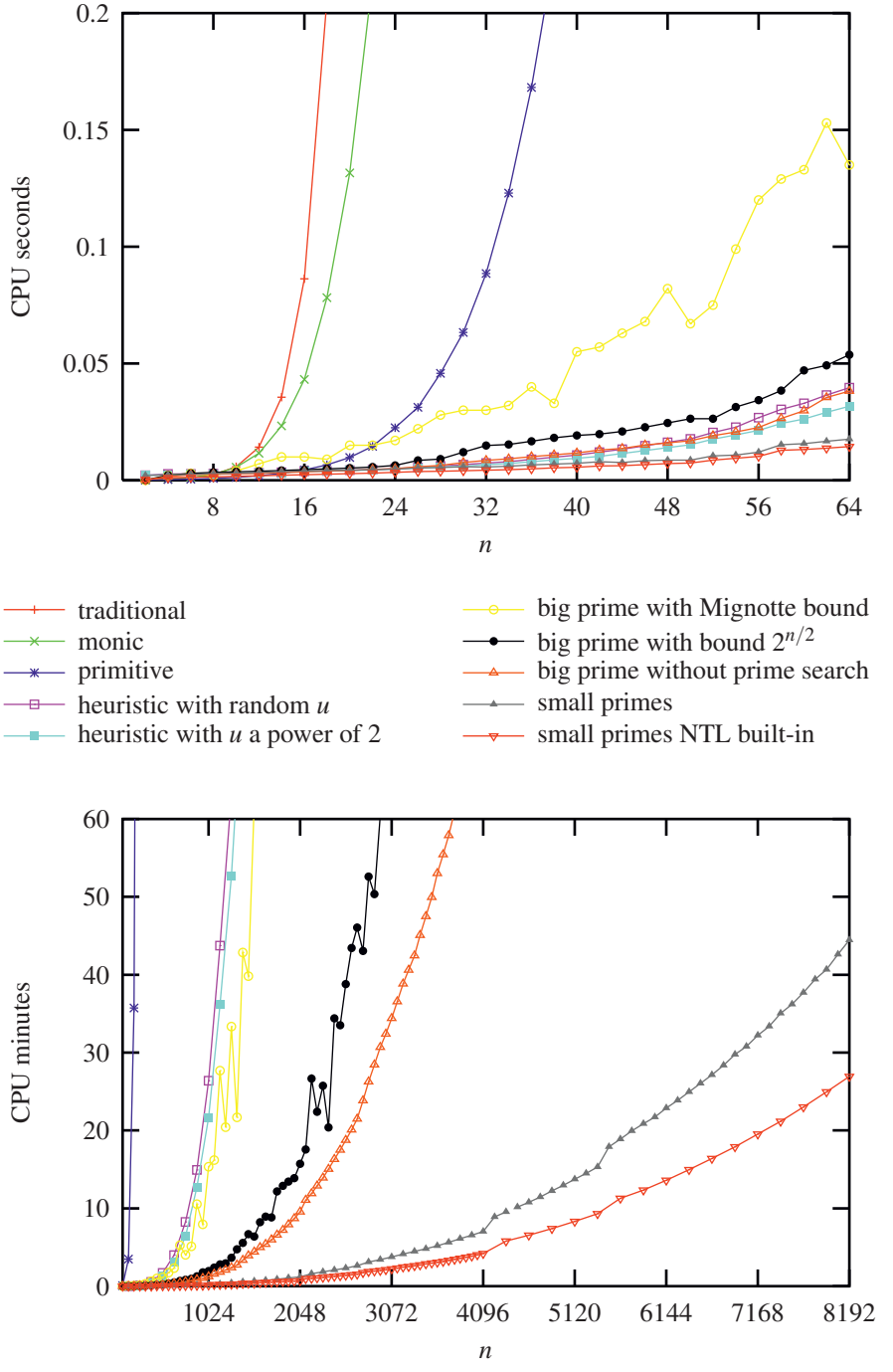


FIGURE 6.4: Various gcd algorithms in $\mathbb{Z}[x]$ for pseudorandom polynomials of degree less than $n-1$ with nonnegative coefficients less than $n2^{n-1}$, for $2 \le n \le 64$ and for $64 \le n \le 8192$.

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

Algorithm	for $\mathbb{Z}[x]$	for $F[x,y]$	time
traditional	Algorithm 3.6		n^8
monic	Algorithm 3.14		n^6
primitive	Algorithm 6.61		n^6
big prime modular EEA	Exercise 6.51		n^6
small primes modular EEA	Algorithm 6.57	Algorithm 6.59	n^6
small primes modular EEA, single row	Algorithm 6.57	Algorithm 6.59	n^5
"heuristic gcd"	Exercise 6.27	Exercise 6.28	n^4
big prime modular gcd	Algorithm 6.34	Algorithm 6.28	n^4
small primes modular gcd	Algorithm 6.38	Algorithm 6.36	n^3

TABLE 6.5: Comparison of various Euclidean Algorithms in $\mathbb{Z}[x]$ and $F[x,y]$. The time (word and field operations, respectively) is for polynomials of degree at most n in x and with coefficients of length or degree at most n, respectively, with a normal degree sequence, and ignores logarithmic factors.

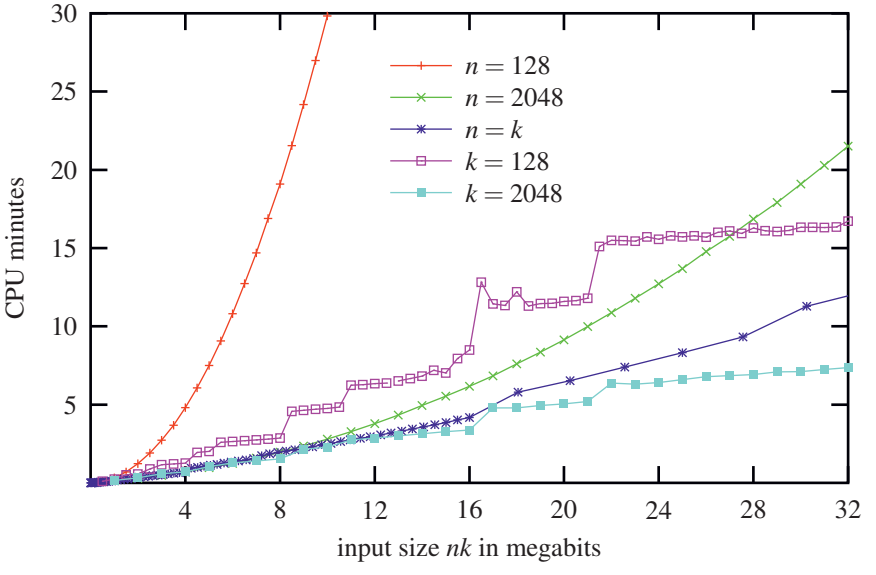


FIGURE 6.6: The small primes modular gcd algorithm in $\mathbb{Z}[x]$ of NTL for various pseudo-random polynomials of degree less than $n-1$ with about k-bit coefficients.

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

ac and bc in $\mathbb{Z}[x]$. Thus the degree of the gcd was at least $\frac{n}{2} - 1$; in fact, it was equal to $\frac{n}{2} - 1$ in all cases when $n \geq 6$.

In these cases where the gcd is essentially c, the Mignotte bound for the length of the coefficients of the gcd is too large by a factor of about 2^n, which discriminates against our implementation of the big prime algorithm. For that reason we also ran a variant of the big prime method using the known bound $2^{n/2}$ on the coefficients of c, and it computed the correct gcd in all cases, in time faster than the original big prime algorithm but still slower than the small primes algorithm. The standard deviations in the experiments with the big prime algorithm are considerably higher than for the other algorithms. The reason is that there were enormous differences in the time spent for finding a big prime with the routines of NTL that implement the probabilistic primality testing algorithms of Chapter 18. Figure 6.4 also shows the timings for the big prime algorithm with bound $2^{n/2}$ and *without* the cost of the prime search; one can see that the corresponding curves are much smoother.

We implemented two variants of the "heuristic" gcd algorithm (Exercise 6.27). In the first variant, the two input polynomials are evaluated at a random point, and the evaluation point is a power of two in the second variant. The most time consuming part in both variants is the gcd calculation of two integers with about n^2 bits.

We give timings for two variants of the small primes algorithm: our implementation and the built-in routine of NTL. Both routines differ from Algorithm 6.38 in that they work in an adaptive fashion. They do not compute the Mignotte bound at all, but take only as many single precision primes as needed to recover the coefficients of the gcd. This is achieved by deterministically adding one new prime each time, discarding the "unlucky" ones which lead to a modular gcd of too large degree, and performing a divisibility test after each new "lucky" prime, starting with the divisibility check for the constant coefficients.

We did not try to optimize our routines and merely implemented them as described in the text, using low-level routines of NTL. The one exception is the small primes algorithm where we employed the adaptivity. By their nature, such comparisons depend on the effort spent on the various subroutines and hence contain an element of unfairness. In particular, the different types of integer and polynomial arithmetic of NTL favor some algorithms and disfavor others.

Nevertheless, the timings confirm the ranking that the theoretical bounds of Table 6.5 suggest, in particular, the efficiency of small primes modular algorithms. Moreover, they show a clear distinction between the monic and the primitive Euclidean Algorithm, which is probably caused by the absence of arithmetic with rational numbers in the latter, and between the big prime and the small primes modular algorithm, which is partly due to the adaptivity and partly reflects our intuition that it is cheaper to solve many "small" problems than one "big" problem of the same total size. In summary, an adaptive small primes modular algorithm appears to be the most favorable method to implement.

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

Notes. 6.1. When researchers started experimenting in the late 1960s with the first computer algebra systems (often built by themselves), they observed an unpleasant phenomenon: the rapid coefficient growth in the traditional Euclidean Algorithm, say in $\mathbb{Q}[x]$, as in Example 6.1 on page 185 (and almost any random example). If pseudodivision is used at each step without removing common factors, then the remainder coefficients actually grow exponentially. As long as each rational number is reduced to lowest terms, Theorem 6.52 guarantees that all coefficient lengths in the (monic) Extended Euclidean Algorithm 3.14 are polynomially bounded. This was discovered by Collins (1966, 1967), and led to several new variants of the traditional Euclidean Algorithm: using the monic versions, as discussed in Chapter 3, or the primitive versions (Section 6.12) of the remainders. See also Collins (1971), Brown (1971), Brown & Traub (1971), and von zur Gathen & Lücking (2000). Computationally, they are inferior to the modular approach, introduced by Brown (1971) and Collins (1971); see also Notes 6.7 and 6.10.

6.2. Gauß (1801) proves his important "Lemma" in article 42. He shows in article 340 of Gauß (1863b) that $\mathbb{F}_p[x]$ is a UFD, for a prime p.

6.3. Leibniz (1683) drafted a letter to Tschirnhaus, but never mailed it. He describes how to calculate the resultant of two polynomials of degree five by Euclid's Algorithm, and says that its vanishing means that the two polynomials have a nontrivial gcd. After earlier work of Newton and Maclaurin, Euler (1748c) and Bézout (1764) introduced the resultant; the name comes perhaps from Bézout's *équation resultante de l'élimination.* Bézout obtains the resultant as the determinant of a matrix, today called *Bézout's matrix*, with only $\min\{n,m\}$ rows and columns (Exercise 6.14). He describes how to calculate an $k \times k$ determinant as the familiar sum of $k!$ terms, and gives the linear equations describing the Bézout coefficients s,t with $sf+tg =$ constant, tacitly assuming the gcd to be trivial.

Later algebraic geometers generalized the resultant to more than two variables and polynomials: Euler (1764), Sylvester (1840, 1853), Cayley (1848), Macaulay (1902, 1916, 1922), and many others. The basics of the subresultant theory for univariate polynomials, as presented here, were developed by Jacobi (1836, 1846), Cayley (1848), Kronecker (1873, 1878, 1881b), and Frobenius (1881). Jacobi (1836), § 4, shows that the resultant is irreducible (as a polynomial in indeterminate coefficients of the two input polynomials) and proves our Uniqueness Lemma 5.15 in his §15. (He only considers the normal case.) He performs Euclid's algorithm with pseudodivision, and obtains the description $s_i f + t_i g = r_i$ of the remainders. Cauchy (1840) discusses various elimination methods, Euler's and Bézout's among them, proves irreducibility of the resultant, and writes down explicitly the 5×5 Sylvester matrix for two polynomials of degrees 2 and 3, respectively. Cauchy presents an early use of indices: he writes "$f = a_0 x^n + \cdots + a_n$". The lack of such a notation had made earlier work cumbersome. Kronecker (1881b) contains many of the (non-computational) results of this section, including Theorem 6.47. A "modern" presentation along these lines is in von zur Gathen (1984b), where the goal is parallel algorithms for the results of the Extended Euclidean Algorithm—not a topic of this text.

The general notion of a field emerged only towards the end of the 19th century; before that, results were proven separately for various cases. As an example, Sylvester (1881) wrote a note saying that Corollary 6.17 is also valid for polynomials with integer coefficients modulo a prime number.

Sylvester (1840) contains an explicit description of the resultant and of subresultants, as determinants of his matrix and its submatrices, and how to compute the remainders in the Euclidean Algorithm—which he calls derivation—from them. Apparently ignorant of

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

Gauß' elimination method, he calculates $n \times n$ determinants as a sum of $n!$ terms, and ends on the following hopeful note: *"Through the well-known ingenuity and kindly proferred [sic!] help of a distinguished friend, I trust to be able to get a machine made for working Sturm's theorem, and indeed all problems of derivation, after the method here expounded; on which subject I have a great deal more yet to say, than can be inferred from this or my preceding papers."* It seems that his "Euclidean engine" was never built.

6.5. Ulam invented probabilistic *Monte Carlo algorithms* to estimate the success probability in a solitaire card game; it was much used in the 1940s to approximate multidimensional integrals in high-energy physics at Los Alamos; see Metropolis & Ulam (1949), and Halton (1970) for a survey. Berlekamp's (1970) algorithm (Section 14.8) was the first to use them in computer science, and Pollard (1974, 1975) used them for two of his integer factorization methods (Sections 19.3 and 19.4), but the breakthrough came in the wake of Solovay & Strassen's (1977) primality test (Section 18.5). Babai (1979) coined the term *Las Vegas algorithm*. Gill (1977) set up the corresponding complexity classes (Section 25.8), and since then they found applications in many areas of computer science: testing (see Section 4.1), sorting, (universal) hashing, routing on networks, geometric algorithms such as triangulations or convex hulls, counting problems, online algorithms, and many more. Motwani & Raghavan (1995) give a nice survey of the numerous areas of application.

Early probabilistic algorithms are in Legendre (1785) (see Notes 14.2 and 14.3), Galois (1830) (see Notes 14.9), and Pocklington (1917), who finds a nonsquare modulo a prime by trial, but calls this *a defect in the method.* Buffon gave in 1777 a probabilistic method for approximating π by dropping a needle of length l randomly over an array of equidistant parallel lines with distance $d > l$. The probability of hitting a line is $2l/\pi d$.

6.6. The lower bound on some cyclotomic coefficients is from Vaughan (1974). Theorem 6.32 and Corollary 6.33 are due to Mignotte (1974). For a variety of bounds for factors, see Mignotte (1982, 1988), Mignotte (1989), §IV.3, and Mignotte & Glesser (1994). Since $|g_m| \leq |f_n|$, Mignotte's bound implies that $\|g\|_2 \leq \alpha \|f\|_2$ with $\alpha = 2^m$. Granville (1990) shows that one can use $\alpha = \phi^n$, where $\phi = (1+\sqrt{5})/2 \approx 1.61803\ldots$ is the golden ratio. Mignotte (1989), Proposition 4.9, proves that for all $m \in \mathbb{N}$ there exist $f, g \in \mathbb{Z}[x]$ with $\deg g = m$, $\deg f = \lfloor (m^2 \ln m)/c \rfloor$, and $g \mid f$ such that $\|f\|_1 \geq c 2^m (m^2 \ln m)^{-1/2} \|g\|_2$, for an absolute constant $c \in \mathbb{R}_{>0}$.

6.7. Modular gcd algorithms for (multivariate) polynomials over a UFD were discovered by Collins (1971) and Brown (1971). Moses & Yun (1973) state a prime power modular algorithm for multivariate polynomials based on Hensel lifting (Chapter 15); see also Lauer (2000). It is particularly useful for sparse multivariate polynomials.

6.8. Bézout stated his theorem in 1779; for conics this was already in a book by the Greek mathematician Apollonius, and it was also given by Maclaurin in 1720 and in Cramer (1750), §46. Euler (1748b, §4, 1748c, §3, 1764) states Bézout's theorem correctly: two plane curves of degrees n and m without common components meet in at most nm points. His proof uses his product representation of the resultant (Exercise 6.12). For a proof that the resultant has degree at most nm, in a similar spirit, see Exercise 6.11. General versions of Bézout's theorem for higher-dimensional varieties were first proven in the modern sense by van der Waerden in the 1920s and by Weil in 1946.

6.9. Lemma 6.44 was discovered independently by DeMillo & Lipton (1978), Zippel (1979), and Schwartz (1980). Von zur Gathen, Karpinski & Shparlinski (1996) and

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

Díaz & Kaltofen (1995), Theorem 6.2 and "Note Added in Proof", analyze Algorithm 6.45, the latter also for multivariate polynomials. A method similar to Algorithm 6.45 also works to calculate the gcd of many integers (von zur Gathen & Shparlinski (2006)). It is particularly useful for calculating the content of a polynomial in $\mathbb{Z}[x]$. Rowland & Cowles (1986), Chen & Kao (1997, 2000), Lewin & Vadhan (1998), and Moeller (1999) use algebraic field extensions to reduce the amount of randomness for zero testing of polynomials. The unsolved question of a deterministic polynomial-time zero test for polynomials given by an arithmetic circuit has an unexpected connection to other areas of complexity theory. See Shpilka & Yehudayoff (2010) for a survey of this active research area.

6.10. Subresultants were introduced into computer algebra by Collins (1966, 1967, 1973); see also Brown (1971, 1978) and Brown & Traub (1971). In fact, they work with a slightly different notion of "subresultant"; in our presentation, both definitions and theorems take a somewhat simpler form. Their "subresultants" are constant multiples of the remainders of the Euclidean Algorithm, rather than just constants. Mulders (1997) describes an error in software implementations of an integration algorithm due to a confusion about subresultants; see Section 22.3.

Subresultants were introduced by Sylvester (1840) and are treated in Trudi (1862) and Gordan's (1885) textbook, §132 ff. Habicht (1948) studies subresultants systematically in the generic case where the coefficients of the input polynomials are indeterminates. He calls our subresultants *Nebenresultanten*[1] and also gives explicit formulas for the s_i, t_i in the EEA in terms of determinants of submatrices of the Sylvester matrix. Von zur Gathen (1984b) discusses the relation between the monic and the traditional Euclidean Algorithm. The "primitive polynomial remainder sequence" discussed in Section 6.12 was introduced by Collins (1967). Two further algorithms, based on the so-called "reduced polynomial remainder sequence" and the "subresultant polynomial remainder sequence", were invented by Collins (1967) and Brown & Traub (1971). Both avoid rational arithmetic by using pseudodivision, but in contrast to the primitive Euclidean Algorithm, they do not divide out the complete content but a divisor of it which can be computed without gcd calculations. The "reduced" algorithm appears to use exponential time in the worst case (Brown 1971, page 485). This is not the case for the "subresultant" algorithm: Brown (1978) gives an estimate for its running time which is essentially the same as the bound for the primitive Euclidean Algorithm from Exercise 6.53. Lickteig & Roy (1996, 2001), Ducos (2000), and Lombardi, Roy & Safey El Din (2000) present clever variants of the "subresultant" algorithm that eliminate the factor δ from the time bound of Exercise 6.53. Von zur Gathen & Lücking (2003) give a historic overview and a systematic treatment of subresultants and polynomial remainder sequences.

Exercises.

6.1 Give a sharp estimate for $\lambda(ab)$ when $a, b \in \mathbb{Q}[x]$.

6.2 Let $a = qb + r$ be a division with remainder, with $a, b, q, r \in \mathbb{Q}[x]$, $-1 + \deg a = \deg b > \deg r$, and $\lambda(a), \lambda(b) \le l \in \mathbb{N}$. Give estimates for $\lambda(q)$ and $\lambda(r)$ in terms of l (a and b need not be monic).

6.3 Let $f \in R[x]$ for a Unique Factorization Domain R. Show that $f = \mathrm{pp}(f)$ if and only if f is primitive.

[1] minor resultants

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

6.4 Prove that Lemma 6.5 and Corollary 6.7 hold when $c \in K$ and $f, g \in K[x]$, where K is the field of fractions of R.

6.5 Let R be a UFD on which a normal form lu_R in the sense of Section 3.4 is given, and let K be the field of fractions of R. We extend cont and pp to $K[x]$ as described in Section 6.2. Prove that taking $\mathrm{lu}_{K[x]}(f) = \mathrm{lu}_R(\mathrm{lc}(\mathrm{pp}(f)))\,\mathrm{cont}(f)$ defines a normal form for $K[x]$. Hint: Exercise 3.8. Why is $\mathrm{lu}_{K[x]}(f) = \mathrm{cont}(f)$ *not* a normal form?

6.6 Let $f \in \mathbb{Z}[x]$ be monic, and $\alpha \in \mathbb{Q}$ be a root of f. Show that $\alpha \in \mathbb{Z}$.

6.7 Let p be a prime and $\varphi\colon \mathbb{Z}[x] \longrightarrow \mathbb{Z}_p[x]$ be defined by taking coefficients modulo p. Show that when $f \in \mathbb{Z}[x]$, $p \nmid \mathrm{lc}(f)$, and $\varphi(f)$ is irreducible in $\mathbb{Z}_p[x]$, then f is irreducible in $\mathbb{Q}[x]$.

6.8 Show that the probability for two random polynomials in $\mathbb{Z}[x]$ of degree at most n and max-norm at most A to be coprime in $\mathbb{Q}[x]$ is at least $1 - 1/(2A+1)$. Hint: Exercise 4.18.

6.9 Consider the ring $R = \mathbb{Z}[1/2] = \{a/2^n : a \in \mathbb{Z}, n \in \mathbb{N}\}$ of binary rationals.

(i) Prove that R is the smallest subring of $\mathbb{Q}$ containing $\mathbb{Z}$ and $1/2$.

(ii) What are the units of R?

(iii) You may use the fact that R is a UFD and that any two elements of R have a gcd which is unique up to associates. Find a normal form on R and use this to define a gcd function on R.

(iv) Determine the content and primitive part of the polynomial $f = 2x^2 + 6x - 4$ with respect to the three rings $\mathbb{Z}$, R, and $\mathbb{Q}$. Is f primitive with respect to R?

6.10 Let $f, g \in \mathbb{Z}[x]$, $r = \mathrm{res}(f,g) \in \mathbb{Z}$, and $u \in \mathbb{Z}$. Prove that $\gcd(f(u), g(u))$ divides r. Hint: Corollary 6.21.

6.11 Let F be a field and $f = \sum_{0 \le i \le n} f_i x^i$ and $g = \sum_{0 \le i \le m} g_i x^i$ in $F[x,y]$ have total degrees n and m, respectively, so that each $f_i, g_i \in F[y]$ with $\deg_y f_i \le n - i$, $\deg_y g_i \le m - i$. Let $r = \mathrm{res}_x(f,g) \in F[y]$. Show that each of the $(n+m)!$ summands contributing to r has degree at most nm, and hence $\deg_y r \le nm$.

6.12* Let R be a UFD with field of fractions F, $f, g \in R[x]$ nonzero of degrees n, m, respectively, and $\alpha_1, \ldots, \alpha_n$ and $\beta_1, \ldots, \beta_m$ the roots of f and g, respectively, in an extension field of F, counted with multiplicities.

(i) Prove:

$$\mathrm{res}(f,g) = \mathrm{lc}(f)^m \prod_{1 \le i \le n} g(\alpha_i) = (-1)^{nm}\,\mathrm{lc}(g)^n \prod_{1 \le j \le m} f(\beta_j) = \mathrm{lc}(f)^m\,\mathrm{lc}(g)^n \prod_{\substack{1 \le i \le n \\ 1 \le j \le m}} (\alpha_i - \beta_j).$$

Hint: First prove the claim in the case where the roots are considered to be indeterminates. Then apply the ring homomorphism which maps them to the actual roots.

(ii) Conclude that $\mathrm{res}(f, gh) = \mathrm{res}(f,g)\,\mathrm{res}(f,h)$ for all $f, g, h \in R[x]$.

6.13 This exercise provides an alternative proof of Corollaries 6.20 and 6.21. Let R be a UFD and $f, g \in R[x]$ nonzero of degrees n, m, respectively, where $n + m \ge 1$.

(i) Prove that

$$(x^{n+m-1}, \ldots, x, 1) \cdot \mathrm{Syl}(f,g) = (x^{m-1}f, \ldots, f, x^{n-1}g, \ldots, g) \text{ in } R[x]^{n+m},$$

and conclude that there exist nonzero $s, t \in R[x]$ with $\deg s < m$ and $\deg t < n$ such that $sf + tg = \mathrm{res}(f,g)$. Hint: Cramer's rule.

(ii) Conclude that $\mathrm{res}(f,g) = 0$ if and only if $\gcd(f,g)$ is nonconstant.

6.14* Let $f = \sum_{0 \le i \le n} f_i x^i$, $g = \sum_{0 \le i \le n} g_i x^i \in F[x]$, with a field F and $f_0 \ne 0$. For $0 \le k < n$, cross-multiply each polynomial by the leading $k+1$ terms of the other polynomial and subtract:

$$b_k = f \cdot \sum_{0 \le j \le k} g_{n-k+j} x^j - g \cdot \sum_{0 \le j \le k} f_{n-k+j} x^j = \sum_{0 \le l} b_{kl} x^l.$$

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

(i) Show that $\deg b_k < n$ for all k.

(ii) Let $B = (b_{kl})_{0\le k,l<n} \in F^{n\times n}$ be the **Bézout matrix** made up from these coefficients. Prove that $\det B = 0$ if $\gcd(f,g) \neq 1$. Hint: If $\gcd(f,g) \neq 1$, then there exists a common root of f and g in some extension field of F.

6.15* The aim of this exercise is to show that Corollaries 6.17 and 6.21 remain essentially true over arbitrary (commutative) coefficient rings R. Let $f,g \in R[x]$ nonzero of degrees n,m, respectively, such that $n+m \ge 1$ and $r = \operatorname{res}(f,g) \in R$ is nonzero.

(i) Show that there exist polynomials $s,t \in R[x]$ such that $sf+tg = \operatorname{res}(f,g)$. Hint: Apply Corollary 6.21 to the generic case where the coefficients of f and g are indeterminates, and then map them to the actual coefficients in R.

(ii) Now assume that f and g are monic. Prove that there exist polynomials $s,t \in R[x]$ with $\deg s < \deg g$ and $\deg t < \deg f$ satisfying $sf+tg = 1$ if and only if $\operatorname{res}(f,g)$ is a unit in R.

6.16* This exercise generalizes Lemma 6.13. Let F be a field, $f,g \in F[x]$ nonzero of degrees n and m, respectively, and $h = \gcd(f,g) \in F[x]$ of degree d.

(i) Prove that there exists a nonzero pair $(s,t) \in P_{m-i} \times P_{n-i}$ with $\varphi_0(s,t) = 0$ if and only if $i < d$.

(ii) Give d pairs $(s_1,t_1),\ldots,(s_d,t_d) \in P_m \times P_n$ such that $\varphi_0(s_i,t_i) = 0$ and s_i is monic of degree $m-i$ for all i.

(iii) Suppose that $(s,t) \in P_m \times P_n$ is linearly independent of all pairs from (ii) and $\varphi_0(s,t) = 0$. Show that there is a nonzero pair $(s^*,t^*) \in P_{m-d-1} \times P_{n-d-1}$ with $\varphi_0(s^*,t^*) = 0$, contradicting (i).

(iv) Conclude that $\dim \ker \operatorname{Syl}(f,g) = \deg \gcd(f,g)$.

6.17 Why are the results of `gcd(x^2+1,x+1) mod 2` and `Gcd(x^2+1,x+1) mod 2` in MAPLE different?

6.18 Let $f = x^4 - 13x^3 - 62x^2 - 78x - 408$ and $g = x^3 + 6x^2 - x - 30$ be polynomials with integral coefficients.

(i) Set up the Sylvester matrix of f and g and compute $\operatorname{res}(f,g)$.

(ii) Let $p_1 = 5$, $p_2 = 7$, $p_3 = 11$, and $p_4 = 13$. Compute $h = \gcd(f,g)$ in $\mathbb{Q}[x]$. For which of the primes is the modular image of h equal to the gcd modulo that prime, and why? Answer the latter question without actually computing the modular gcd's, then check your answer.

6.19 Let $\alpha \in \mathbb{R}$ be a parameter and $f,g_\alpha \in \mathbb{R}[x]$ monic polynomials with $\operatorname{res}(f,g_\alpha) = \alpha^3 + \alpha^2 + \alpha + 1$. Determine all values of α for which $\gcd(f,g_\alpha) \neq 1$.

6.20 Let F be a field, $f,g \in F[x,y]$ nonzero with $\deg_x f, \deg_x g \le n$, $\deg_y f, \deg_y g \le d$, and $\operatorname{lc}_x(f) = \operatorname{lc}_x(g) = 1$. Suppose that $\gcd(f(x,u),g(x,u)) \neq 1$ for at least $2nd+1$ values $u \in F$. Conclude that $\deg_x \gcd(f,g) > 0$.

6.21 Let $\mathbb{F}_q$ be a finite field with q elements. What is wrong in the following argument: Exercise 4.18 shows that the probability that two random nonconstant polynomials in $\mathbb{F}_q[x]$ are coprime is $1 - 1/q \ge 1/2$. Thus there is a probabilistic algorithm for computing gcds in $\mathbb{F}_q[x]$ which always outputs 1, independently of the inputs, and errs with probability at most $1/2$.

6.22 Let $f = x^n - 1$ and $g = \Phi_n \in \mathbb{Z}[x]$ be the nth cyclotomic polynomial, of degree $\varphi(n) \le n-1$. Compare Vaughan's lower bound on $\|\Phi_n\|_\infty$ (valid for certain, but not all, values of n) to Mignotte's upper bound 6.33.

6.23* (i) Let $f = \sum_{0\le i\le n} f_i x^i \in \mathbb{C}[x]$ of degree $n > 0$ and $\alpha \in \mathbb{C}$ a root of f. Prove that $|\alpha| \le 2b$, where $b = \max_{0\le i<n} |f_i/f_n|^{1/(n-i)}$. Hint: Start with the case $f_n = 1$, and assume that $|\alpha| > b$.

(ii) Conclude that $|\alpha| \le 2\|f\|_\infty$ if $f \in \mathbb{Z}[x]$, and compare this to Mignotte's bound for $|\alpha|$ that you obtain from Theorem 6.32.

(iii) Show that in fact $|\alpha| \le \|f\|_\infty$ if $f \in \mathbb{Z}[x]$ and $\alpha \in \mathbb{Z}$.

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

6.24* (i) Let R be a UFD and $f,g,h \in R[x]$ nonzero of degrees n,m,k, respectively, such that h divides f and g, and $f^* = f/h$ and $g^* = g/h$. Moreover, let $S = \mathrm{Syl}(f^*,g^*) \in R^{(n+m-2k)\times(n+m-2k)}$, $r = \det S = \mathrm{res}(f^*,g^*) \in R$, and

$$H = \begin{pmatrix} h_k & 0 & 0 & \cdots & 0 \\ h_{k-1} & h_k & 0 & \cdots & 0 \\ h_{k-2} & h_{k-1} & h_k & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \vdots & \vdots & \vdots & \cdots & h_k \end{pmatrix} \in R^{(n+m-2k)\times(n+m-2k)}$$

the Toeplitz matrix whose rows are shifts of the coefficient sequence $h_0,\ldots,h_k \in R$ of h. Prove that $T = HS$ is a matrix whose first $m-k$ columns are shifts of the coefficient sequence of f and whose last $n-k$ columns are shifts of the coefficient sequence of g, and show that $\det T = \mathrm{lc}(h)^{n+m-2k} r$. (In fact, $\det T$ is the kth subresultant of f and g.)

(ii) Let $R = \mathbb{Z}$, $m \le n$, and $\|f\|_\infty, \|g\|_\infty \le A$. Show that $|r| \le (n+1)^n A^{2(n-k)}$.

6.25 Complete the proof of Theorem 6.35 by showing that $p \nmid r$ holds if and only if $\mathrm{pp}(w) = h$.

6.26 Let F be a field. This exercise discusses the task to decide whether a polynomial $g \in F[x,y]$ divides another polynomial $f \in F[x,y]$, and if so, to compute the quotient $f/g \in F[x,y]$, using a modular approach. Suppose that $\deg_x g \le \deg_x f = n$ and $\deg_y g \le \deg_y f = d$, and let $p \in F[y]$ be nonconstant and coprime to $\mathrm{lc}_x(g)$.

(i) Convince yourself that $g \bmod p$ divides $f \bmod p$ in $(F[y]/\langle p\rangle)[x]$ if g divides f in $F[x,y]$.

(ii) Now assume that $g \bmod p$ divides $f \bmod p$. One might be tempted to assume that $\deg_y p > d$ is sufficient to conclude that g divides f. Prove that this is wrong in general by considering the example $f = x^n + (y^d + y^{d-1})x^{n-2}$, $g = x - y^d$, and $p = y^{d+1} + y + 1$ for $n \ge 2$ and $d \ge 1$.

(iii) Assume that $g \bmod p$ divides $f \bmod p$ and $\deg_y p > d$, and let $h \in F[x,y]$ be the modular quotient, with $\deg_x f \ge \deg_x g + \deg_x h$, $\deg_y h < \deg_y p$, and $f \equiv gh \bmod p$. Prove that $f = gh$ if $\deg_y f \ge \deg_y g + \deg_y h$. Given p, what is the running time of this method? Compute h in the example of (ii).

(iv) Find and prove analogs of (ii) and (iii) for $f,g \in \mathbb{Z}[x]$ and $p \in \mathbb{Z}$. Hint: Use Mignotte's bound 6.33 and look at the proof of Theorem 6.35.

6.27* This exercise discusses a modular gcd algorithm for $\mathbb{Z}[x]$ by Char, Geddes & Gonnet (1989) (who in fact give an algorithm for multivariate polynomials over $\mathbb{Z}$; they call it "heuristic gcd") and Schönhage (1985, 1988). The modulus is not a prime but a linear polynomial $x-u$. Let $f,g \in \mathbb{Z}[x]$ be nonzero and primitive of degree at most n and with max-norm at most A, $h = \gcd(f,g) \in \mathbb{Z}[x]$, and $u \in \mathbb{N}$ such that $u > 4A$.

(i) Prove that $h(u) \mid c = \gcd(f(u),g(u))$ in $\mathbb{Z}$ and that $h(u) \ne 0$.

(ii) Let $v \in \mathbb{Z}[x]$ whose coefficients v_i satisfy $-u/2 < v_i \le u/2$, and $v(u) = c$. Give an algorithm for computing v from c.

(iii) Now assume that $\mathrm{pp}(v) \mid f$ and $\mathrm{pp}(v) \mid g$. Writing $h = \mathrm{pp}(v)w$ with a primitive $w \in \mathbb{Z}[x]$, prove that $w(u) \mid \mathrm{cont}(v)$. Use Exercise 6.23 to show that $u/2 \ge |w(u)| \ge |\mathrm{lc}(w)| \cdot (u-2A)^{\deg w} > (u/2)^{\deg w}$ if w is nonconstant, and conclude that $h = \pm\mathrm{pp}(v)$.

(iv) Compute $\gcd(3x^4+6x^3+5x^2-2x-2, x^3+4x^2+6x+4)$ by the above method. Find out the smallest evaluation point where the method works, and compare it to your choice of u.

Schönhage (1988) proves that for $u > 4(n+1)^n A^{2n}$ the divisibility conditions assumed in (iii) are always satisfied, so that the method always terminates. He also discusses a probabilistic variant where u is chosen at random from a "small" interval and the length of u is dynamically increased (say, doubled) on failure, and proves that the expected length of a successful u is only $O^\sim(n)$, in contrast to $O^\sim(n \log A)$ for the deterministic algorithm.

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

6.28* Find an analog of Exercise 6.27 for bivariate polynomials.

6.29⟶ Compute (with explanation) the gcd over $\mathbb{Z}[x]$ of the polynomials $36x^4+72x^3+68x^2+104x+60$ and $36x^5+24x^4+116x^3+126x^2+150x+150$ with the small primes modular algorithm.

6.30⟶ (Newton 1707, page 46) Compute the gcd of $x^4-3ax^3-8a^2x^2+18a^3x-8a^4$ and $x^3-ax^2-8a^2x+6a^3$ in $\mathbb{Q}[x,a]$ using the small primes modular algorithm.

6.31* A bin contains w white balls, labeled $1,\ldots,w$, and $0<b\le w$ black balls. Consider choosing l balls without replacement, where $0<l\le 2b$, and show that the probability that at least $\lceil l/2\rceil$ balls are white is at least $1/2$. Hint: Use induction on w, starting with $w=b$, and for the induction step from w to $w+1$, distinguish the two cases whether the white ball number $w+1$ has been chosen or not.

6.32* Let $\mathbb{F}_q$ be a finite field with q elements and $f,g\in\mathbb{F}_q[x,y]$ of degree at most n in x and at most d in y.

(i) Determine the smallest value $t\in\mathbb{N}$ such that $\mathbb{F}_{q^t}$ has at least $(4n+2)d$ elements and conclude that the modular gcd algorithm 6.36 with $F=\mathbb{F}_{q^t}$ correctly computes the gcd over $\mathbb{F}_q$ (hint: Example 6.19) and takes $O(nd(n+d)\log^2(nd))$ operations in $\mathbb{F}_q$.

(ii) Let $f=x^2+y^2+xy+x+y$ and $g=x^2y+xy^2+xy$ in $\mathbb{F}_2[x,y]$. Check that $t=5$, and that $\gcd(f \bmod p, g \bmod p)\ne 1$ for the linear moduli $x,x+1,y,y+1$. Compute $\gcd(f(x,\alpha),g(x,\alpha))$ in $\mathbb{F}_4[x]$, where $\mathbb{F}_4=\mathbb{F}_2[z]/\langle z^2+z+1\rangle$ and $\alpha=z \bmod z^2+z+1\in\mathbb{F}_4$, and conclude that $\gcd(f,g)=1$.

The construction of the field extension $\mathbb{F}_{q^t}$, that is, of an irreducible polynomial of degree t over $\mathbb{F}_q$, is discussed in Section 14.9.

6.33⟶ We consider the plane curves

$$\begin{aligned} X &= \{(a,b)\in\mathbb{R}^2\colon b-a^3+7a-5=0\},\\ Y &= \{(a,b)\in\mathbb{R}^2\colon 20a^2-5ab-4b^2+35a+35b-21=0\} \end{aligned}$$

in $\mathbb{R}^2$. Determine the intersection of X and Y in two ways: by projecting it to the first coordinate, and by projecting it to the second coordinate. Comment on the differences. Plot the two curves and mark their intersection points.

6.34 Compute the minimal polynomial $f\in\mathbb{Q}[x]$ of $\sqrt{2}+\sqrt{3}$ over $\mathbb{Q}$. Let $\mathbb{F}_{19^2}=\mathbb{F}_{19}[z]/\langle z^2-2\rangle$ and $\alpha=z \bmod x^2-2\in\mathbb{F}_{19^2}$ a square root of 2. Check that 7α is a square root of 3, and compute the minimal polynomial of $\alpha+7\alpha$ over $\mathbb{F}_{19}$. How is it related to f?

6.35* Let α,β be two nonzero algebraic numbers, with (monic) minimal polynomials $f,g\in\mathbb{Q}[x]$ of degrees n,m, respectively.

(i) Prove that the reversal $\mathrm{rev}(f)=x^nf(x^{-1})$ of f is the minimal polynomial of α^{-1}.

(ii) Let $r=\mathrm{res}_y(\mathrm{rev}(f)(y),g(xy))\in\mathbb{Q}[x]$. Show that $\deg_x r=nm$ and $r(\alpha\beta)=0$ (hint: Exercise 6.12), and conclude that r is the minimal polynomial of $\alpha\beta$ if it is irreducible.

(iii) Find multiples of degree nm of the minimal polynomials of $a\alpha+b\beta$, where $a,b\in\mathbb{Q}\setminus\{0\}$ are arbitrary, and of α/β.

(iv) Compute the minimal polynomials of $\sqrt{2}-2\sqrt{3}$ over $\mathbb{Q}$ and of $\sqrt{2}\sqrt[3]{3}$ over $\mathbb{Q}$ and over $\mathbb{F}_{13}$.

6.36* Let α be an algebraic number and $f,g\in\mathbb{Q}[x]$ of degrees $n,m\in\mathbb{N}_{\ge 1}$ such that f is the minimal polynomial of α. We want to compute the minimal polynomial of $g(\alpha)$ and therefore may assume that $n>m$.

(i) Let $r=\mathrm{res}_y(f(y),x-g(y))\in\mathbb{Q}[x]$. Show that $\deg_x r=n$ and that the minimal polynomial of $g(\alpha)$ divides r (hint: Exercise 6.12). (In fact, r is a power of the minimal polynomial of $g(\alpha)$, which equals $r/\gcd(r,r')$.)

(ii) Compute the minimal polynomials of $\sqrt{3}+1$ and $2^{2/3}+2^{1/3}+1$ over $\mathbb{Q}$.

A different algorithm is given in Exercise 12.10.

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

6.37 This exercise discusses a variant of Lemma 6.44, due to Zippel (1993). Let R be an integral domain, $n \in \mathbb{N}$, $S \subseteq R$ finite with $s = \#S$ elements, and $r \in R[x_1, \ldots, x_n]$ a polynomial of degree at most $d_i \leq s$ in the variable x_i.

(i) Show that r has at most $s^n - (s-d_1)\cdots(s-d_n) \leq (d_1 + \cdots + d_n)s^{n-1}$ zeroes in S^n if it is not the zero polynomial. Hint: Prove inductively that the number of elements of S that are not zeroes of r is at least $(s-d_1)\cdots(s-d_n)$.

(ii) Let $u_1, \ldots, u_s$ be the elements of S. Prove that the polynomial

$$r = \prod_{1 \leq i \leq n} \prod_{1 \leq j \leq d_i} (x_i - u_j)$$

achieves the first bound from (i) exactly.

6.38 Modify Algorithm 6.45 so as to also produce $s_1, \ldots, s_n \in F[x]$ of degree less than d such that $s_1 f_1 + \cdots + s_n f_n = h$, taking at most $3(n-2)(d+1) + O(d^2)$ operations in F.

6.39 Let R be a Unique Factorization Domain, $f_1, \ldots, f_n \in R$, $m = f_1 \cdots f_n$, and $g_i = m/f_i$ for $1 \leq i \leq n$. Show that $\operatorname{lcm}(f_1, \ldots, f_n) = m/\gcd(g_1, \ldots, g_n)$. Derive a probabilistic algorithm for computing $\operatorname{lcm}(f_1, \ldots, f_n)$ if $R = F[x]$ for a field F, and analyze its cost and success probability.

6.40 For each $n \in \mathbb{N}$, find polynomials $f_1, \ldots, f_n \in \mathbb{Q}[x]$ such that $\gcd(f_1, \ldots, f_n) = 1$ and any proper subset of them has a nonconstant gcd.

6.41 Let R be a UFD and $f, g \in R[x]$ nonzero with $\deg g \leq \deg f$. Prove that for $0 \leq k < \deg g$, $\gcd(\operatorname{lc}(f), \operatorname{lc}(g))$ divides the kth subresultant of f and g.

6.42$^{\longrightarrow}$ Let $f = x^8 + x^6 - 3x^4 - 3x^3 + 8x^2 + 2x - 5$ and $g = 3x^6 + 5x^4 - 4x^2 - 9x + 21$ be polynomials in $\mathbb{Z}[x]$ and $S_k \in \mathbb{Z}^{(14-2k)\times(14-2k)}$ be the submatrix of the Sylvester matrix of f and g whose determinant is the kth subresultant σ_k, as in Section 6.10, for $0 \leq k \leq 6$.

(i) Trace the Extended Euclidean Algorithm 3.14 for f and g over $\mathbb{Q}$, and check that $s_i f + t_i g = r_i$ for $2 \leq i \leq \ell$. Compute the degree sequence $n_0 = 8, n_1 = 6, n_2, \ldots, n_\ell$.

(ii) Set up the matrices S_k and compute σ_k for $0 \leq k \leq 6$. Explain which of the S_k are singular and what this has to do with the remainders in the EEA.

(iii) Let $2 \leq i \leq \ell$ and $u_{n_i} \in \mathbb{Q}^{14-2n_i}$ such that its last entry is 1 and all other coefficients are zero. Say why the linear system $S_{n_i} v = u_{n_i}$ has a unique solution $v \in \mathbb{Q}^{14-2n_i}$ and state the coefficients of the solution *without calculation*. Then check your answer by actually computing $S_{n_i} v$.

(iv) For those $k \in \{0, \ldots, 5\}$ where $\sigma_k = 0$, find a nonzero vector $v \in \mathbb{Q}^{14-2k}$ such that $S_k v = 0$ *without calculation*. Then check your answer by actually computing $S_k v$.

6.43 Let F be a field, $m, n \in \mathbb{N}$ with $m < n$, and $f = \sum_{0 \leq i \leq m} f_i x^i \in F[x]$ of degree m. For $0 \leq h \leq m$, we define square matrices $A_h = (f_{h-i+j})_{0 \leq i,j < n-h} \in F^{(n-h)\times(n-h)}$ and $B_h = (f_{h-i+j})_{0 \leq i,j < n-h-1} \in F^{(n-h-1)\times(n-h-1)}$, so that B_h is the leading principal submatrix of A_h of order $n-h-1$.

(i) Prove that the hth subresultant of x^n and f is $\sigma_h = \det A_h$, for $0 \leq h \leq m$.

(ii) Let $0 \leq h \leq m$ be such that $\sigma_h \neq 0$ and $r, s, t \in F[x]$ the row in the EEA for (x^n, f) that has $\deg r = h$. Prove that the constant coefficient of t is $\det B_h / \det A_h$. Hint: Theorem 6.48 and Cramer's rule.

(iii) Let $0 \leq h < k \leq m$ be such that h is maximal with $\sigma_h \neq 0$. Conclude from (ii) that there exists a $(k, n-k)$-Padé approximant to f if and only if $\det B_h \neq 0$. Hint: Corollary 5.21.

6.44* You are to estimate the coefficient size of quotient and remainder in a pseudodivision and its cost. Let $a, b \in \mathbb{Z}[x]$, with $n = \deg a \geq m = \deg b > 0$, $k = n - m$, and $c = \operatorname{lc}(b) \in \mathbb{Z}$. Furthermore, let $q = \sum_{0 \leq i \leq k} q_i x^i$ and r in $\mathbb{Z}[x]$ be such that $a^* = c^{k+1} a = qb + r$ and $\deg r < m$, and $r_i \in \mathbb{Z}[x]$ be the remainder in the ith iteration of the polynomial division algorithm 2.5 applied to $r_{k+1} = a^*$ and b, for $k \geq i \geq 0$ (see also Exercise 2.9).

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

(i) Let $\|a\|_\infty \le A$, $\|b\|_\infty \le B$, and $|c| \le C$. Prove that we have $|q_i| \le A(B+C)^{k-i}C^i$ and $\|r_i\|_\infty \le A(B+C)^{k+1-i}C^i$ for $k \ge i \ge 0$, and $\|r\|_\infty \le A(B+C)^{k+1}$.

(ii) Conclude that the cost for computing q, r from a, b is $O(mk^2 \log^2 B)$ word operations if $A \le B$.

(iii) Let $A, B, C \in \mathbb{N}_{>0}$, and assume that a has positive coefficients not smaller than A, and that $b = Cx^m - b^*$ for a polynomial $b^* \in \mathbb{Z}[x]$ with $\deg b^* < m$ and positive coefficients greater or equal to B. Show that $q_i \ge A(B+C)^{k-i}C^i$ for $k \ge i \ge 0$ and $\|r\|_\infty \ge A(B+C)^{k+1}$. (Thus the bound from (i) is essentially sharp.)

(iv) Give statements analogous to (i) and (ii) for pseudodivision of bivariate polynomials.

6.45 Let F be a field, $f, g \in F[x]$ nonzero of degree $n \ge m$, $r_i, s_i, t_i \in F[x]$ the ith row in the EEA for f, g and some $i \ge 1$, $n_i = \deg r_i$, S the n_ith submatrix of the Sylvester matrix of f and g, and $\sigma = \det S$ the n_ith subresultant.

(i) We have shown in Theorem 6.52 that the coefficients of σs_i and σt_i are determinants of submatrices of S of order $n+m-2n_i-1$. Let U, V be the matrices that arise from S by replacing the last row by $(x^{m-n_i-1}, \ldots, x, 1, 0, \ldots, 0)$ and $(0, \ldots, 0, x^{n-n_i-1}, \ldots, x, 1)$, respectively. Prove that $\sigma s_i = \det U$ and $\sigma t_i = \det V$.

(ii) Let W be the matrix S with its last row replaced by $(x^{m-n_i-1}f, \ldots, xf, f, x^{n-n_i-1}g, \ldots, xg, g)$. Conclude from (i) that $\sigma r_i = \det W$.

(iii) Prove that every coefficient of σr_i has absolute value at most $(n+1)^{n-n_i}A^{n+m-2n_i}$ if f, g are in $\mathbb{Z}[x]$ with max-norms at most A. Hint: The coefficient of x^j in σr_i is obtained by taking only terms containing x^j in the last row of W.

6.46* Let $\mathbb{F}_q$ be a finite field with q elements and $n, m \in \mathbb{Z}$ with $n \ge m \ge 0$.

(i) Let $X_0, \ldots, X_{m-1}$ be independent random variables with $\operatorname{prob}(X_i = 0) = q^{-1}$ for all i, and ρ the random variable that counts the longest run of zeroes in $X_0, \ldots, X_{m-1}$:

$$\rho = \max\{0 \le i \le m : \exists j \le m - i \quad X_j = X_{j+1} = \cdots = X_{j+i-1} = 0\}.$$

Prove that $\operatorname{prob}(\rho \ge d) \le (m-d+1)q^{-d}$ for $1 \le d \le m$, and conclude that the expected value

$$\mathcal{E}(\rho) = \sum_{0 \le d \le m} d \operatorname{prob}(\rho = d) = \sum_{0 \le d \le m} \operatorname{prob}(\rho \ge d)$$

of ρ is at most $1 + m/(q-1)$. (In fact, the better bound $\mathcal{E}(\rho) \in O(\log m)$ holds; see Guibas & Odlyzko (1980) for a proof and references.)

(ii) For two uniform random polynomials $f, g \in \mathbb{F}_q[x]$ of degrees n and m, respectively, let δ denote the maximal degree difference of two consecutive remainders in the Euclidean Algorithm for f, g, with $\delta(f, g) = 0$ if $g \mid f$ (the first difference $\deg f - \deg g$ is not counted). Use Exercise 4.17 to conclude that $\mathcal{E}(\delta) \in O(\log m)$.

(iii) Let $A \ge 1$. Derive the same upper bound on the expected value of δ for random $f, g \in \mathbb{Z}[x]$ of degrees n and m, respectively, and with $\|f\|_\infty, \|g\|_\infty \le A$.

6.47* Prove Theorem 6.53.

6.48* Prove Theorem 6.54.

6.49* The aim of this exercise, which follows Shoup (1991), is to prove a bound on the coefficients of the results r_i^*, s_i^*, t_i^* of the traditional EEA for two nonzero elements f, g of $\mathbb{Z}[x]$ or $F[x, y]$, where F is a field, that is independent of the value δ from Theorems 6.53 and 6.54. So let $\alpha_i = \operatorname{lc}(r_i^*)$ and $n_i = \deg r_i^*$ for $0 \le i \le \ell$, σ_k the kth subresultant, and S_k the submatrix of the Sylvester matrix $\operatorname{Syl}(f, g)$ whose determinant is σ_k, for $0 \le k \le n_1$, as usual.

(i) Let κ_i, λ_i be the constant coefficients of $s_i = \alpha_i^{-1} s_i^*$ and $t_i = \alpha_i^{-1} t_i^*$, respectively, for $2 \le i \le \ell$. By Theorem 6.48 and Cramer's rule, we have

$$\kappa_i = \frac{\det Y_i}{\sigma_{n_i}}, \quad \lambda_i = \frac{\det Z_i}{\sigma_{n_i}} \quad \text{for } 2 \le i \le \ell,$$

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

where Y_i, Z_i are matrices that result from S_{n_i} by replacing a certain column by a unit vector. Let

$$\gamma_2 = \det Y_2, \quad \gamma_i = \det Y_{i-1} \cdot \det Z_i - \det Z_{i-1} \cdot \det Y_i \quad \text{for } 3 \le i \le \ell,$$

and prove that

$$\alpha_2 = \frac{\sigma_{n_2}}{\gamma_2}, \quad \alpha_i = \frac{(-1)^{i-1}\sigma_{n_i}\sigma_{n_{i-1}}}{\gamma_i \alpha_{i-1}} \text{ for } 3 \le i \le \ell.$$

Hint: Lemma 3.15 and Theorem 6.53. Conclude that

$$\alpha_i = (-1)^{(i+1)(i+2)/2}\sigma_{n_i} \prod_{2 \le j \le i} \gamma_j^{(-1)^{i+j-1}} \text{ for } 2 \le i \le \ell.$$

(ii) Let $f, g \in \mathbb{Z}[x]$ with max-norm at most A, $n = n_0 \ge m = n_1$, $B = (n+1)^n A^{n+m}$, as in Theorem 6.52, and $2 \le i \le \ell$. Prove that both the numerator and the denominator of α_i are absolutely at most $(2B)^i$. Using the bounds from Theorem 6.52 on the coefficients of the results $r_i = \alpha_i^{-1} r_i^*, s_i, t_i$ of the monic EEA, show that the numerators and denominators of r_i^*, s_i^*, t_i^* are absolutely bounded by $(2B)^{i+1}$. Conclude that their length is $O(nm\log(nA))$.

(iii) Let $f, g \in F[x,y]$ of degrees at most d in y, $n = n_0 \ge m = n_1$, and $2 \le i \le \ell$. Prove that the numerator and denominator of α_i has degree in y at most $i(n+m)d$. Using the bounds from Theorem 6.54 on the coefficients of the results r_i, s_i, t_i of the monic EEA, conclude that the numerators and denominators of r_i^*, s_i^*, t_i^* have degree at most $(i+1)(n+m)d$.

6.50* Prove Theorem 6.60 and try to find a "small" constant $c \in \mathbb{Q}$ such that the running time is at most $cn^3md^2 + O(n^3d(n+d))$ arithmetic operations in F in the normal case where $\deg r_{i+1} = \deg r_i - 1$ for $1 \le i < \ell = m+1$.

6.51* Design big prime modular algorithms for the monic EEA in $\mathbb{Z}[x]$ and $F[x,y]$, where F is a field. Prove that your algorithms are correct and on input f, g of degree $n \ge m$ in x take $O(n^3 m \log^2(nA))$ word operations if $\|f\|_\infty, \|g\|_\infty \le A$ and $O(n^3md^2)$ field operations if $\deg_y f, \deg_y g \le d$, respectively. You may ignore the cost for finding a big prime or irreducible polynomial.

6.52 Let f, g be primitive polynomials in $R[x]$, where R is a UFD, and let $\operatorname{prem}(f,g)$ be the remainder in the pseudodivision of f by g. Show that $\gcd(f,g) = \gcd(g, \operatorname{prem}(f,g))$.

6.53* (i) Prove the running time estimate of Theorem 6.62 for the primitive Euclidean Algorithm 6.61 for bivariate polynomials.

(ii) Modify the primitive Euclidean Algorithm 6.61 so as to also compute $s_i, t_i \in R[x]$ with $r_i = s_i f + t_i g$ for $0 \le i \le \ell$, and prove that the time estimate from Theorem 6.62 is still valid for the modified algorithm.

(iii) Use Exercise 6.44 to show that Algorithm 6.61 can be executed using only $O(n^3 m\delta \log^2(nA))$ word operations for $R = \mathbb{Z}$ and only $O(n^3 m\delta d^2)$ operations in F when $R = F[y]$ for a field F.

6.54* Let F be a field and $f, g \in F[x,y]$ with $\deg_x f = n \ge \deg_x g = m$ and $\deg_y f, \deg_y g \le d$. Assuming that all coefficients of the remainders r_i in the primitive Euclidean Algorithm 6.61 for f, g achieve the degree bounds from Theorem 6.54 on the numerators of the corresponding coefficients in the monic Euclidean Algorithm exactly, and also the degree bounds for the pseudodivision of r_{i-1} by r_i according to Exercise 6.44 are in fact equalities, prove that $\deg_y \operatorname{cont}_x(a_{i-1} \operatorname{rem} r_i) = (n_{i-1} - n_i + 1)(n+m-2n_i)d - 2(n_{i-1} - n_{i+1})d$.

6.55⟶ You are to compare five methods for computing gcds in $\mathbb{Z}[x]$ in your favorite computer algebra system. The first method is simply to call its appropriate routine. Try to find out which algorithm it uses. The other methods are the primitive Euclidean Algorithm 6.61, the big primes modular algorithm 6.34, the small primes modular algorithm 6.38, and the "heuristic" algorithm from Exercise 6.27. Implement and run your algorithms on several examples. Choose examples of increasing complexity; construct examples so that the resulting gcds have several different degrees, including cases where the gcd is 1. Measure running times and see if you can draw any conclusions about relative merits of the algorithms.

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

6.56 Let F be a field and $n \in \mathbb{N}_{\geq 2}$.

(i) Determine the discriminant $\operatorname{disc}(f) = \operatorname{res}(f, f')$ of a binomial $f = x^n + b$, with $b \in F^\times$.

(ii) (Swan 1962) Determine the discriminant of a trinomial $f = x^n + ax^k + b$, with $1 \leq k < n$ and $a, b \in F^\times$.

6.57 Derive Lemma 3.15 for $R = F[x]$, where F is a field, from Lemmas 3.8 and 3.10 and Theorem 6.53 (i).

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press

I would have my son mind and understand Business,
read a little History, study the Mathematics and Cosmography:
—these are good, with subordination to the things of God.
[. . .] These fit for Public services, for which man is born.
Oliver Cromwell (1649)

They that are ignorant of Algebra cannot imagine
the wonders in this kind are to be done by it: and what
further improvements and helps advantageous to other parts
of knowledge the sagacious mind of man may yet find out,
it is not easy to determine.
John Locke (1690)

What kind of world do we live in where 11 and 7 equal 2?
John Cougar Mellencamp (1989)

Scientists pretended that history didn't matter,
because the errors of the past were now corrected
by modern discoveries. But of course their forebears
had believed exactly the same thing in the past, too.
Michael Crichton (1995)

Bei dem Kinde aber muss man im Unterrichte allmälig
das Wissen und Können zu verbinden suchen. Unter allen
Wissenschaften scheint die Mathematik die einzige der Art zu seyn,
die diesen Endzweck am besten befriedigt.[1]
Immanuel Kant (1803)

[1] The instruction of children should aim gradually to combine knowing and doing. Among all sciences mathematics seems to be the only one that satisfies this purpose best.

https://doi.org/10.1017/CBO9781139856065.008 Published online by Cambridge University Press